

Proyecto Final

Predicción de comportamiento sedentario a partir del análisis de datos de
lifelogging



Universidad Nacional del Centro de la Provincia de Buenos Aires

Facultad de Ciencias Exactas

Ingeniería de Sistemas

Alumno: Santillán Cooper, Martín

Directores

Armentano, Marcelo
Tommasel, Antonela

Índice de contenidos

Capítulo 1: Introducción	6
Motivación	7
Objetivos	9
Solución propuesta	9
Capítulo 2: Marco Teórico	11
Inteligencia Artificial	12
Machine Learning	13
Aprendizaje Supervisado y No Supervisado	13
Tipos de tareas	14
Entrenamiento de los modelos	16
Evaluación de los modelos	18
Generalización, underfitting y overfitting	19
Regularización	20
Métricas de evaluación	22
Métricas de regresión	22
Deep Learning	24
Hiperparámetros	24
Modelos de Deep Learning utilizados	26
Perceptrón Multicapa	26
Redes Neuronales Recurrentes	29
Redes Neuronales Convolucionales	31
Redes Temporales Convolucionales	32
Predicción de actividades	34
Capítulo 3: Trabajos relacionados	35
Monitoreo y modelado del comportamiento humano utilizando smartphones	36
Predicción del Comportamiento Sedentario Futuro	39
Capítulo 4: Predicción del Comportamiento Sedentario Futuro	44
Enfoque	45

Descripción del dataset	46
Tipos de datos del dataset StudentLife	46
Datos no disponibles	47
Datos del acelerómetro	47
Datos de sensado	48
Implementación (Evaluación Experimental)	50
Análisis de datos	50
Tamaño de buckets	50
Disponibilidad de los datos	52
Registros desconocidos	56
Depuración de buckets	58
Nivel de MET	59
Análisis del nivel de MET	61
Inconsistencias encontradas	63
Procesamiento del dataset	65
Selección y cómputo de características	66
Características Discretas	67
Características por intervalo	71
Variable objetivo	72
Eliminación de valores nulos	72
Tratamiento de variables categóricas	73
Generación de datasets con lags	73
Separación entre X e y	76
Separación entre entrenamiento y testeo	76
Normalización	77
Ajuste de las dimensiones de X	77
Validación de los modelos	78
Métricas de evaluación	81
Tuning de los modelos	81
Optimización de Bayes	81
Selección de experimentos modelo	82
Selección de usuarios modelo	83
Diseño de los experimentos	84
Capítulo 5: Resultados	87
Resultados del proceso de tuning	88
MLP	88
CNN	89
TCN	89

RNN	90
Resultados de los experimentos	91
Comparación entre las redes neuronales a partir del MSE	93
Modelos Personales vs. Modelos Impersonales	96
Comparación entre las redes neuronales a partir del tiempo requerido para ser entrenadas	98
Resultados detallados de los usuarios modelo	100
Capítulo 6: Discusión	103
Impacto de los usuarios modelos	104
Modelos personales e impersonales	105
Desempeño de las diferentes redes neuronales	109
Tiempo de entrenamiento a lo largo de las iteraciones	112
Número de lags	114
Granularidad	115
Exploding gradient	116
Capítulo 7: Conclusión	119
Resumen	120
Limitaciones	121
Trabajos futuros	121
Bibliografía	123

Capítulo 1: Introducción

Este capítulo está dividido en 3 secciones. En la [Sección 1.1](#) se presentan el problema que se aborda en esta tesis y qué hechos motivan su abordaje. En la [Sección 1.2](#) se enuncia el objetivo principal de la tesis, como así también las actividades que requieren ser llevadas a cabo para cumplir dicho objetivo. Por último, en la [Sección 1.3](#) se expone la solución propuesta.

1.1. Motivación

La definición de comportamiento sedentario ha ido evolucionando a lo largo de los años, al mismo tiempo que lo hizo la forma de medirlo. La Red de investigación del comportamiento sedentario¹ define el comportamiento sedentario como cualquier comportamiento de vigilia caracterizado por un gasto de energía $\leq 1,5$ MET (equivalente metabólico de tareas) mientras se está sentado o reclinado (Sedentary Behaviour Research Network, 2012). MET mide la intensidad de una actividad en múltiplos del gasto energético en reposo. Ejemplos de actividades sedentarias son mirar televisión (1,0 MET), comer sentado (1,5 MET), jugar videojuegos (1,0 MET) y conducir (1,3 MET).

La investigación realizada en el área demuestra asociaciones sólidas y consistentes entre el tiempo sedentario y la diabetes, las enfermedades cardiovasculares y la mortalidad en general (Wilmot et al., 2012) (Carter et al., 2017). Sin embargo, las asociaciones informadas fueron en gran medida independientes de la actividad física. Por lo tanto, es importante tener en cuenta que el comportamiento sedentario no representa lo contrario de la actividad física y que es posible que un individuo tenga simultáneamente niveles altos de actividad física moderada a vigorosa (MVPA, por sus siglas en inglés) y comportamiento sedentario. En general, se ha demostrado que el tiempo en el que un individuo se encuentra en estado sedentario está asociado de forma perjudicial con la salud y con marcadores de riesgo metabólico en diversos grupos poblacionales. Además, se ha destacado la importancia de no solo estimular la MVPA, sino también de reducir el tiempo de sedentarismo, ya que la conducta sedentaria es un factor de riesgo para la mortalidad, independiente de la MVPA (Koster et al., 2012).

Otros trabajos se han centrado en las asociaciones entre los descansos breves en el tiempo sedentario con los resultados metabólicos (Paing et al. 2018) y con la optimización de las operaciones cognitivas (Felez-Nobrega et al., 2018; Magnon et al., 2018; Paing et al., 2018). (Benatti & Ried-Larsen, 2015) afirmaron que existe evidencia suficiente para demostrar los efectos positivos en el metabolismo de reducir los tiempos de conductas sedentarias.

Los métodos utilizados para medir la conducta sedentaria se pueden clasificar en métodos subjetivos (cuestionarios y diarios de autoevaluación) y métodos objetivos (sensores comunes de dispositivos ubicuos). Los métodos subjetivos están siendo superados por las nuevas tecnologías que pueden proporcionar, para todos los grupos de la población, información segundo a segundo sobre la postura, el movimiento (o su falta), y los patrones dentro y entre días (Atkin et al., 2012).

El área de *mobile health* (*mHealth*) ha crecido mucho en los últimos años gracias a la miniaturización de los dispositivos móviles, su poder de cómputo y su conectividad. *MHealth* puede definirse como “el uso de comunicaciones móviles y tecnologías de red emergentes para el cuidado de la salud”. *MHealth* tiene la capacidad de beneficiar la comunicación entre paciente y entidades de la salud, proveer servicios de salud y promover comportamientos preventivos de la salud (Dutta et al., 2018). De esta forma, existe un gran potencial para que las tecnologías de *mHealth* rediseñen casi todas las facetas de la atención médica (Steinhubl et al., 2015). En particular, aplicaciones de *mHealth* han demostrado tener el potencial de promover cambios en el comportamiento sedentario y la actividad física (Yerrakalva et al., 2019).

¹ <https://www.sedentarybehaviour.org/>

Si bien los dispositivos móviles pueden considerarse como una de las causas del comportamiento sedentario (He & Agu, 2016b), también ofrecen nuevas oportunidades para prevenirlo. Hoy en día, los dispositivos móviles portátiles, como teléfonos inteligentes, relojes inteligentes y rastreadores de ejercicios están equipados con una amplia variedad de sensores que se pueden usar para registrar la actividad humana y llevar a cabo análisis de comportamiento a través de una técnica conocida como *lifelogging*. Se define a *lifelogging* como una forma pervasiva de realizar cómputos, que consiste en un registro digital unificado de la totalidad de las experiencias de un usuario, capturado a partir de sensores multi-modales digitales almacenados como un archivo personal multimedia. Estos sensores permiten a los *smartphones* capturar información sobre nosotros y sobre nuestros contextos y entornos de manera no intrusiva y continua (Ali et al., 2014). El uso de métodos objetivos para evaluar el comportamiento sedentario está creciendo a medida que los costos de los dispositivos móviles portátiles disminuyen y son más fáciles de usar. En febrero de 2019, el 96% de las personas en los EE. UU. poseía un teléfono, de los cuales el 81% eran teléfonos inteligentes, también llamados *smartphones*. Los porcentajes ascienden a 99% y 96% respectivamente en la población de entre 18 a 49 años². En este contexto, estos dispositivos pueden verse como una oportunidad para desarrollar métodos objetivos complejos para medir y predecir el comportamiento sedentario.

1.2. Objetivos

Se han implementado muchas aplicaciones para teléfonos inteligentes con el objetivo de alertar al usuario cuando se reconoce un comportamiento sedentario (Fahim et al., 2017; Grundgeiger et al., 2017; He & Agu, 2014). Predecir conductas sedentarias futuras puede ayudar a habilitar intervenciones preventivas, como recordatorios y sugerencias para diferentes actividades basadas en la Teoría de la conducta planificada (TPB) (Ajzen, 1991). TPB postula que es más probable que un sujeto participe en las intervenciones recomendadas para reducir las conductas sedentarias si tales actividades están incluidas en sus planes. Siguiendo esta idea, la hipótesis que motiva esta tesis es que *si se pudiese predecir en el momento t que un sujeto será sedentario en el tiempo $t + 1$, se podrían recomendar actividades a modo de intervenciones con la finalidad de cambiar su rutina sedentaria a largo plazo*. Este tipo de intervenciones puede dar lugar a mejores oportunidades para cambiar el comportamiento de los sujetos y así tener estilos de vida más saludables.

El objetivo principal de esta tesis es:

- Predecir el comportamiento sedentario futuro mediante modelos de *Deep Learning* entrenados con datos de *lifelogging*.

Para lograr el anterior objetivo principal, se llevarán a cabo las siguientes actividades:

- El análisis de datos de *lifelogging* para la definición de perfiles dinámicos de usuario capaces de capturar y representar el comportamiento del usuario, basado en fuentes de datos heterogéneas.
- La identificación de *datasets* apropiados para llevar a cabo el objetivo anterior.

² <https://www.pewresearch.org/internet/fact-sheet/mobile/>

- La utilización de los perfiles definidos para identificar patrones de comportamiento sedentario.
- El estudio, análisis y comparación de algoritmos de *Deep Learning* para la tarea de Predicción del Comportamiento Sedentario Futuro.
- La proposición de diferentes marcos donde los modelos propuestos pueden ser utilizados.

1.3. Solución propuesta

El problema de predecir el comportamiento sedentario futuro se ha abordado previamente analizando solo el tiempo inactivo/estacionario de un sujeto y se ha medido y comparado el rendimiento de varios modelos. Alternativamente, en esta tesis se define la Predicción del Comportamiento Sedentario Futuro (PCSF) como la tarea de predecir si la actividad física de un usuario superará o no, en promedio, 1,5 MET en un futuro próximo. Aunque MET es una métrica estándar en el área de la salud para medir la intensidad de una actividad en términos del gasto energético, el uso de esta para predecir el comportamiento sedentario mediante el uso de dispositivos portátiles y dispositivos móviles permanece inexplorada.

En este trabajo, se presenta un enfoque para predecir el comportamiento sedentario futuro de un sujeto en términos de su nivel de MET a partir de diferentes modelos de Deep Learning, basado en la observación de valores obtenidos de múltiples sensores de dispositivos portátiles y móviles.

El resto de la tesis está organizado de la siguiente manera. Primero, en el Capítulo 2 se expone el Marco Conceptual, en el cual se explican los conceptos más importantes requeridos para comprender lo desarrollado en esta tesis. Luego, en el Capítulo 3 se presentan los trabajos relacionados, es decir, aquellos cuyo contenido ha servido de referencia para el desarrollo de esta tesis. Más adelante, en el Capítulo 4 se aborda el enfoque que se le da al problema de la Predicción del Comportamiento Sedentario Futuro, así como también se desarrolla cuál fue el modo de implementación. Después, en el Capítulo 5 se muestran y comentan los principales resultados arrojados por los experimentos llevados a cabo. Seguidamente, en el Capítulo 6 se discuten los resultados obtenidos. Finalmente, en el Capítulo 7 se realiza la conclusión del presente trabajo de tesis.

Capítulo 2: Marco Teórico

En este capítulo se presentan y desarrollan los conceptos más importantes para la comprensión de lo desarrollado en el presente trabajo de tesis. La mayor parte del presente capítulo está dedicada a las áreas concernientes de la inteligencia artificial. En la [Sección 2.1](#) se define brevemente el área de la inteligencia artificial. Luego, en la [Sección 2.2](#) se desarrollan los conceptos más importantes relacionados al área de *Machine Learning*, como los tipos de tareas, el marco teórico en el cual se basa esta área, el modo de entrenamiento y validación de los modelos, entre otros. Seguidamente, en la [Sección 2.3](#) se desarrollan los conceptos específicos que añade el área de *Deep Learning* a la de *Machine Learning*, como los diferentes tipos de redes neuronales e hiperparámetros. Finalmente, en la [Sección 2.4](#) se introduce el concepto de Predicción de Actividades, donde la tarea de la PCSF pertenece.

2.1. Inteligencia Artificial

El objetivo principal de esta tesis es evaluar el uso de modelos de *Deep Learning* para el problema de Predicción del Comportamiento Sedentario Futuro a través de datos de *lifelogging*. Por lo tanto, es importante dar una base a los conceptos más importantes en ambas áreas. Es debido notar que tanto *Deep Learning* como *Machine Learning* pertenecen al campo de Inteligencia Artificial y que a su vez *Deep Learning* pertenece al campo de *Machine Learning*. Por lo tanto, a continuación, se presentan los conceptos más importantes de *Machine Learning*, que servirán para comprender el proceso llevado a cabo para generar los modelos. Luego, se presentan los conceptos más importantes de *Deep Learning*, que son una extensión de los presentados en la sección de *Machine Learning*.

(Mitchell, 1997) define que un programa de computadora aprende a partir de una experiencia E con respecto a una tarea T y un determinado desempeño medido por D , si su desempeño en la tarea T , mejora con la experiencia E . Esta definición puede aplicarse a la totalidad del área de la Inteligencia Artificial. Los primeros programas de Inteligencia Artificial en tener éxito fueron aquellos basados en conocimiento o *knowledge based*. En estos, se definía el conocimiento al que tenía acceso el programa a partir de lenguajes formales. Luego, el programa de Inteligencia Artificial razonaba a partir de reglas de inferencia lógica. Estos primeros programas consiguieron resolver problemas que son difíciles de resolver para seres humanos, como jugar al ajedrez. Irónicamente, los problemas más difíciles de resolver para un programa de inteligencia artificial son aquellos que los seres humanos llevan a cabo sin ninguna dificultad en la vida diaria, como reconocer objetos. Esto se da debido a que el conocimiento utilizado para llevar a cabo estas tareas es subjetivo e intuitivo, y, por lo tanto, difícil de articular de manera formal. Esto sugiere que los sistemas de Inteligencia Artificial deben ser capaces de adquirir su propio conocimiento, extrayendo patrones de los datos. Esta capacidad es conocida como *Machine Learning*. Por lo tanto, en el área de *Machine Learning*, el algoritmo de aprendizaje debe encontrar patrones en los datos de entrada, generando así conocimiento.

Una característica importante de *Machine Learning* es que el desempeño de sus algoritmos depende fuertemente de la representación de los datos de entrada (conocida como *hand-crafted*). Es decir, dicha representación es calculada a partir de los datos crudos, ya que los algoritmos de *Machine Learning* no pueden aprender de ellos o les es más difícil. En este contexto, cada uno de los datos de entrada calculados es denominado característica o feature.

(Goodfellow et al., 2016) incluyen, además, el área *Representation Learning*, que es un subconjunto de *Machine Learning* pero a su vez un superconjunto de *Deep Learning*. En *Representation Learning* se utiliza *Machine Learning* no solo para aprender del mapeo de los datos de entrada a la salida (predicción), sino que también para descubrir la representación óptima de los datos de entrada. De esta forma se puede obtener mejor desempeño, sobre todo para tareas complejas, para las cuales es difícil decidir cuál es el conjunto de características óptimo. Igualmente, existen casos en los cuales es igual de difícil aprender la representación óptima que resolver el problema original. Esta forma de aprendizaje de la representación óptima de los datos de entrada (por ejemplo, una foto, audio, datos de sensores) es llamada comúnmente *end-to-end* e implica que no es necesario realizar la búsqueda manual de la representación de los datos de entrada, sino que es hallada automáticamente.

Deep Learning resuelve el problema de *Representation Learning*, introduciendo representaciones que son expresadas en términos de otras representaciones más simples. De esta forma, el algoritmo de

aprendizaje puede aprender conceptos más complejos a partir de conceptos más simples a través de diferentes niveles de representación. Un ejemplo de esto son las Redes Neuronales Convolucionales aplicadas a imágenes, donde se aprenden automáticamente filtros que transforman las imágenes originales. Diferentes niveles de filtros son aprendidos, de forma que a imágenes ya filtradas se les aplican nuevos filtros, pudiendo así encontrar la representación óptima de las imágenes para la tarea de aprendizaje que se quiere llevar a cabo. Por lo tanto, la representación aprendida por un mismo algoritmo va a diferir si la tarea en cuestión es, por ejemplo, la de encontrar objetos en la imagen o si esta es la de predecir si en la imagen es de noche o de día.

2.2. Machine Learning

Machine Learning puede ser definido de varias maneras, dependiendo del autor y del campo de estudio desde el cual se aproxime uno. (Murphy, 2012) lo define como un conjunto de métodos que pueden detectar automáticamente patrones en los datos de entrada, pudiendo usar luego esos patrones descubiertos para realizar diferentes tipos de decisiones bajo incertidumbre, dependiendo de si la tarea que realiza el modelo es supervisada o no supervisada. Por su parte, de acuerdo con (Goodfellow et al., 2016), *Machine Learning* es “una forma de estadística aplicada que hace gran énfasis en el uso de computadoras para estimar estadísticamente funciones complejas y un menor énfasis en probar intervalos de confianza en dichas funciones”.

En general, en el área de *Machine Learning* las tareas son descritas a partir de cómo el modelo de *Machine Learning* debería procesar un determinado caso o ejemplo. Un caso puede ser descrito como un conjunto de características que han sido cuantitativamente medidas a partir de cierto evento u objeto, que queremos que el modelo de *Machine Learning* procese. Dichas características pueden o no haber sido calculadas a partir de los datos originales.

Típicamente, cada caso x se representa como un vector perteneciente a $\mathbb{R}^n$, donde cada componente x_i del vector representa una característica diferente. Además, los casos pueden ser representados en forma matricial $\mathbb{R}^{n \times m}$ u en otros espacios con mayores dimensiones. La representación de los datos va a diferir de acuerdo a la naturaleza de los casos y a la estructura de los datos de entrada que espera el algoritmo de aprendizaje. Por ejemplo, las características de una imagen son usualmente los valores de los píxeles de dicha imagen. Dependiendo de las imágenes, pueden estar representadas en 2 dimensiones (alto x ancho) o en 3 dimensiones (alto x ancho x canales) si la imagen tiene más de un canal, como es el caso de las imágenes satelitales. A su vez, el algoritmo de aprendizaje puede esperar como entrada un vector (como sería el caso de un Perceptron Multicapa) o alguna de las dos estructuras anteriormente nombradas. De esta forma, puede apreciarse que la estructura de los datos de entrada puede ser muy variada de acuerdo a su naturaleza, a si los datos han sido preprocesados para generar nuevas características y al algoritmo de aprendizaje que se desea utilizar.

2.2.1. Aprendizaje Supervisado y No Supervisado

Los diferentes algoritmos pueden ser clasificados a partir de la forma de los datos de entrada. En el caso del aprendizaje supervisado, los algoritmos utilizan *datasets* donde cada caso x está asociado a una etiqueta y . Dicha etiqueta es llamada variable objetivo. Las tareas de aprendizaje supervisado se basan en

lograr predecir el valor de y a partir de las características de x . Durante la fase de entrenamiento, los algoritmos de este tipo pueden saber cuán acertadas son las predicciones de acuerdo a cuán alejadas están de la variable objetivo y así cambiar su forma de predecir para minimizar su error de predicción. En el caso de los algoritmos no supervisados se utilizan *datasets* que no poseen una variable objetivo. Este tipo de algoritmos lleva a cabo la tarea de reconocimiento de patrones. Es decir, las variables de entrada son utilizadas para identificar grupos a partir del aprendizaje de propiedades útiles acerca de la estructura del *dataset*. Ejemplos: encontrar grupos con ejemplos similares (*k-nearest-neighbours*) o aprender la distribución de probabilidades del *dataset* (*denoising, synthesis*) (Berry et al., 2019).

2.2.2. Tipos de tareas

Muchos tipos de problemas o tareas pueden ser resueltos utilizando *Machine Learning*. A continuación se presentan brevemente las tareas más comunes, ya que serán de ayuda al momento de discutir cuál es el tipo de tarea que se busca abordar en esta tesis:

- *Transcripción*: en este tipo de tarea el sistema de *Machine Learning* observa una representación relativamente desestructurada de algún tipo de dato y transcribe la información al formato textual. Por ejemplo, en reconocimiento óptico de caracteres el programa de computadora recibe una fotografía que contiene una imagen con texto y debe generar como resultado el texto en la forma de una secuencia de caracteres. A forma de otro ejemplo, Google Street utiliza técnicas de *Deep learning* para procesar la dirección de los números de las casas utilizando transcripción.
- *Traducción de máquina*: en este tipo de tarea la entrada consiste en una secuencia de símbolos en un determinado lenguaje y el programa debe convertirlos en una secuencia de símbolos de otro lenguaje. Este tipo de tarea es comúnmente aplicada a lenguajes naturales como, por ejemplo, traducir del español al portugués.
- *Detección de anomalías*: en este tipo de tarea se analiza un conjunto de eventos u objetos y reporta algunos de ellos como inusuales o atípicos. Un ejemplo de esta tarea es la detección de fraudes de tarjetas de crédito. Al modelar los hábitos de compra de una compañía de tarjetas de crédito -es decir, modelando su distribución de probabilidad- es posible detectar el mal uso de dichas tarjetas. De esta forma, si un ladrón roba una tarjeta de crédito, sus compras deberían ser detectadas como *outliers* u anomalías de la distribución de probabilidad modelada.
- *Denoising*: En este tipo de tarea el algoritmo de aprendizaje tiene como entrada un ejemplo corrompido obtenido a partir de un proceso de corrupción no conocido aplicado a un ejemplo limpio. El algoritmo debe, entonces, reconstruir el ejemplo limpio a partir del ejemplo corrompido.
- *Estimación de densidad o Estimación de la función de probabilidad de masa*: en esta tarea el algoritmo debe aprender una función que puede ser interpretada como la función de densidad de probabilidad (si la entrada es continua) o como la función de probabilidad de masa (si la entrada es discreta). En general todos los tipos de tareas descritos arriba requieren que el algoritmo de

aprendizaje al menos capture la estructura de la distribución de probabilidades. Este tipo de tarea es también utilizado para completar valores faltantes en los datos.

- *Clasificación:* en este tipo de tarea un programa de computadora debe especificar a qué categoría (o clase) pertenece un determinado valor de entrada. Para resolver esta tarea el algoritmo de aprendizaje debe aprender una función $f: \mathcal{X} \rightarrow \{1, \dots, k\}$. Si, para un determinado ejemplo, $f(x) = y$, entonces quiere decir que el modelo asigna un ejemplo descrito por el vector x una categoría identificada como y . En general, la función f , en lugar de dar como resultado una categoría específica, genera una distribución de probabilidades sobre las clases, que puede ser interpretada como la confianza del clasificador sobre el hecho de que la entrada pertenezca a cada clase. Luego, a partir de dicha distribución se decide la clase predicha de la entrada. Un ejemplo de tarea de clasificación es el reconocimiento de objetos, donde la entrada del algoritmo de aprendizaje es una imagen, generalmente descrita como un conjunto de píxeles, y la salida es un código numérico identificando al objeto en la imagen.
- *Regresión:* en este tipo de tarea el algoritmo de aprendizaje debe predecir un valor numérico dado un vector de entrada. Para resolver esta tarea el algoritmo de aprendizaje debe aprender una función $f: \mathcal{X} \rightarrow \mathcal{Y}$. Este tipo de tarea es similar a las tareas de clasificación, excepto por el hecho de que el formato de salida es diferente. Un ejemplo de tarea de regresión es la de predecir el valor monetario al cual debería venderse una casa dadas las características de dicha casa como entrada para el algoritmo de aprendizaje.

En esta tesis, se define a la Predicción del Comportamiento Sedentario Futuro (PCSF) como el problema de predecir si la actividad de un usuario va a superar, o no, 1,5 MET en el futuro. En este punto, podemos divergir el camino de investigación en dos. Por un lado, se pueden tomar los datos de la actividad física, calcular el nivel de MET y generar modelos que puedan predecir el valor de MET en el futuro cercano. En este primer caso, la variable a predecir es un valor continuo y por lo tanto abordaríamos un problema de regresión. Por otro lado, se puede utilizar el valor de MET calculado para generar dos clases, una que indique comportamiento sedentario y otra que no. En este caso, la variable a predecir es discreta y puede tomar solo dos valores diferentes, abordándose así un problema de clasificación.

Estos dos tipos de caminos representan diferentes formas de atacar el problema. Algunas de las diferencias más importantes son:

- En los modelos de regresión la variable objetivo es un valor continuo mientras que en los modelos de clasificación la variable objetivo es un valor discreto.
- En los modelos de regresión se busca encontrar la línea que mejor se ajuste lo mejor posible al valor real de la variable objetivo, mientras que en los modelos de clasificación se busca encontrar una frontera de decisión - o *decision boundary*- que pueda dividir lo más precisamente posible al *dataset* en los diferentes grupos correspondientes a los valores de la variable objetivo.
- Usos comunes de modelos de regresión: predicción de clima, predicción del valor de inmuebles. Usos comunes de modelos de clasificación: identificación de mails *spam*, identificación de transacciones bancarias fraudulentas.

- Tanto las métricas de evaluación como la función de error a optimizar de ambos tipos de modelos son diferentes.

Como puede observarse, los dos tipos de modelos tienen una gran cantidad de diferencia entre ellos. Por lo que se tomó la decisión de elegir uno solo de ellos. En esta tesis, se aborda el problema de PCSF como un problema de regresión, principalmente por el hecho de que la naturaleza de la variable objetivo es continua, por lo que se deduce que convertirla en una variable discreta implicaría perder información importante. Por lo tanto, el hecho de tratar al problema en cuestión como uno de regresión permitirá una mayor granularidad al llevar a cabo las predicciones, ya que no solo se podrá determinar, a partir de las predicciones del modelo, si el usuario tendrá un comportamiento sedentario en el futuro, sino que además se podrá conocer mas precisamente cuan sedentario -o no sedentario- será su comportamiento.

2.2.3. Entrenamiento de los modelos

El entrenamiento de un modelo paramétrico de *Machine Learning* para una tarea supervisada consiste en encontrar el valor de los parámetros del modelo para los cuales el desempeño del modelo es el mayor. La **función de costo** $J(\theta)$ (también llamada función de pérdida o función de error) indica cuán erradas son las predicciones de dicho modelo dado el valor de los parámetros θ . Se define al entrenamiento de un modelo de *Machine Learning* como el proceso de optimización de la función $J(\theta)$. Este proceso de optimización puede ser de maximización o de minimización. Aunque por lo general, si se trata de un problema de maximización se lo transforma en uno de minimización simplemente cambiando el signo de la función de costo. En algunos casos, la optimización de los parámetros de un modelo para encontrar el menor error posible puede ser hallado analíticamente, como se da en el caso de la regresión lineal, aunque en la mayoría de los casos debe usarse un proceso iterativo ya que no es posible hallar una solución analítica.

Por lo general, los métodos de optimización de la función de costo de un modelo de *Machine Learning* están basados en un algoritmo conocido como Descenso por el Gradiente, o *Gradient Descent*. Este algoritmo utiliza la idea de gradiente para decidir cómo deben variar los parámetros del modelo para minimizar $J(\theta)$. El gradiente es la extensión del concepto de derivada para espacios multidimensionales. El gradiente es, entonces, un vector director unitario, donde cada uno de sus componentes es calculado como la derivada parcial de cada dimensión. En cada paso del Descenso por el Gradiente, se computa el valor de la función de costo y luego, el gradiente. A partir de él se actualizan los parámetros del modelo utilizando la siguiente fórmula:

$$\theta' = \theta - \epsilon \Delta_{\theta} J(\theta)$$

Donde ϵ representa a la tasa de aprendizaje o *learning rate* y $\Delta_{\theta} J(\theta)$ es el gradiente de la función de costo con respecto a los parámetros. La tasa de aprendizaje es un escalar positivo y decide el tamaño del paso que dará el vector de los parámetros en la dirección en la que la función de costo disminuye.

El valor de ϵ debe ser elegido cuidadosamente. Un valor demasiado grande puede provocar que el proceso de minimización nunca llegue a converger y que el valor de la función de costo se vuelva caótico aumentando y disminuyendo iteración tras iteración. Por otra parte, un valor demasiado pequeño puede provocar que el proceso de minimización sea extremadamente lento, ya que, aunque los parámetros del

modelo se ajustarán -en dirección del gradiente-, se necesitarán demasiadas iteraciones para que la función de costo converja. Para evitar este problema, suele utilizarse un valor de ϵ expresado a través de una función de la cantidad de épocas de entrenamiento, donde ϵ disminuye en función del número de iteración. Muchos de los algoritmos que extienden la idea del Descenso por el Gradiente utilizan también otros parámetros que son aprendidos.

La función de costo se calcula a partir de la fórmula

$$J(\theta) = \frac{1}{m} \sum_{i=1}^m L(x^{(i)}, y^{(i)}, \theta),$$

donde $x^{(i)}$ es un caso de entrenamiento y $y^{(i)}$ su etiqueta, m la cantidad de casos de entrenamiento disponibles y L es la función que calcula el error o costo para un caso de entrenamiento. El resultado de esta fórmula es, entonces, la esperanza de la sumatoria del error de cada uno de los casos de entrenamiento.

Se considera que el proceso de optimización que se realiza al entrenar un modelo de *Machine Learning* no responde a un proceso de optimización normal. La optimización de un modelo de *Machine Learning* se realiza indirectamente, ya que se lleva a cabo a partir del error del predictor con respecto al conjunto de casos de entrenamiento, cuando, en realidad, su desempeño está dado por su rendimiento en el conjunto de casos de testeo. Otra clara diferencia es que al entrenar un modelo de *Machine Learning* no se optimiza su función de costo hasta hallar la convergencia, o hasta que los gradientes sean muy pequeños, es decir, hasta llegar a un mínimo local o global. Sino, que se optimiza la función de costo hasta que ciertas condiciones sean cumplidas, como pueden ser que se haya llegado al desempeño deseado, que se hayan llevado a cabo la cantidad de iteraciones planeadas o que algún algoritmo como *Early Stopping* indique que el proceso de entrenamiento debe detenerse debido a que detectó *overfitting*.

Existe una buena cantidad de algoritmos de optimización de modelos de *Machine Learning* que utilizan la idea del Descenso por el Gradiente. Al método que, para actualizar el cálculo de $J(\theta)$, utiliza todos los casos de entrenamiento disponibles se lo conoce como Descenso por el Gradiente Determinístico. En la práctica, realizar el cálculo de la función de costo para realizar una única actualización de los parámetros puede ser demasiado costoso cuando la cantidad de instancias de entrenamiento asciende de cientos de miles a millones. En su lugar, se utiliza un m menor a la cantidad total de casos de entrenamiento disponibles para ajustar los parámetros, es decir que $J(\theta)$ se calcula, en cada iteración, a partir de uno a varios cientos de casos de entrenamiento. Cuando el valor de m es uno se le llama Descenso por el Gradiente Estocástico (SGD, por sus siglas en inglés), mientras que cuando m es mayor a 1 se lo conoce como Descenso por el Gradiente Mini-Batch (aunque, en ocasiones, también llamado estocástico).

Utilizar un m más pequeño permite que el algoritmo converja más rápidamente en términos de la computación requerida -y no en el número de actualizaciones requeridas-. Además, una ventaja de poder seleccionar m arbitrariamente es que, en caso de que se esté usando una unidad de procesamiento que permita realizar cálculos paralelamente como GPUs y CPUs de varios núcleos, puede aprovecharse la oportunidad de paralelización del cálculo del error de la predicción de cada caso de entrenamiento perteneciente al *batch*.

Los algoritmos de optimización utilizados en la práctica extienden la idea de SGD. El algoritmo *Momentum* utiliza el promedio de decadencia exponencial del gradiente para actualizar los parámetros y

no el gradiente en sí, lo que permite reducir la varianza introducida por SGD. La función de costo es muy sensible con respecto a algunos parámetros mientras que es insensible para otros. *Momentum* tiene lo anterior en cuenta, a expensas de introducir otro hiperparámetro (el que decide cuánto se tiene en cuenta los gradientes pasados). Otros algoritmos introducen la idea de tener una tasa de aprendizaje individual para cada parámetro. Algunos ejemplos de este último tipo de algoritmo de optimización son AdaGrad, RMSProp, ADAM y NADAM (Mustapha et al., 2020).

2.2.4. Evaluación de los modelos

Existen diferentes métodos para evaluar el desempeño de un modelo de *Machine Learning*. El uso de tal o cual modelo depende de la naturaleza de los datos y la cantidad de ellos.

La forma más simple de evaluar un modelo es dividirlo en un *dataset* de entrenamiento y otro de testeo. En este caso, se entrena el modelo utilizando como datos de entrada el *dataset* de entrenamiento y luego se evalúa al modelo a partir de los datos de testeo. Es importante notar que los datos de testeo deben ser datos a partir de los cuales el modelo no fue entrenado, por lo que estos dos conjuntos deben ser siempre disjuntos. De esta forma, se evalúa la capacidad de **generalizar** del modelo, es decir, de desempeñarse bien en datos que no habían sido observados.

En el proceso de entrenamiento se buscan los valores óptimos de los parámetros que permitan el mejor desempeño del modelo en el conjunto de entrenamiento, a partir de un método de optimización, como el descenso por el gradiente. Para llevar a cabo dicho proceso de entrenamiento es necesario definir antes el valor de los hiperparámetro, ya que diferentes valores suponen diferentes modelos a ser entrenados, y por lo tanto, posiblemente diferente desempeño en la tarea a realizar. Bajo este esquema, lo más común es entrenar y testear al modelo a partir de diferentes combinaciones de hiperparámetro, seleccionando al final la combinación a partir de la cual el modelo ha obtenido el mejor desempeño en el conjunto de testeo. Estas combinaciones pueden crearse de manera manual o mediante algún método de optimización, como la Optimización de Bayes.

Puede observarse ahora que, de la misma forma que se optimizan los parámetros del modelo para obtener un buen desempeño en el conjunto de entrenamiento, los hiperparámetro también deben ser optimizados para alcanzar un buen desempeño en el conjunto de testeo. Por lo tanto, existe la necesidad de un tercer conjunto, que permita verificar el poder de generalización del modelo que mejor desempeño alcanzó en el conjunto de testeo. Este último conjunto es llamado *dataset* de validación.

Cuando los datos disponibles son escasos, el método anterior no debe ser utilizado, ya que las diferentes particiones dejarán de ser estadísticamente representativas de la fuente original de los datos (sobre todo los conjuntos de testeo y validación, que, en general, representan un porcentaje mínimo del *dataset*). En estos casos, se utiliza la validación cruzada o *cross-validation*. La validación cruzada es un método estadístico para evaluar y comparar modelos de *Machine Learning* (Refaeilzadeh et al., 2009). La técnica más básica de validación cruzada es llamada *k-fold*. Esta técnica consiste en dividir el *dataset* en k particiones. Para cada partición i , se entrena el modelo en las $k - 1$ particiones restantes y se lo evalúa en la partición i . La evaluación final del modelo es igual a la media del desempeño en cada una de las iteraciones. La técnica anterior tiene la característica de tener baja replicabilidad. Una extensión de *k-fold* es conocida como *multiple run k-fold* y consiste en realizar la validación cruzada sobre las k particiones

repetidas veces, donde cada corrida genera diferentes particiones, aumentando así la replicabilidad (Bouckaert, 2003).

En el caso en que se esté prediciendo el futuro a partir del pasado, como es el caso de las series de tiempo, ciertas consideraciones especiales deben ser tomadas en cuenta. Para empezar, los datos no deben ser reordenados aleatoriamente. Asimismo, sin importar la técnica de validación, los datos de testeo deben ser siempre temporalmente posteriores a los datos de entrenamiento. Existen técnicas de validación específicas para las series de tiempo que serán explicadas en la [Sección 4.3.4](#) (Chollet, 2017).

2.2.5. Generalización, *underfitting* y *overfitting*

El objetivo principal en *Machine Learning* es que el modelo entrenado para llevar a cabo una tarea determinada tenga un buen desempeño en datos de entrada nuevos. Esto es, en casos que no hayan sido vistos antes por el modelo, y no solamente en los casos que fueron utilizados para entrenar a dicho modelo.

Como se explicó en la sección anterior, al entrenar un algoritmo de *Machine Learning*, se utiliza un conjunto de datos de entrenamiento, a partir del cual se intenta minimizar el error, lo que puede considerarse un problema de optimización. A partir de esto, se calcula el error de generalización o de testeo. El error de testeo es el error esperado de un modelo al predecir a partir de un caso no observado, que es calculado a partir del conjunto de testeo. Los parámetros del modelo (aquellos que se ajustan en el proceso de optimización) son ajustados de acuerdo al conjunto de entrenamiento, por lo que se espera que el error de testeo sea siempre mayor al error de entrenamiento.

Por lo tanto, se busca lograr dos objetivos claros al momento de validar un algoritmo de *Machine Learning*:

- Lograr que el error de entrenamiento sea bajo, y
- Lograr que el error de testeo sea lo más similar posible al error de entrenamiento.

Estos dos objetivos se corresponden con dos desafíos importantes a superar en *Machine Learning*: *underfitting* y *overfitting*. *Underfitting* se da cuando el modelo no logra obtener un error de entrenamiento lo suficientemente bajo, mientras que *overfitting* ocurre cuando la diferencia entre el error de entrenamiento y el de testeo es demasiado grande.

La tarea a la hora de entrenar un modelo es encontrar un punto medio de tal modo de que no se produzca ni *overfitting* ni *underfitting*. Esto último se logra aumentando o disminuyendo la capacidad del modelo. La capacidad se refiere a la complejidad del espacio de hipótesis, esto es, todas las funciones que el algoritmo de aprendizaje puede llegar a seleccionar como posible solución hipótesis. Por ejemplo, el espacio de hipótesis de la regresión lineal es el conjunto de todas las posibles funciones lineales de la forma

$$y = w \cdot x + b.$$

La capacidad de un algoritmo de este tipo puede aumentarse generalizando el espacio de hipótesis para incluir polinomios de grado n de la forma

$$y = \sum_{i=1}^n w_i x^i.$$

El desempeño de los modelos de *Machine Learning* va a ser mejor cuanto mejor se adapte su capacidad a la complejidad de la tarea a llevar a cabo. De esta forma, los modelos con una baja capacidad

son incapaces de resolver tareas complejas, por lo que va a producirse *underfitting*. Mientras que los modelos con alta capacidad pueden resolver tareas complejas, pero si su capacidad es más alta que la necesaria para resolver una determinada tarea, pueden cometer *overfitting*, ya que la función aprendida no tendrá un buen desempeño en los casos de testeo.

En la Figura 2.1 pueden verse tres modelos diferentes entrenados a partir de un conjunto de datos. Dicho conjunto de datos fue generado evaluando valores aleatorios de la variable independiente en una función cuadrática. En este caso, la tarea de los modelos es la de predecir el valor de y para cada valor de x . Los modelos fueron ajustados a una función lineal, una función cuadrática y un polinomio de grado 9, respectivamente. Cada modelo ajustó un número diferente de parámetros, uno por cada grado del polinomio. Es decir, el primer modelo debió ajustar 2 parámetros (el peso asociado a la variable de grado 1 y aquel asociado a la variable de grado 0, también llamado bias), el segundo 3 y el último 9. Por lo tanto, la capacidad de los modelos aumenta de acuerdo al grado del polinomio, ya que el conjunto de hipótesis de un polinomio de grado n incluye al de uno de grado m , si $m < n$. Puede observarse entonces que:

- El modelo de la izquierda produce *underfitting*, ya que debido a su baja capacidad no puede obtener un bajo error de entrenamiento.
- El modelo de la derecha produce *overfitting*, ya que tendrá un error alto en los casos de testeo por no haber podido aprender la función verdadera a partir de la cual el conjunto de datos fue generado.
- Por último, en el modelo del centro la capacidad es la adecuada para la tarea a realizar, por lo que el modelo ha logrado generalizar a partir de los casos de entrenamiento.

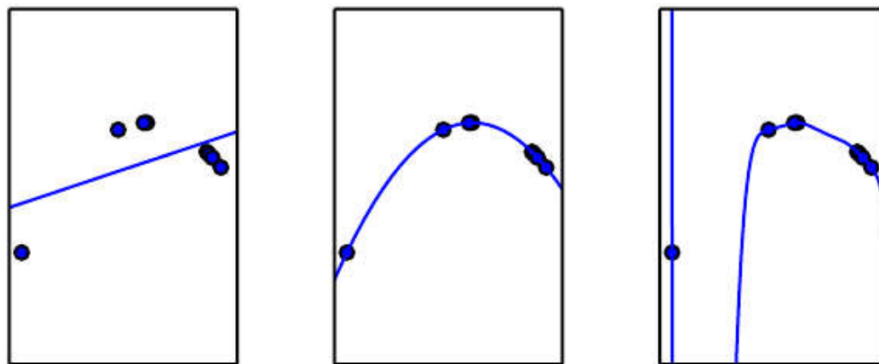


Figura 2.1. (Goodfellow et al., 2016). Funciones polinómicas de diferente grado ajustadas a un dataset

2.2.6. Regularización

En la sección anterior, se ha presentado un trade-off entre *overfitting* y *underfitting* a partir de aumentar o disminuir la capacidad de un algoritmo de aprendizaje al cambiar el modelo del cual aprende (la función). Este proceso puede llevarse a cabo de forma más general, ya que es posible configurar a un algoritmo para preferir algunas funciones pertenecientes a su espacio de hipótesis por sobre otras de acuerdo a cierto

criterio utilizando una técnica de regularización. Es decir, las funciones menos preferidas van a ser elegidas solo si se desempeñan mucho mejor que las preferidas en el conjunto de entrenamiento. Puede interpretarse a la regularización como una forma general de variar la capacidad de un algoritmo de aprendizaje, ya que quitar a una función del espacio de hipótesis puede interpretarse como una preferencia infinita de las funciones que aún pertenecen al espacio de hipótesis por sobre la función excluida. Por lo tanto, las técnicas de regularización tienen como objetivo disminuir la capacidad de un algoritmo de aprendizaje sin excluir explícitamente ninguna función del espacio de hipótesis, sino que dejando que el algoritmo defina cuáles son las funciones más apropiadas a aprender en el proceso de entrenamiento.

Se define a la regularización como cualquier cambio que se le haga al algoritmo de aprendizaje para reducir el error de testeo o generalización, pero no su error de entrenamiento. Dicho de otra manera, se busca que el modelo aprenda la función más simple posible que consiga un bajo error de entrenamiento, de tal forma que pueda aun generalizar, es decir, tener un bajo error de testeo.

A modo de ejemplo, supóngase que se generan conjuntos de entrenamiento y testeo a partir de una función cuadrática. A continuación, se entrena un modelo cuya capacidad son todas las funciones polinómicas. Si no se utilizase ninguna técnica de regularización, el algoritmo aprendería una función utilizando un polinomio de grado n (es decir, no tendrá ninguna preferencia por aprender polinomio de grado 2). De esta forma, el error de testeo será elevado, ya que el modelo predecirá el conjunto de testeo a partir del polinomio aprendido y no específicamente a partir de la función cuadrática. En este caso, una técnica de regularización podría penalizar al algoritmo por cuanto más grande sea el grado del polinomio. Debido a esto, el algoritmo deberá minimizar, no solo el error de entrenamiento, sino que también el grado del polinomio. Por lo tanto, el algoritmo terminará aprendiendo la función cuadrática. Nótese que el algoritmo no debería aprender un polinomio de grado 1 (función lineal) ya que de esta forma aumentaría el error de testeo (suponiendo que el conjunto de entrenamiento es lo suficientemente grande como para no parecerse a una función lineal). A continuación se presentan las técnicas de regularización más comunes que pueden ser utilizadas tanto en algoritmos de *Machine Learning* como de *Deep Learning*.

La “Penalización de la norma de los parámetros” consiste en añadir una penalidad de la norma de los parámetros a la función de pérdida J , de tal manera que el algoritmo de optimización deba minimizar, no solo el error, sino que también la norma de los parámetros. Si llamamos $\hat{J}$ a esta nueva función objetivo, entonces:

$$\hat{J} = J(\theta, X, y) = J(\theta, X, y) + \alpha \Omega(\theta),$$

donde $\alpha \in [0, \infty)$ es un hiperparámetro que modifica la contribución del término de penalización Ω a la función objetivo. Si α es igual a 0 entonces no hay regularización, mientras que cuanto mayor es α , mayor es la regularización aplicada. Existen diferentes funciones de la norma de los parámetros que pueden ser usadas, como el término Ω en la función objetivo regularizada:

- Regularización L^2 : esta técnica de regularización es también conocida como *weight decay* o ridge regression. La regularización L^2 acerca el vector de los parámetros al origen añadiendo el siguiente término de regularización

$$\Omega(\theta) = \frac{1}{2} \|w\|_2^2 = \frac{1}{2} w^T w,$$

donde w es el vector de los parámetros.

- Regularización L^1 : esta técnica de regularización tiende a que algunos parámetros tomen el valor de 0, por lo que puede ser usada para seleccionar las características más importantes para el modelo (*feature selection*). Está definida por el término

$$\Omega(\theta) = ||w||_1 = \sum_i |w_i|,$$

- Selección de un tamaño de bache pequeño: puede considerarse a la selección de un tamaño de bache pequeño como un tipo de regularización, ya que añade ruido al proceso de aprendizaje.

2.2.7. Métricas de evaluación

Para evaluar las habilidades de un algoritmo de *Machine Learning* debe utilizarse una métrica cuantitativa de su desempeño. Las métricas utilizadas poseen diferentes cualidades y existen diferentes de ellas para cada tipo de tarea. Además, cada una de las métricas existentes se desempeña mejor o peor dependiendo de las características específicas del problema. A continuación, se detallan y discuten algunas de las métricas de evaluación más utilizadas para las tareas de regresión.

2.2.7.1. Métricas de regresión

Las métricas de regresión son utilizadas para comparar los valores predichos por modelos de regresión con los valores reales. Sean f la función aprendida por el modelo e y la variable objetivo, con $f(x) = \hat{y} \in \mathbb{R}$ e $y \in \mathbb{R}$, las métricas de regresión tienen por objetivo evaluar cuán precisamente los valores predichos por el modelo se asemejan a la variable objetivo y . En otras palabras, buscan resumir la distancia entre $\hat{y}$ e y para cada uno de los casos de testeo. Para todas las métricas, el resultado va a ser mejor cuanto más cercanos sean $\hat{y}$ e y , pero difieren en distintos aspectos, como el método de agregación de la distancia entre los puntos para el *dataset* de testeo y el método para obtener la distancia entre dos puntos.

A continuación se presentan las métricas de regresión más simples o primarias:

- **Magnitud del error:** esta métrica no es utilizada, pero sirve a modo de comprensión de las siguientes métricas. Esta métrica se basa en obtener la magnitud para todos los casos de testeo y obtener la media. De esta forma, para cada caso se puede determinar claramente si el modelo tiende a predecir un valor mayor o menor que el valor real. El problema de este método se da en la agregación, donde errores negativos y positivos se cancelan entre ellos, por lo que el resultado no es confiable. Por esta razón, las métricas más utilizadas se basan ya sea el cuadrado o el valor absoluto al momento de obtener cada error individual.

- **Error cuadrático medio o *Mean Squared Error (MSE)*:** esta métrica se calcula como la media de la suma de los cuadrados de la diferencia entre los valores $\hat{y}$ e y . Su fórmula es:

$$MSE = \frac{1}{n} \sum (y - \hat{y})^2$$

En este caso, podemos observar que la función de distancia es el cuadrado de la diferencia. De esta forma, los errores no pueden ser negativos y cancelarse. Por otro lado, la función de agregación, que resume los errores individuales, es la media.

Como consecuencia de utilizar el cuadrado de la diferencia, el error individual crece cuadráticamente con respecto a la diferencia entre $\hat{y}$ e y . De esta forma, se sobre-penalizan los errores grandes. Por lo tanto, esta no es una métrica recomendada en casos donde el *dataset* posea muchos valores atípicos, también conocidos como *outliers*, ya que puede darse un error muy grande aun cuando el modelo haya aprendido bien la tarea a realizar. En otras palabras, se produce una sobreestimación de cuán erróneo es el modelo.

Una importante ventaja de la métrica MSE es que es diferenciable en todos sus puntos, por lo que puede ser utilizada sin problemas como función de optimización.

- **Raíz del Error cuadrático medio o *Root Mean Squared Error (RMSE)*:** esta métrica es muy similar a MSE. A diferencia de esta última, el método de agregación aplica la raíz cuadrada luego de calcular la media. Por lo tanto, su fórmula es:

$$RMSE = \sqrt{\frac{1}{n} \sum (y - \hat{y})^2} = \sqrt{MSE}$$

Aplicar la raíz cuadrada tiene la propiedad de que el error estará expresado en la misma unidad que la variable objetivo, lo que le otorga más significado al error. Este no es el caso de MSE, ya que el error estará expresado en el cuadrado de la dimensión de la variable objetivo.

- **Error medio absoluto o *Mean Absolute Error (MAE)*:** este error difiere de las dos métricas anteriores en el método utilizado para calcular la distancia entre el valor de la variable objetivo y el del valor predicho. En este caso, en lugar del cuadrado de la distancia, se utiliza el valor absoluto de ella. MAE, por lo tanto, está definida por la siguiente fórmula

$$MAE = \frac{1}{n} \sum |y - \hat{y}|$$

Al utilizar valor absoluto, el error estará expresado en la misma unidad que lo está la variable objetivo, como se da en el caso de RMSE. La no utilización del cuadrado para calcular la distancia entre los puntos implica ciertas consecuencias. Primero, MAE no es diferenciable, por lo que presenta ciertas dificultades para ser utilizada para la optimización de parámetros de modelos de *Deep Learning*. Segundo, MAE penaliza linealmente los errores, por lo que si se quiere que el modelo preste más atención a los *outliers* tal vez no es la métrica adecuada.

Existen multitud de métricas de normalización que no están mencionadas en esta sección. Por tal razón, a continuación se listan ciertas características por las cuales difieren de las métricas previamente presentadas. Existen métricas que utilizan otros métodos de agregación. Por ejemplo, en lugar de la media, se utiliza la media geométrica o simplemente la suma de todos los errores. Otras métricas, conocidas como adimensionales o *dimensionless*, aplican una normalización a los errores individuales. Por ejemplo, puede dividirse a cada error por el valor de la variable objetivo, a forma de normalizar el resultado y hacerlo adimensional. La ventaja de la normalización es que permite la comparación de resultados de la performance del modelo entre diferentes *datasets*.

2.3. *Deep Learning*

Como fue notado brevemente en la introducción, *Deep Learning* se diferencia de *Machine Learning* en que el primero utiliza una sucesión de representaciones, lo que permite conseguir representaciones más complejas. Igualmente, como es en el caso de *Machine Learning*, todos los algoritmos de aprendizaje tienen por objetivo resolver una tarea específica al aproximar una función f^* .

Todos los modelos utilizados en esta tesis son casos de redes neuronales. La palabra red de “Redes Neuronales” se debe a la estructura de grafo a partir de la cual estos modelos pueden ser representados. Cada uno de los modelos tiene características que los diferencian de los demás y que los hacen más o menos propensos a ser usados en ciertos tipos de tareas. A su vez, estas diferencias implican la existencia de ventajas y desventajas a la hora de utilizarlos, y no solo teniendo en cuenta los tipos de tareas en los cuales se desempeñan mejor, sino que también entran en juego otros factores como el tiempo de entrenamiento necesario, el número de parámetros a entrenar, los hiperparámetros a ajustar, el tamaño del modelo, etc.

Durante el entrenamiento de una red neuronal, se intenta que la función f sea lo más parecida posible a la función subyacente que da origen a los datos de entrada, f^* . Como en el caso de *Machine Learning*, en las tareas supervisadas, como la que se ataca en esta tesis, cada caso de entrenamiento está acompañado por una etiqueta o *label* y . En *Deep Learning*, se busca que el resultado de la *output layer* sea lo más parecido posible a y .

2.3.1. Hiperparámetros

La mayoría de los modelos de *Deep Learning* tienen ciertas configuraciones que deben ser ajustadas para que el modelo se desempeñe de la mejor manera posible. Estos hiperparámetros no son aprendidos por el algoritmo de aprendizaje que utiliza el modelo. Por esta razón, no son considerados parámetros. Como se verá más adelante en la [Sección 4.3.5.1](#), pueden utilizarse algoritmos de aprendizaje para encontrar los mejores valores para los hiperparámetros para el modelo.

Como ya se ha visto en la Sección [2.2.4](#), los hiperparámetros introducen la necesidad de un nuevo *dataset*, diferente al *dataset* de testeo, en el cual sea posible evaluar la mejor configuración de los hiperparámetros hallada. Este tercer *dataset* -en adición a los de entrenamiento y testeo- es conocido como *dataset* de validación.

Los hiperparámetros pueden ser configuraciones relacionadas al algoritmo de aprendizaje o la arquitectura del modelo. Por lo tanto, al existir diferentes arquitecturas, cada modelo tendrá hiperparámetros específicos. A continuación, se definen los hiperparámetro más generales y que forman parte de todos los modelos. Los hiperparámetros que difieren de modelo en modelo serán introducidos en su respectiva sección.

- **Número de capas:** los modelos de *Deep Learning* está formado por una secuencia de capas, donde cada capa está formada por neuronas. Este hiperparámetro define cuán profundo es el modelo, en relación a la cantidad de relaciones intermedias que este va a poder aprender.
- **Número de neuronas:** cada capa del modelo está formada por neuronas, donde cada neurona realiza cierto procesamiento a partir de la salida de neuronas de la capa anterior. La cantidad de neuronas en una capa puede diferir de la cantidad de neuronas presentes en otra capa. Además, la entrada de una neurona y el procesamiento llevado a cabo por ella dependen en gran medida del tipo de modelo al que ésta pertenezca.
- **Función de activación:** es la función que se aplica a la salida de las neuronas que pertenecen a las capas ocultas de la red neuronal. La función de activación tiene la finalidad de introducir no-linealidades a la red neuronal. Es decir, sea z la función de activación, entonces z es no-lineal. Típicamente, el valor de una neurona es la suma de sus entradas ponderadas por sus respectivos pesos, esto es, un número real. Por lo tanto, $z: z(x) = y$ es una función tal que $x \in \mathbb{R}$ e $y \in \mathbb{R}$, donde z no puede ser expresada como una función lineal, de la forma $ax + b$.

Existen muchas funciones de activación. Por ejemplo, la función sigmoide $\frac{1}{1+e^{-x}}$, la función tangente $\frac{e^x}{1+e^{-x}}$. Asimismo, la función de activación que se ha vuelto el estándar en la actualidad es llamada ReLU, por su nombre en inglés "*Rectified Linear Unit*" donde $z(x) = \max(0, x)$. Esta función tiene la propiedad de ser "casi" lineal, ya que está compuesta por dos funciones lineales, aunque sigue siendo no lineal. De esta forma, comparte ciertas propiedades con las funciones lineales, lo que hace que los modelos que las utilicen sean fáciles de optimizar a partir de métodos de optimización basados en el gradiente (Goodfellow et al., 2016).

- **Número de épocas:** número de iteraciones que se recorre el *dataset* de entrenamiento completo en el proceso de entrenamiento.
- **Tamaño de batch:** define el número de casos de entrenamiento que serán tomados del *dataset* de entrenamiento para realizar una iteración del algoritmo de aprendizaje. Cuanto mayor sea el tamaño de batch, menor será la cantidad de iteraciones que deberá llevarse a cabo en cada época. Para usar batches, el algoritmo de aprendizaje divide el *dataset* de entrenamiento en n batches. Luego, para cada batch, se computa el error del modelo y se actualizan los parámetros

mediante la optimización basada en el gradiente. La ventaja de actualizar los parámetros al final de cada bache y no luego de recorrer el dataset de entrenamiento entero es que permite actualizar los parámetros con mayor frecuencia, lo que permite que la función de optimización converja más rápidamente. Si la cantidad de baches es 1, se realizan tantas actualizaciones a los parámetros como número de épocas.

- **Tasa de aprendizaje o *learning rate*:** define el factor en que se actualizan los parámetros del modelo en relación al gradiente computado en cada iteración. Sean θ los parámetros del modelo, g el gradiente computado y ϵ la tasa de aprendizaje, entonces los parámetros del modelo se actualizan como $\theta \leftarrow \theta - \epsilon g$ al final de cada iteración.

2.3.2. Modelos de Deep Learning utilizados

A continuación, se presentan los diferentes modelos utilizados, todos ellos pertenecientes al área de Aprendizaje Profundo o *Deep Learning*. Para cada uno de ellos, se realiza una breve introducción y se desarrollan sus características, ventajas y desventajas de su utilización. Además, se discute el uso de cada tipo de modelo en el marco de problemas donde los datos de entrada poseen una organización secuencial.

2.3.2.1. Perceptrón Multicapa

El Perceptrón Multicapa o *Multilayer Perceptron* -MLP, por sus siglas en inglés- es el modelo más sencillo dentro del campo de *Deep Learning* y puede aprender múltiples niveles de representación de los datos de entrada para modelar relaciones complejas entre dichos datos. De esta forma, las características de alto nivel están definidas en términos de características de bajo nivel. Por lo tanto, los MLP se componen de una secuencia de capas, donde cada capa modela una función que utiliza como entrada la salida de la capa/función anterior. De esta forma, puede representarse a los MLP como una composición de funciones (Goodfellow, Bengio, and Courville 2016).

En las MLP, cada capa está compuesta por un cierto número de unidades computacionales o neuronas. A su vez, cada neurona posee conexiones directas con todas las neuronas de la capa anterior. Cuando existen conexiones entre todas las neuronas de la capa i y la capa $i - 1$, la capa i es llamada capa densa. Cada neurona lleva a cabo cierto cálculo y produce como salida un valor que es luego emitido a hacia todas sus conexiones salientes. Cada conexión tiene un peso que corresponde a cuán fuertemente están conectadas dos neuronas. Estos pesos son los parámetros de las MLP, y más particularmente, de las capas densas, que deben ser ajustados por el algoritmo de aprendizaje. Típicamente, la computación que lleva a cabo cada neurona está separada en dos etapas: primero realiza la suma ponderada de todas sus conexiones entrantes y luego aplica una función de activación.

Como ya se ha mencionado, la función aplicada por cada capa puede ser vista como una representación diferente de la salida de la capa anterior. Es interesante analizar esto gráficamente a través de elementos del álgebra lineal. Para ello, es necesario mostrar qué cálculos lleva a cabo cada capa densa. Sean:

- n la dimensión de entrada y m la cantidad de neuronas de la capa densa i

- x el vector de salida la capa $i - 1$, perteneciente a $\mathbb{R}^n$.
- W la matriz que contiene todos los pesos entre la capa i e $i - 1$, perteneciente a $\mathbb{R}^{m \times n}$.
- b el *bias*, perteneciente a $\mathbb{R}^m$.
- A una función de activación no lineal.

Entonces, la salida y de la capa i es la función f :

$$y_i = f(x)_i = A(W_i x + b_i), \text{ con } x = y_{i-1}$$

Donde W puede ser interpretada como una matriz de transformación lineal que mapea un vector de $\mathbb{R}^n$ a $\mathbb{R}^m$, b es un desplazamiento del vector resultante de la transformación lineal, y $A(*)$ es aplicada a cada componente del vector resultado. Por lo tanto, $f: \mathbb{R}^n \rightarrow \mathbb{R}^m$.

En la Figura 2.2 puede observarse un ejemplo de red MLP, compuesta por 5 capas, la capa de entrada, la capa de salida, y 3 capas ocultas o *hidden layers*. Los vectores de entrada de la red pertenecen a $\mathbb{R}^4$, los de salida a $\mathbb{R}^1$ y las dimensiones de entrada de las capas ocultas pertenecen a $\mathbb{R}^8$. Los pesos -es decir, la matriz W - están representados en la figura a partir de las líneas que unen a las neuronas de dos capas adyacentes. El color rojo significa un valor positivo del peso, mientras que el azul un valor negativo. Cuanto más visible es la línea entre dos neuronas, mayor es el valor absoluto del peso. La red neuronal del ejemplo puede estar representando a un modelo ya entrenado o a uno sin entrenar, donde los pesos fueron inicializados aleatoriamente.

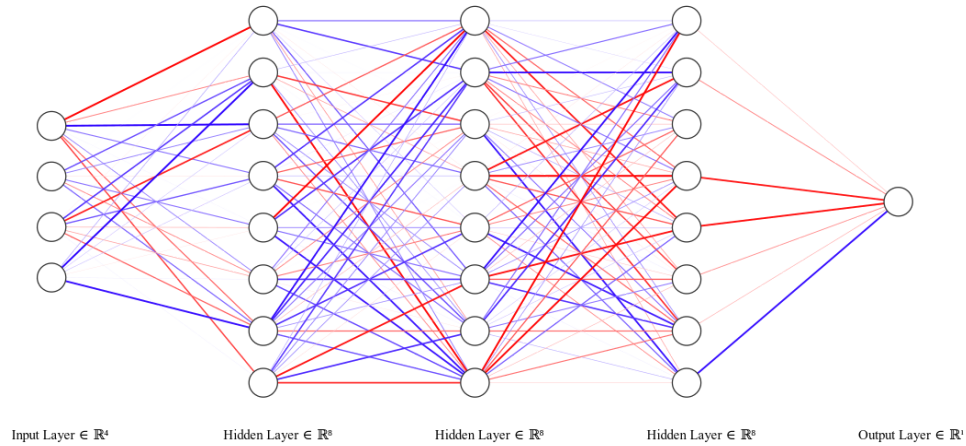


Figura 2.2. Ejemplo de red neuronal densa

A modo de visualizar lo explicado anteriormente, implementé una *Jupyter Notebook*³ en la cual se entrena a un MLP simple cuya tarea es la de clasificar un *dataset* artificial formado por 2 clases de puntos concéntricos pertenecientes a $\mathbb{R}^2$. Una vez que el modelo ha sido entrenado, puede observarse la predicción para cada punto del espacio, junto a su confianza, como muestra la Figura 2.3. La herramienta implementada permite, además, obtener las salidas de las capas intermedias y graficar, paso a paso, cómo

³ <https://github.com/martinscooper/Making-it-linearly-separable>

se van llevando a cabo las transformaciones y cuáles son las representaciones de cada capa. En la Figura 2.4, la primera fila muestra las transformaciones lineales de los vectores generados por la capa anterior más un desplazamiento, mientras que las columnas muestran el resultado de la aplicación de la función de activación (en este caso la función es sigmoide). Debe notarse que, como las transformaciones aplicadas por cada capa se muestran de forma acumulativa, los espacios no pertenecen a la salida de una transformación lineal. Por el tipo de *dataset*, las capas internas de la red entrenada tienen 3 neuronas y no 2. Por lo tanto, todas las representaciones que se muestran son parciales, es decir, se grafican 2 de las 3 dimensiones. Sin importar cual sea el *dataset*, la MLP intentará hallar un hiperplano que separe ambas clases⁴. Como en este caso no existe una recta que separe ambas clases, es necesario proyectar el *dataset* a una dimensión mayor, en cuyo caso la red termina aprendiendo un plano que divide ambas clases. En las 2 dimensiones graficadas, se puede observar que las dos clases son linealmente separables.

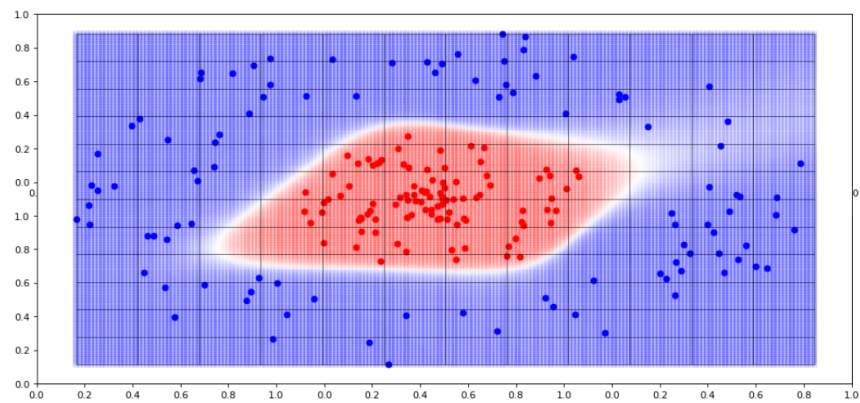


Figura 2.3. Predicciones de una MLP

⁴ <https://colah.github.io/posts/2014-03-NN-Manifolds-Topology/>

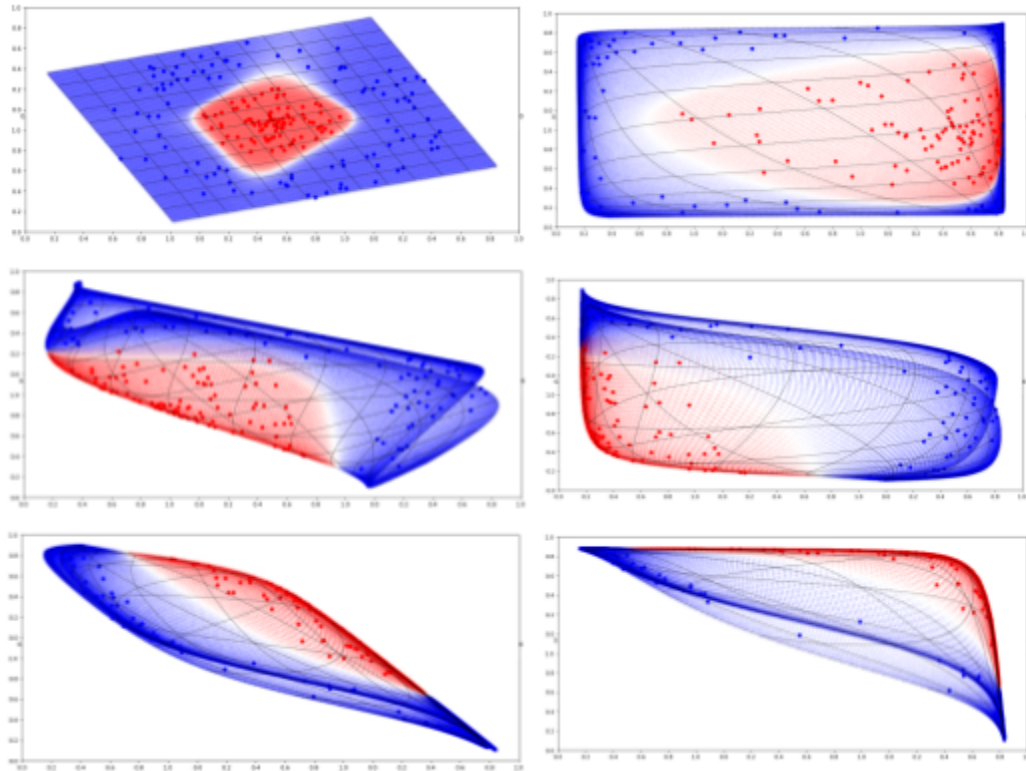


Figura 2.4. Transformaciones internas de una MLP

Una de las desventajas de utilizar este tipo de redes para el modelado de secuencias es que ignora completamente la topología de la entrada, es decir, *timesteps* y características se representan en la misma dimensión, por lo que no es posible “obligar” a la red a que tenga en cuenta estas representaciones. Dicho de otra manera, las variables de entrada podrían ser presentadas en cualquier orden sin afectar el resultado del entrenamiento. En este trabajo se consideró dicha arquitectura con el fin de evaluar si utilizar arquitecturas que tengan en cuenta la estructura secuencial de los datos, representa o no una mejora en la precisión de los modelos.

2.3.2.2. Redes Neuronales Recurrentes

Las Redes Neuronales Recurrentes o *Recurrent Neural Network* -RNN por sus siglas en inglés- son modelos donde la entrada de la red son secuencias. Las unidades de las RNN mantienen un estado interno que es propagado a través del tiempo. Dicho estado interno funciona como la memoria de las unidades y puede actuar como una representación de todo aquello que la red haya visto en la secuencia de entrada hasta el momento. En síntesis, las RNN tiene como entrada un vector que representa una secuencia (por ejemplo: una oración) y las neuronas o unidades procesan cada elemento de dicho vector uno a la vez e iterativamente (“Finding Structure in Time,” 1990). Las RNN son representadas computacionalmente mediante grafos cíclicos, es decir que poseen ciclos. Dichos ciclos representan la influencia del valor actual de una variable en su valor en un momento futuro.

Las RNN han ganado mucha popularidad por su buen desempeño en tareas relacionadas al modelado del lenguaje (Hermans y Schrauwen, 2013) y traducción de máquina (Mahata et al., 2019). Desgraciadamente, las arquitecturas más simples de RNN son difíciles de entrenar (Pascanu et al. 2013), ya que presentan problemas como *vanishing gradient* y *exploding gradient* (Bengio et al., 1994), por lo que en la práctica se utilizan arquitecturas más complejas que logran sortear en cierta medida dichos problemas. Estas variantes utilizadas son las LSTM (Hochreiter & Schmidhuber, 1997) y las GRU (Cho et al., 2014) que añaden mayor cantidad de información a cada neurona en comparación con las RNN más simples, como las *forget gate*, que son parámetros que se aprenden cuando se entrena el modelo y deciden que parte del vector de entrada será descartado.

Más allá del tipo de neurona que se use en una RNN, el mecanismo es similar a todas ellas. Como se muestra en la Figura 2.5, la entrada de cada neurona es una secuencia que es procesada iterativamente, es decir, un elemento de la secuencia a la vez. Cada neurona tiene un estado interno que va cambiando a medida que la secuencia es procesada (más o menos complejo dependiendo del tipo de neurona recurrente), emulando la memoria de la red. Es importante señalar aquí una diferencia clara entre las RNN y el tipo de arquitectura explicada en la subsección anterior, las MLP. Como ya ha sido explicado, las unidades de las RNN reciben y procesan la información de manera iterativa. De esta manera, el orden en que la secuencia de entrada es procesada es importante y debe respetar su orden natural (por ejemplo, el del tiempo, o de izquierda a derecha en el caso del procesamiento de texto) ya que la red está diseñada para sacar provecho de dicho orden. Si se compara este punto con las MLP, puede verse que, en ellas, cada neurona recibe toda la información “al mismo tiempo”, impidiendo que pueda tomarse en cuenta el orden secuencial. De esto se desprende otra importante diferencia: las RNN comparten los mismos parámetros para cada componente de la secuencia, a diferencia de las MLP que poseen parámetros diferentes para cada parte de ella (Goodfellow et al., 2016).

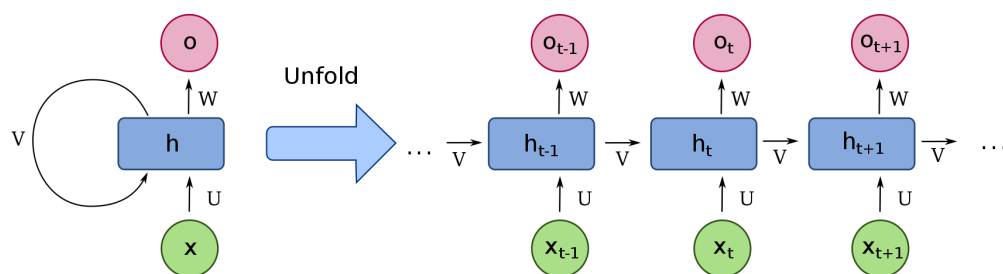


Figura 2.5. Red Neuronal recurrente desdoblada o “unfold”

A continuación se desarrollan dos ejemplos de la utilización de las RNN:

En Yan and Ouyang (2017) exploraron la aplicabilidad y validez de aplicar redes LSTM a series de tiempo financieras. En particular, abordaron la tarea de predecir el precio de cierre del Índice Compuesto de Shanghái. Para llevar a cabo esta tarea, los investigadores tomaron datos históricos desde enero del

2012 hasta junio del 2017. A partir de esos datos, para cada día se calcularon diferentes índices, como el precio de cierre del día anterior, el precio de apertura del día anterior, el mayor precio del día anterior, etc. Luego, a la variable objetivo le fue aplicada una transformación que permite reducir el ruido en la serie de tiempo, con el fin de aumentar la capacidad de generalización de los modelos, ya que impide que aprenda a partir de una serie de tiempo ruidosa. Finalmente, los modelos LSTM fueron comparados con otros modelos más sencillos, como Perceptrones Multicapa, *Support Vector Machine* y *K-nearest neighbors*. Los modelos LSTM obtuvieron un mejor desempeño en comparación con los otros modelos gracias a su mejor capacidad para capturar las características no lineales de las series de tiempo financieras.

En(Wang et al. (2020) los investigadores abordaron la tarea de modelar la tendencia epidémica a largo plazo de la epidemia COVID-19 mediante el uso de modelos LSTM. El estudio se realizó a partir de 3 países: Rusia, Perú e Irán, donde en cada uno de ellos los gobiernos aplicaron medidas diferentes ante la pandemia. Los modelos fueron entrenados utilizando los datos proveídos por la Universidad John Hopkins. Los investigadores tomaron los registros disponibles desde el 22 de enero al 7 de julio de 2020. Se utilizaron los modelos LSTM para predecir la curva del total de casos confirmados por el virus COVID-19 para los 150 días siguientes. Al ser poca la cantidad de casos de estudio para entrenar el modelo los investigadores utilizaron el método de optimización continua, mediante el cual los casos ya predichos son utilizados como nuevos casos de entrenamientos. Wang et. al reportan que los resultados de las predicciones son altamente consistentes con los casos positivos diarios notificados, lo que demuestra que el método propuesto puede analizar con precisión la tendencia de la epidemia.

2.3.2.3. Redes Neuronales Convolucionales

Las Redes Neuronales Convolucionales o *Convolutional Neural Networks* -CNN, por sus siglas en inglés- son muy comunes en el procesamiento de imágenes y presentan varias diferencias con respecto a las MLP. Por un lado, las CNN toman como entrada datos claramente estructurados, en los que las variables cercanas están fuertemente correlacionadas (por ejemplo, imágenes). Por otro lado, este tipo de redes reduce en gran medida el número de parámetros a entrenar, ya que estos son compartidos por todas las neuronas de una determinada capa. Estos parámetros son llamados filtros, y a partir de aplicar la convolución a cada sector de los datos de entrada (sean de la entrada de la red o de una capa intermedia), extraen características locales que luego son combinadas sucesivamente generando características globales (Lecun & Bengio, 1995). Por lo tanto, dichos filtros pueden ser interpretados como características que crecen en complejidad con cada capa de la red. Comúnmente, entre cada capa de convolución se aplica un método llamado *pooling* que reduce la cantidad de neuronas y, por lo tanto, la resolución de los datos procesados, obteniendo invariancia ante la translación y mejorando la capacidad de generalización de la red.

Las CNN han sido utilizadas con éxito en tareas de predicción de series de tiempo en diversas investigaciones (Abdel-Hamid et al., 2013; Lee et al., 2009; Palaz et al., 2013). Más recientemente, este tipo de redes han alcanzado un rendimiento que representa el estado del arte en tareas que eran dominadas por las Redes Neuronales Recurrentes (RNN), como la síntesis de audio, modelado de lenguaje a nivel de palabra, traducción de máquina (Dauphin et al., 2017; Kalchbrenner et al., 2016; van den Oord &

Dieleman, n.d.). A partir de estos trabajos, se han implementado diferentes tipos de CNN para el modelado de secuencia, y, en especial, para la predicción en series de tiempo. A continuación se desarrolla un caso del uso de la CNN.

En Pelletier et al. (2019) se utilizan Series de Tiempo de Imágenes Satelitales (SITS, por sus siglas en inglés) en sistemas de clasificación que apuntan a obtener mapas actualizados y precisos de la cobertura terrestre de las superficies de la Tierra. La tarea de clasificación en este trabajo fue la de clasificar imágenes SITS, la cobertura terrestre de diferentes porciones de mapa en el este de Toulouse, Francia. En este trabajo se expresa el buen desempeño de métodos de clasificación tradicionales y se propone el uso de métodos de *Deep Learning* para mejorar dicho desempeño. Más específicamente, los investigadores estudiaron el aporte de las CNN y RNN en las tareas donde la componente temporal es aprovechada y propusieron el uso de CNN temporales. Las CNN temporales propuestas aplican la convolución de una dimensión, la temporal, por lo que pueden denotar como 1D-CNN. En resumen, en este trabajo se compararon modelos que no utilizan explícitamente el carácter secuencial (*Random Forest* y *Support Vector Machines*), RNN, y diferentes tipos de CNN cuyos filtros varían en relación a la dimensión en la que se lleva a cabo la convolución. En el caso de las imágenes satelitales la convolución se puede llevar a cabo en la dimensión temporal, en la dimensión espectral o en ambas. Los investigadores compararon también estos tres casos. Los resultados mostraron que las 1D-CNN que toman en cuenta ambas dimensiones obtuvieron un mejor desempeño en la tarea de clasificación llevada a cabo

2.3.2.4. Redes Temporales Convolucionales

Las Redes Temporales Convolucionales o *Temporal Convolutional Networks* -TCN, por sus siglas en inglés-, son un tipo de CNN con ciertas características que la hacen especialmente aplicables a tareas donde la entrada del modelo es una secuencia. (Bai et al., 2018) realizan una evaluación sistemática de arquitecturas convolucionales (TCN) y recurrentes. Específicamente, se comparan ambos tipos de modelos en tareas que han sido históricamente utilizadas como *benchmark* para comparar diferentes arquitecturas RNN. Los resultados muestran que la común asociación entre el modelado de secuencias y las RNN debe ser reconsiderada, ya que fueron las TCN las arquitecturas que alcanzaron el mayor desempeño en los *benchmarks*. Las TCN ya habían sido utilizadas en otros trabajos (Lea et al., 2017; van den Oord & Dieleman, n.d.) con ciertas variaciones. En (Bai et al., 2018) se implementa una versión de una TCN genérica. En dicha versión, se busca que las TCN puedan aprovechar las ventajas que presentan las CNN más simples, pero siendo adaptadas para el modelado de secuencias y la predicción de series de tiempo. Estos cambios introducen ciertas diferencias entre las TCN genéricas en (Bai et al., 2018) y las CNN tradicionales. Las diferencias más importantes son:

- Las TCN no utilizan capas de *pooling*, de forma que la dimensión de entrada es igual a la de salida.
- Las TCN siempre aplican la convolución de una sola dimensión (la del tiempo).
- Las convoluciones llevadas a cabo por las TCN son causales, es decir, las convoluciones nunca consideran información del futuro.
- Utilizan dilataciones para aumentar el campo receptivo exponencialmente y así poder cubrir largas dependencias en los datos secuenciales.

Las TCN propuestas en (Bai et al., 2018) están compuestas por capas convolucionales densas, esto es, todas las capas de la red son convolucionales, con excepción de las capas de activación y aquellas utilizadas para regularización. Las TCN están organizadas en bloques residuales. Cada bloque residual está compuesto por dos bloques formados por: una capa convolucional causal con tamaño de kernel k y una dilatación, una capa de activación Relu, una capa de regularización -la técnica de regularización por normalización utilizada varía entre *Batch Normalization*, *Layer Normalization* y *Weight Normalization* según la implementación, aunque en (Bai et al., 2018) se propone la última-, y un *Dropout* espacial. La diferencia entre una capa de *Dropout* normal y una espacial es que la última le da el valor de 0 a mapeos de característica enteras, es decir, al resultado completo de la aplicación de un filtro sobre la entrada. Luego la salida de cada bloque utiliza conexiones residuales en las que la entrada y la salida se suman. Esta suma puede no ser posible si la entrada y la salida del bloque residual tienen dimensiones diferentes, en cuyo caso, se le aplica al vector de entrada una convolución de dimensión 1×1 para hacer coincidir las dimensiones de la entrada y la salida. Esta inconsistencia entre las dimensiones de entrada y la salida de los bloques residuales solo se da en la primera entrada, es decir, en el input de la red, ya que, todas capas internas tienen el mismo tamaño y en todas ellas se aplica *zero padding*.

Un concepto muy importante para las TCN es el de campo receptivo, o *receptive field*, que indica la cantidad de componentes de la entrada a los que el filtro de una capa convolucional arbitraria tiene acceso. Esto es, el campo receptivo indica de cuantos componentes de la entrada un filtro está utilizando información al llevar a cabo la convolución. A diferencia de las CNN comunes, donde el campo receptivo aumenta linealmente con la cantidad de capas convolucionales -aunque esto puede variar a partir del uso de técnicas como *stride* o *max-pooling*- (Goodfellow et al., 2016) el uso de las dilataciones en las TCN permite que el campo receptivo aumente exponencialmente con cada bloque residual. Esto se da gracias a que las dilataciones se determinan a partir de potencias de 2 (por ejemplo: 1, 2, 4, 8, etc.). Esto permite que las TCN puedan tener una historia larga con pocas capas computacionales, pudiendo reducir la cantidad de parámetros necesarios a ajustar.

En las TCN, el campo receptivo es un hiperparámetro a ajustar. Este hiperparámetro no se ajusta directamente, sino que varía a través de la cantidad de dilataciones y el tamaño del filtro. Es importante que el campo receptivo sea lo suficientemente largo como para cubrir toda la secuencia de entrada (Bai et al., 2018).

Hay diversas ventajas de las TCN sobre las RNN que se suman al mejor desempeño en diferentes *benchmark* (Bai et al., 2018):

- A diferencia de las RNN donde las predicciones para un *timestep* futuro tienen que esperar a que se hayan llevado a cabo todas las predicciones anteriores, las convoluciones pueden llevarse a cabo en paralelo dado que se utiliza el mismo filtro para todas ellas. Esto permite que tanto el entrenamiento como la evaluación de la red pueda llevarse a cabo más rápidamente.
- Es sencillo ajustar el campo receptivo de las TCN a partir de la variación del tamaño del filtro y el valor de las dilataciones, lo que permite un mejor control del tamaño de la memoria del modelo, permitiendo la fácil adaptación para diferentes dominios.
- Como en las TCN el camino de *backpropagation* es diferente de la dirección temporal de la secuencia, éstas no sufren de los llamados problemas *exploding/vanishing gradient*.
- Al entrenarse, las RNN deben mantener en memoria los resultados parciales de cada capa, pudiendo llegar a requerir mucha memoria, sobre todo para secuencias largas. Este hecho no ocurre en las TCN porque los filtros son compartidos por toda la capa.

- Las TCN pueden, al igual que la mayoría de las RNN, recibir secuencias de longitud variable.

En (Bai et al., 2018) también se muestran dos desventajas de la utilización de las TCN:

- En el momento de la evaluación, las TCN deben mantener en memoria la secuencia entera para realizar una predicción. En cambio, las RNN solo necesitan mantener en memoria un *timestamp* de la totalidad de la secuencia.
- Si se desea cambiar de dominio a partir de un modelo ya entrenado, por ejemplo, para realizar *fine-tuning*, el campo receptivo requerido por la nueva tarea a realizar puede ser mayor que el campo receptivo del modelo ya entrenado, por lo que el modelo puede no tener un buen desempeño en la nueva tarea.

A continuación, se desarrolla un ejemplo de la utilización de las TCN.

En (Lea et al., 2016) se propone un enfoque unificado al problema de la segmentación de acciones en videos. Comúnmente, el enfoque para este problema se divide en dos partes. Primero se lleva a cabo el proceso de extracción de características de cada imagen, con el fin de codificar información espacio temporal. Esto puede realizarse utilizando un CNN. Luego, las características obtenidas son utilizadas por una RNN capaz de capturar relaciones temporales de alto nivel.

En este trabajo, los investigadores proponen utilizar Redes Temporales Convolucionales para unificar el proceso anterior. Las redes diseñadas fueron puestas a prueba en 3 *datasets* públicos de segmentación de actividades basados en videos o sensores. Las TCN obtuvieron igual o mayor desempeño que el estado del arte, logrando capturar jerárquicamente relaciones en escalas de tiempo de nivel bajo, intermedio y alto. Además, los investigadores reportan que se requiere solo una fracción de tiempo para entrenar a las TCN en comparación con el necesario para entrenar a las RNN.

2.4. Predicción de actividades

La predicción de actividades es un área de investigación que consiste en hipotetizar acerca de las actividades en las que un sujeto va a estar involucrado en el futuro basándose en datos de sensores. No debe confundirse al área de predicción de actividades con el de reconocimiento de actividades. La diferencia entre ambas es que, en la segunda, se utilizan datos de sensores tanto del pasado como del presente y se los mapea con un tipo de actividad que está ocurriendo en el momento actual. Mientras que en la primera no se tiene ningún dato de sensor sobre la actividad que va a llevarse a cabo en el futuro y que es la que se quiere predecir. La predicción de actividades puede ser utilizada, por ejemplo, para anticipar las actividades de una persona en una casa, facilitando la automatización de dispositivos IOT, como calentar la casa antes de que la persona llegue del trabajo.

En el Capítulo 7 de *Activity learning Discovering, recognizing, and predicting human behavior from sensor data* (Cook & Krishnan, 2015) se le da un enfoque al área de Predicción de Actividades basado en algoritmos de compresión para la predicción de actividades. Más específicamente, los algoritmos de compresión que se propone utilizar son LZ78 y ALZ. Ambos algoritmos pueden ser interpretados como tipos de modelos de Markov. En este trabajo, se analizan las ventajas de cada uno de estos algoritmos para la predicción de secuencias de una manera general. Es decir, se generalizan para la predicción de símbolos, sean cuales sean (por ejemplo, símbolos que representan ubicaciones).

Similarmente a lo expuesto anteriormente, la predicción de actividades también puede ser estudiada como la predicción de cuando cierta actividad va a volver a suceder. En este caso, a diferencia de predecir el siguiente elemento de una secuencia, se busca hallar la cantidad de unidades de tiempo luego de la cual cierta actividad se repetirá.

Resumen

En este capítulo se introdujeron los conceptos más importantes y necesarios a la hora de comprender lo desarrollado en este trabajo de tesis. En la [Sección 2.1](#) se explicó, de manera general, que es la Inteligencia Artificial. En la [Sección 2.2](#) se definieron los conceptos más importantes relacionados con *Machine Learning*. En ella, se introdujeron los tipos de tareas y se argumentó por qué el problema de PCSF es de tipo regresión, se expuso cómo son entrenados y evaluados los modelos y otros conceptos importantes como las técnicas de regularización. En la [Sección 2.3](#), se expusieron los modelos de *Deep Learning* que fueron utilizados en esta tesis. Por último, en la [Sección 2.4](#) se presentó el área de predicción de actividades, a la cual pertenece el problema tratado en esta tesis.

Capítulo 3: Trabajos relacionados

En este capítulo se introducen los trabajos relacionados al problema abordado en esta tesis. La Predicción del Comportamiento Sedentario Futuro es un problema complejo y puede ser descompuesto en diferentes aristas que lo caracterizan. En la [Sección 3.1](#) se presentan trabajos relacionados al monitoreo y modelado del comportamiento humano, en la mayoría de los casos en los cuales se utilizan técnicas de *Machine Learning*. Mientras tanto, en la [Sección 3.2](#) se describen los trabajos relacionados que abordaron el problema de la PCSF a través de diferentes técnicas.

3.1. Monitoreo y modelado del comportamiento humano utilizando smartphones

A partir de que los dispositivos móviles y “wearables”, como *smartphones*, comenzaron a volverse populares, se desarrollaron numerosos estudios para encontrar patrones de uso que permitan relacionar, inferir y predecir diferentes tipos de comportamientos humanos en el contexto de la salud y el bienestar.

Harari et al. (2017), en su investigación, parten del hecho de que los avances recientes en tecnología de sensado móvil revolucionaron la evaluación del comportamiento al permitir el seguimiento no obstrusivo y continuo del comportamiento a través de dispositivos móviles (como acelerómetros y micrófonos) incluidos en smartphones.

Para examinar la viabilidad de usar métodos móviles de sensado para obtener estilos de vida diarios de comportamiento de estudiantes, los investigadores presentaron un estudio utilizando un *dataset* generado por una aplicación de sensado en smartphones llamada *StudentLife* para medir, de forma objetiva, la actividad natural de estudiantes y comportamientos sociales a lo largo de un ciclo académico de 10 semanas de duración. Al hacerlo, abordaron una brecha existente en la literatura de *mHealth* al proporcionar una descripción de los patrones de granularidad fina de estabilidad y cambio que caracterizan las conductas relacionadas con la salud de los estudiantes durante un período académico.

Los resultados de este estudio indican que, a medida que el ciclo académico progresa, los estudiantes tendían a participar en más comportamientos sedentarios y relacionarse menos con otros estudiantes, lo que puede estar relacionado a la necesidad de concentrarse en estudiar y preparar exámenes. En conclusión, Harari et. al. demostraron la viabilidad de utilizar métodos objetivos de sensado basados en smartphones para realizar el seguimiento de aquellos comportamientos relacionados a la salud en el contexto de la vida cotidiana de estudiantes. Los investigadores identificaron varias correlaciones significativas entre los datos objetivos de los sensores de los dispositivos móviles y el bienestar mental y el rendimiento académico.

Gong et al. (2019), en una colaboración entre ingenieros y psicólogos, utilizaron los datos del acelerómetro y del GPS de smartphones en los momentos antes, durante y después de realizar llamadas y enviar mensajes de texto. La hipótesis de esta investigación fue que los movimientos sutiles (medidos con acelerómetros) covarían con el nivel de ansiedad social y el contexto social. Por ejemplo, las manos pueden temblar más cuando el estudiante está en un ambiente público, debido a que la probabilidad de evaluación negativa por otros es alta. Los autores denominan a estos movimientos como comportamientos dinámicos del usuario.

Para realizar el estudio, 52 estudiantes, con edad media de $20,5 \pm 3,8$, fueron reclutados del grupo de participantes del departamento de psicología de la Universidad de Virginia. De cada participante se tomaron los datos del acelerómetro, el GPS y datos relacionados a las llamadas y a los mensajes de texto. Luego, modelaron los datos como un sistema lineal dinámico. Para ello, primero utilizaron Foursquare⁵ para obtener los Puntos de Interés a partir de los datos crudos del GPS de cada participante, y luego, los dividieron en sus locaciones semánticas (hogar, transporte, vida personal, lugar de comida). Respecto a los datos sociales (llamadas y mensajes de texto), los autores analizan por separado períodos de

⁵ <https://enterprise.foursquare.com/products/places>

observación α_1 minutos antes y después de una llamada, para detectar, por ejemplo, cómo varían los indicadores de ansiedad de un sujeto cuando debe realizar una llamada y no recibe una respuesta a la llamada realizada. Además, antes de que comenzase el experimento, cada estudiante fue evaluado de acuerdo a la Escala de Ansiedad de Interacción Social (SIAS, por sus siglas en inglés).

Los resultados obtenidos en el estudio mostraron una relación estadística significativa entre los datos de los sensores de *smartphones* y la ansiedad social de los estudiantes. Estos resultados sugieren que es posible integrar datos de acelerómetros y de GPS para identificar marcadores de comportamiento asociados con la ansiedad social en varias locaciones. Por lo anterior, este trabajo abre nuevas posibilidades para monitorear pasivamente marcas de comportamiento de ansiedad social a través de la integración de los datos del acelerómetro y los de GPS.

Kanjo et al. (2019) adoptó un enfoque de *Deep Learning* para la clasificación de emociones a partir de datos de diferentes tipos de sensores portables. La clasificación de emociones a menudo requiere del modelado de tipos de datos de entrada de diferentes modalidades, como señales fisiológicas, datos ambientales, video y datos de movimiento y ubicación. Este problema ya había sido abordado desde la utilización de algoritmos del área de *Machine Learning*, pero, aunque los procesos de ingeniería de características a menudo integrados en estos algoritmos son beneficiosos para la modelización de emociones, heredan algunas limitaciones críticas que pueden dificultar el desarrollo de modelos fiables y precisos.

En este trabajo, los investigadores adoptaron un enfoque de aprendizaje profundo para la clasificación de emociones a través de un proceso iterativo mediante la adición y eliminación de un gran número de señales de sensores de diferentes modalidades. El conjunto de datos fue recogido en un estudio del mundo real desde teléfonos inteligentes y dispositivos portátiles. Dicho conjunto de datos combina la interacción local de tres modalidades de sensores: en el cuerpo, el medio ambiente y la ubicación en un modelo global que representa la dinámica de la señal junto con las relaciones temporales de cada modalidad. El enfoque propuesto emplea una serie de algoritmos de *Deep Learning* que incluyen un enfoque híbrido usando una CNN-LSTM en los datos crudos del sensor. Utilizando dichos datos como entrada para las redes neuronales se consigue eliminar las necesidades de extracción manual de las características. Los resultados muestran que la adopción de enfoques de aprendizaje profundo es efectiva en la clasificación de la emoción humana cuando se utiliza un gran número de sensores de entrada y que los modelos híbridos superan a la red neuronal profunda tradicional totalmente conectada. Además, los modelos híbridos superan los algoritmos de tipo *Ensemble* previamente desarrollados que utilizan la ingeniería de características para entrenar el modelo

Wu et al. (2018) utilizaron redes de encuentro basadas en los registros de encuentros Bluetooth para monitorear el nivel de estrés de los usuarios. Bluetooth es una red de corto alcance comúnmente disponible en los dispositivos móviles. Al mismo tiempo, la red Bluetooth puede ser utilizada como un sensor móvil ampliamente disponible que puede detectar el entorno social de un individuo basado en la proximidad física. Los autores señalan que en la bibliografía existe una falta de ingeniería sistemática de características, así como también evidencia de su poder predictivo en problemas de salud mental. En este trabajo se extrajeron características de los datos de encuentro de Bluetooth de los teléfonos inteligentes de 49 estudiantes a lo largo de 66 días, provenientes del *dataset StudentLife*. El análisis de correlación

sugiere que las características generadas pueden explicar la varianza en el estrés significativamente mejor que los modelos nulos.

En este trabajo los autores evaluaron la predictibilidad del nivel de estrés que podía conseguirse utilizando las redes de encuentro de Bluetooth, más otros datos tales como la calidad del sueño y otros datos de sensores como GPS, acelerómetro, uso del teléfono, etc. Sus predictores pueden ser clasificados de acuerdo diferentes características: predecir el nivel de stress o el cambio del mismo, utilizar la técnica de evaluación *Leave-One-Subject-Out* (LOSO) o *Leave-one-observation-out* (LOOO), predecir el presente o el futuro. Los modelos utilizaron la técnica *Random Forest* en todos los casos. Los resultados indican que las características extraídas de las redes de encuentro representan una mejora en términos de predictibilidad del nivel de estrés y del cambio de éste en el presente, pero no en el futuro. Estos resultados proveen evidencia del valor potencial de mejora al incorporar las redes de encuentro basadas en datos de redes Bluetooth a aplicaciones de monitoreo de estrés.

Zia et al. (2016) destacan que utilizar los datos de dispositivos portátiles puede ayudar a que pacientes con Parkinson vivan de forma más segura e independientemente. Partiendo de esta idea, el objetivo de este trabajo consistió en determinar la viabilidad de utilizar redes neuronales para predecir la congelación de la marcha, que es un síntoma del Parkinson. Particularmente, los investigadores usaron una clase de red neuronal conocida como Redes Recurrentes en capas (LRN). El *dataset* utilizado para entrenar las redes fue uno público donado por el Repositorio de Aprendizaje Profundo de la Universidad de California y comprende un registro de los datos de movimiento (3 acelerómetros) de 3 sujetos, donde los momentos en los cuales se predice el Congelamiento de la Marcha están anotados como un timestamp. El trabajo no especifica las características obtenidas a partir de los datos del acelerómetro, por lo que se supone que utilizaron un modelo end-to-end, en el que al igual que en otros trabajos ya expuestos, son las primeras capas de las redes quienes se ocupan de aprender a generar las características. Los parámetros que fueron variando en las diferentes redes neuronales testeadas fueron la cantidad de capas, la cantidad de neuronas por capas, el tamaño de *batch* y el factor de down sampling -esto es, la cantidad de ejemplos de entrenamiento que se quitaron del *dataset*-.

Los resultados mostraron que las redes que mejor precisión y recall tuvieron son aquellas de una sola capa, baja cantidad de neuronas y alto *down sampling*. Los autores propusieron, como trabajo futuro, utilizar otros tipos de datos para que se complementen con los datos de movimiento.

A partir de los trabajos expuestos es posible notar el potencial que tienen los dispositivos ubicuos equipados con diferentes tipos de sensores, como lo son los *smartphones*, para permitir el monitoreo y modelado del comportamiento de los usuarios que los portan. Estos tipos de comportamiento pueden ser de lo más variados, desde aquellos relacionados a la salud mental hasta otros relacionados a la salud física. La gran disponibilidad de datos heterogéneos y de diferentes poblaciones (por ej: un grupo de estudiantes) ofrece una clara oportunidad para la evaluación de diferentes técnicas del área de *Deep Learning*.

En esta tesis, motivado por lo expuesto en el párrafo anterior, se busca sacar provecho de la ubicuidad de los dispositivos móviles, la heterogeneidad de sus sensores y la facilidad para almacenar y acceder a los datos producidos por ellos. Para ello, se toma el *dataset StudentLife* para procesar los datos disponibles y generar modelos que representen a los usuarios, buscando permitir, a partir de ellos, predecir el comportamiento sedentario futuro a través de su gasto energético.

3.2. Predicción del Comportamiento Sedentario Futuro

El comportamiento sedentario puede ser clasificado como un tipo de tarea de predicción de actividades. En esta sección se presentan los trabajos relacionados al área en la que se contextualiza esta tesis, es decir, la predicción del comportamiento sedentario. Cabe aclarar que los trabajos que buscan resolver el problema de predecir el comportamiento sedentario futuro, son escasos y hacen suposiciones diversas a la hora de tratar de resolverlo.

Para realizar la búsqueda de trabajos relacionados a la predicción de comportamiento sedentario futuro se utilizaron las librerías *IEEE Xplore*⁶, *ACM*⁷, *Wiley*⁸, *ScienceDirect*⁹ y *Scopus*¹⁰ utilizando los motores de búsqueda propios de cada una. Hasta el momento, Q. He and E. Agu son los únicos autores que pueden considerarse como trabajos relacionados a la predicción del comportamiento sedentario a partir de los resultados de dicha búsqueda. A continuación se analizan todos los trabajos de los ya nombrados autores, quizás con más detalle debido a la relación cercana con el trabajo motivo de esta tesis.

((*sedentary OR sedentarism*) AND *behaviour*) OR (*physical activity*) OR (*physical activity Intensity*) AND *prediction*

A diferencia del enfoque propuesto en el presente trabajo, Q. He y E. Agu determinaron el nivel de sedentarismo como el porcentaje de registros de actividad estacionarios que se tomaron de los datos de actividad física de cada usuario en cada hora, con respecto a los demás tipos de registros de actividad (corriendo, caminando, etc.). En este primer estudio Q. He y E. O. Agu (2016a) propusieron un algoritmo de dominio de frecuencia para detectar patrones sedentarios recurrentes a partir de los datos de los usuarios. Este trabajo buscó ajustar funciones periódicas (seno, coseno) al porcentaje de registros de actividad estacionaria de cada hora y de cada usuario, tratando así de identificar los patrones subyacentes en su actividad física.

En este trabajo, como en los demás, los autores utilizaron el *dataset StudentLife* para validar sus hipótesis. Con respecto al procesamiento de los datos, primero, discretizaron los datos de actividad física en *buckets* de distintas escalas de tiempo, ya que, en dicho *dataset*, cada registro está acompañado de un timestamp. Luego, para cada *bucket*, computan lo que ellos llaman nivel de sedentarismo, que definen como el porcentaje del total de actividades físicas etiquetadas como estacionarias pertenecientes a ese determinado *bucket*.

A partir del nivel de sedentarismo calculado, los investigadores llevaron a cabo un análisis sobre el nivel de sedentarismo a lo largo de las horas del día y de los días de la semana, que les permitió observar en qué momentos del día los estudiantes eran más sedentarios y cuánto difieren entre sí los patrones de comportamiento sedentario de los diferentes estudiantes y a lo largo del ciclo lectivo.

En base a esto, los autores propusieron un modelo computacional capaz de capturar los patrones recurrentes a diferentes escalas de tiempo. Estos modelos utilizan el nivel de sedentarismo considerándolo como una señal periódica, donde cada elemento de dicha señal corresponde a un *bucket* de un

⁶ <https://ieeexplore.ieee.org/Xplore/home.jsp>

⁷ <https://dl.acm.org/>

⁸ <https://www.wiley.com/en-ar>

⁹ <https://www.sciencedirect.com/>

¹⁰ <https://www.scopus.com/home.uri>

determinado usuario. Para generar los modelos, los autores aplicaron una transformación de *Fourier* sobre la señal periódica, descomponiéndola en una suma de funciones coseno y tomando las k primeras funciones con el fin de minimizar el error cuadrado y eliminar el ruido. Los modelos son generados para cada uno de los usuarios y a diferentes escalas de tiempo, es decir, con diferentes tamaños de *bucket*.

Los investigadores obtuvieron resultados muy variados de acuerdo a los diferentes usuarios y escalas de tiempo utilizadas. Algunos modelos obtuvieron un error cuadrado muy bajo lo que significó que el usuario al cual corresponde el modelo presenta comportamientos sedentarios de forma recurrente y periódica. Sin embargo, otros modelos no lograron conseguir un error cuadrado bajo, lo que mostró que el usuario al cual corresponde el modelo no presenta comportamientos sedentarios de forma recurrente y periódica.

En un segundo estudio, He y Agu (2016b) resaltaron que los teléfonos inteligentes pueden monitorear los comportamientos sedentarios llevados a cabo por sus usuarios, así como también pueden monitorear el contexto en que dichos comportamientos sedentarios ocurren. En este trabajo, los investigadores exploraron si es posible utilizar los datos de contexto para predecir el comportamiento sedentario futuro. Para probar sus hipótesis, He y Agu utilizaron el *dataset* público *StudentLife*, que contiene datos de actividad física y datos del contexto de 49 estudiantes universitarios a lo largo de 10 semanas. El *dataset StudentLife* ha sido utilizado por todos los trabajos de investigación que se encuentran en esta sección. Además, es el *dataset* utilizado en esta tesis, por lo que su estructura y contenido será detallado en la [Sección 4.2](#).

En el *dataset StudentLife*, todo datos, ya sea de actividad física o de contexto están asociados a un *timestamp*. La frecuencia de muestreo depende del tipo de dato. Con el fin de normalizar los datos disponibles, el paso siguiente que llevaron a cabo los investigadores fue el de discretizar el tiempo en *buckets* de una hora. Es decir, todos los datos de sensado de los usuarios fue asociado a un hora y día específico. En total, He y Agu obtuvieron como resultado 65,601 *buckets*.

Luego, a partir de los datos de actividad física, los investigadores computan, para cada *bucket*, lo que ellos llaman nivel de sedentarismo, que definen como el porcentaje de actividades físicas etiquetadas como estacionario. Esta forma de cuantificar el comportamiento sedentario será discutida más adelante en la [Sección 4.1](#). Además, discretizaron el nivel de comportamiento sedentario en 3 clases, cada una asociada a un rango específico: muy sedentario (99,99-100,00), sedentario (90,52 - 99,99) y menos sedentario (0 - 92,52). Cabe resaltar que dichos rangos no fueron determinados con ningún tipo de base teórica, sino que fueron así definidos para obtener la misma cantidad de registros de actividad para cada clase.

A continuación, los investigadores analizan los datos de contexto que provee el *dataset* utilizado y su relación con el nivel de sedentarismo computado. A partir de este análisis, los investigadores definen y calculan, en total, 22 variables de contexto asociadas a cada *bucket*, como el día de la semana y la actividad física que más llevó a cabo un determinado usuario en una hora específica.

Por último, los investigadores intentaron predecir el nivel de sedentarismo futuro, es decir, si en la siguiente hora el usuario iba a ser muy sedentario, sedentario o menos sedentario, a partir de las variables de contexto que definieron. Para realizar esta tarea, y dado que la variable que se intenta predecir es categórica, los autores de este trabajo utilizaron los modelos Naive Bayes y Regresión Logística. Además, para validar los resultados utilizaron la técnica de validación cruzada. Naive Bayes alcanzó una precisión de 59,7% y recall de 64,2%, mientras que Regresión Logística obtuvo una precisión de 60,8% y recall de 62,6%. Como último experimento, los investigadores agregaron al conjunto de datos de entrenamiento el

nivel de sedentarismo actual, mejorando ligeramente los resultados. Como conclusión de estos resultados, los autores resaltaron que para predecir si el usuario iba a ser muy sedentario en la hora siguiente, consiguieron una precisión de 73,1% y recall de 87,7%.

En otro estudio He y Agu (2016c) destacaron que el tiempo, el ritmo diario y los hábitos sedentarios del pasado son 3 determinantes del comportamiento sedentario. Los investigadores propusieron entonces un método para descubrir automáticamente los 3 determinantes analizando los registros de las series de tiempo de datos obtenidos de sensores de *smartphones*. Dicho método analiza el historial de comportamiento de un usuario para encontrar patrones temporales que puedan ser utilizados para predecir el comportamiento sedentario futuro.

En este trabajo, los investigadores suponen que el comportamiento sedentario de los usuarios está correlacionado linealmente en el dominio temporal de tal forma que la probabilidad de que un usuario tenga un comportamiento sedentario en el siguiente período de tiempo depende de su comportamiento actual y pasado. Para hallar estos patrones, los autores utilizan Modelos Autorregresivos junto al Método de Máxima Entropía para estimar los coeficientes de dichos modelos.

Para evaluar los modelos propuestos los autores utilizaron el *dataset StudentLife*. Al igual que en los trabajos anteriores, discretizaron las series de tiempo en *buckets* de una determinada duración, calculando para cada una el porcentaje de registros etiquetados como estacionarios. Luego, generaron en total 324 modelos variando el orden del Modelo Autorregresivo (cantidad de información del pasado que el modelo utiliza) y el tamaño de *bucket* (1, 2, 3 o 6 horas). Por último, calcularon el Error Cuadrado de cada modelo y para cada usuario.

Los resultados mostraron que los modelos que tuvieron un mayor MSE fueron aquellos que utilizaban *buckets* de 6 horas. Los investigadores hipotizaron que esto se debió a que la influencia de factores aleatorios es menor cuanto más grande sea el tamaño de *bucket*.

En 2017, He y Agu (2017) publicaron un último trabajo de investigación en el cual plantearon un nuevo enfoque para poder predecir el comportamiento sedentario de una persona. En este trabajo, los autores propusieron modelos que están basados en el concepto de análisis de ritmo, propuesto por Lefebvre, que postula que muchos comportamientos humanos siguen ritmos naturales. Por lo tanto, en este trabajo los investigadores se concentraron en detectar los ritmos predominantes de comportamiento sedentario y en modelar los ritmos cíclicos y los ritmos lineales utilizando funciones cíclicas y funciones lineales, respectivamente. Como ejemplo de ritmo cíclico, los autores ejemplifican con una persona que se sienta en un sillón todos los días a la misma hora. Como ejemplo de ritmo lineal, los autores ejemplifican con una persona que se acuesta exhausta luego de llevar a cabo un ejercicio de alto requerimiento físico.

Los autores propusieron dos tipos de modelos, buscando modelar los tipos de ritmos previamente mencionados. Por un lado, se denomina a los modelos que utilizan funciones cíclicas como libres de la historia (o *history-free*), ya que una vez determinadas las funciones cíclicas no es necesario guardar los datos de gasto energético del usuario para predecir su siguiente comportamiento sedentario. Por otro lado, se denomina a los modelos que utilizan funciones lineales como dependientes de la historia (o *history-dependent*) ya que estos deben guardar los datos de gasto energético del usuario aun cuando el modelo ya ha sido generado. Para generar los modelos libres de historia los autores utilizan un procedimiento similar al que siguieron en (He & Agu, 2016a), mientras que para generar los modelos dependientes de historia utilizan un procedimiento similar al que siguieron en (He & Agu, 2016c). Los

autores también propusieron modelos híbridos, que combinan los modelos libres de historia y los modelos dependientes de historia. Analíticamente, los modelos híbridos se obtienen como una suma ponderada de los dos tipos de modelos, donde los primeros buscan calcular los patrones causados por ritmos cíclicos, mientras que los segundos buscan capturar los patrones dentro de esos ciclos.

En total He y Agu generaron 49248 modelos híbridos para los 49 participantes del *dataset*. Para cada uno de estos modelos, los autores debieron hallar los valores óptimos de l (el orden de los Modelos Autorregresivos) y k (la cantidad de funciones periódicas utilizadas). Los resultados mostraron que el modelo de base, donde k vale 0 y l vale 0, fue el óptimo en casi la mitad de los modelos generados, es decir, el que menos MSE obtuvo. Estos resultados contradicen su hipótesis inicial sobre que el comportamiento sedentario puede ser explicado a partir de los ritmos cíclicos y lineales de las personas. Los autores indicaron que este resultado era de esperar debido a las horas en que los usuarios duermen, en las cuales los modelos pueden simplemente predecir el promedio de los resultados de las horas anteriores.

Resumen

En este capítulo se presentaron los trabajos relacionados al problema abordado en esta tesis. Primero, en la [Sección 3.1](#) se expusieron trabajos donde se muestra la utilidad de utilizar modelos de Machine Learning para el monitoreo y modelado del comportamiento humano. Los trabajos fueron muy variados, desde la identificación de los niveles de estrés hasta la predicción de la congelación de la marcha en pacientes con Parkinson. En todos los trabajos se utilizaron datos obtenidos a partir de los sensores de *smartphones* para entrenar los modelos de *Machine Learning*. Luego, en la [Sección 3.2](#) se presentaron todos los trabajos que abordaron el problema de la Predicción del Comportamiento Sedentario Futuro. Como ya ha sido mencionado, todos ellos pertenecen a los mismos autores. A pesar de ser el mismo problema, en los diferentes trabajos se lo define de diferentes maneras desde el punto de vista técnico. En algunos casos se lo trata como un problema de regresión, y en otros, como un problema de clasificación. En todos ellos, el cálculo de la variable objetivo se realiza a partir del porcentaje de tiempo estacionario. En cambio, en esta tesis se propone una mejora a dicho cálculo utilizando el valor de MET estimado de las diferentes actividades.

Capítulo 4: Predicción del Comportamiento Sedentario Futuro

En este Capítulo se exponen las cuestiones más importantes sobre cómo se aborda el problema de PCSF en esta tesis. Primero, en la [Sección 4.1](#) se presenta el enfoque propuesto para abordar la tarea de PCSF. Para ello, se describen los aspectos a partir de los cuales se buscan llevar a cabo los experimentos y se describe detalladamente el *dataset* a ser utilizado. A continuación, en la [Sección 4.2](#) se describe el *dataset* utilizado. Finalmente, en la [Sección 4.3](#) se describen exhaustivamente las cuestiones relacionadas a la implementación de los experimentos, que incluyen el análisis de los datos disponibles en el *dataset* y las inconsistencias encontradas en él, la descripción de las etapas del procesamiento del *dataset*, la técnica elegida para validar los modelos y aquella para realizar el tuning de los mismos y el diseño de los experimentos.

4.1. Enfoque

El problema que se aborda en esta Tesis es el de predecir el comportamiento sedentario futuro mediante modelos de *Deep Learning* entrenados con datos de *Lifelogging*. El gasto energético es medido en MET, que es la medida estándar en la comunidad científica que estudia la salud en relación con la actividad física. En el ámbito de la salud, se ha llegado a un acuerdo entre los investigadores en determinar cómo actividad sedentaria a toda aquella actividad cuyo MET asociado sea menor o igual a 1,5. En el contexto de este trabajo, se define la Predicción del Comportamiento Sedentario Futuro (PCSF) como la tarea de predecir el valor de MET de la actividad física realizada por un usuario en un futuro próximo en pos de estudiar su comportamiento sedentario futuro.

En el marco del aprendizaje de máquina, al ser el MET un valor continuo, el problema a resolver es una tarea de regresión. Dentro de los algoritmos de aprendizaje profundo, las Redes Neuronales Recurrentes (RNN), las Redes Neuronales Convolucionales (CNN) y las Redes Temporales Convolucionales (TCN) son especialmente aptas para el problema a tratar porque los datos registrados de la actividad del usuario están organizados en series de tiempo, en las que la secuencialidad es importante. Por lo tanto, todos estos tipos de redes neuronales serán evaluados y comparados.

En este trabajo, se buscará predecir el valor de MET para un usuario en el siguiente intervalo temporal. Es decir, dados los datos del pasado se utilizarán Redes Neuronales para hallar el gasto energético en el futuro. El objetivo principal es evaluar la factibilidad de utilizar redes neuronales capaces de aprender, a partir de una secuencia de datos de la actividad de un usuario, patrones de comportamiento que sean útiles para el problema de PCSF.

Además de evaluar diferentes tipos de redes neuronales, se evaluarán diferentes modelos. Dichos modelos determinan, por un lado, el intervalo de tiempo del cual la arquitectura toma datos y partir del cual es entrenada y realiza las predicciones y, por otro lado, qué datos utiliza la red neuronal para ser entrenada en el contexto de un *dataset* multi-usuario.

Para seguir con el análisis de la tarea de PCSF se definen a continuación las características esenciales que debe poseer el *dataset* a utilizar:

- Es de tipo *time-series*: los datos están organizados secuencialmente y en orden temporal. De esta forma cada registro del *dataset* puede ser asociado a un *timestamp* particular. De preferencia, dichos registros deberían estar igualmente espaciados en el tiempo.
- Poseer datos heterogéneos: el *dataset* posee información heterogénea sobre cada usuario. De esta forma, dicho *dataset* puede ser descompuesto en varios *datasets*, donde cada uno de ellos contiene la información de un tipo de sensor.
- Tener una larga duración: como se busca obtener patrones de comportamiento sedentario, es importante que el registro del *dataset* vaya desde semanas a meses. De esta forma, se puede evaluar la efectividad de los modelos, no solo para aprender patrones de comportamiento sedentario, sino que también para predecir cambios en dichos patrones.

A partir de las características anteriores, los tipos de modelos que se propone evaluar son:

- Por los usuarios utilizados:

- Personal: en este tipo de modelo tanto los datos utilizados para entrenamiento como aquellos utilizados para testeo pertenecen al mismo usuario. Se pretende aprender modelos particulares para cada usuario del *dataset*.
- Impersonal: en este tipo de modelo se utilizan los datos disponibles de todos los usuarios menos uno para entrenar el modelo y se utilizan datos del usuario restante para testearlo. Se pretende aprender un modelo general, que pueda ser útil para predecir el comportamiento de usuarios nuevos.
- Por el *dataset*:
 - Número de *lags*: define la cantidad de retrasos que son añadidos al *dataset*. En otras palabras, establece cuantos *buckets* del pasado estarán a disposición del modelo para predecir el valor de la variable objetivo.
 - Períodos: define el espacio temporal entre cada *lag* añadido al *dataset*.
 - Granularidad: define el tamaño del intervalo temporal en los que se divide el *dataset*.
- Por las modelos de *Deep Learning*:
 - MLP
 - CNN
 - RNN
 - TCN

4.2. Descripción del *dataset*

Para validar la eficiencia de los modelos propuestos se utilizó el *dataset StudentLife* (Wang et al., 2014). Este *dataset* fue recolectado a partir de la aplicación de sensado continuo *StudentLife*. Los participantes en este estudio fueron 48 estudiantes a lo largo de 10 semanas en la primavera de 2013. Más específicamente, el *dataset* se recogió de 30 estudiantes de pregrado y 18 graduados. De todo el grupo de estudiantes, 38 eran varones y 10 mujeres. Del grupo de estudiantes de pregrado, 2 eran de primer año, 14 de segundo año, 6 de tercer año y 8 de cuarto año. También hubo 13 estudiantes de maestría de primer año y 1 de segundo año, y 3 estudiantes de doctorado. Los participantes fueron racialmente diversos, con 23 caucásicos, 23 asiáticos y 2 afroamericanos.

4.2.1. Tipos de datos del *dataset StudentLife*

Los tipos de datos disponibles en el *dataset StudentLife* son los siguientes:

- *App Usage*: contiene información sobre el uso de las aplicaciones de los smartphones.
- *Calendar*: contiene información sobre el calendario de los usuarios. Es decir, cada registro indica una fecha en la que el usuario tiene un evento en el calendario -no especifica el tipo de evento-.
- *Call-log*: contiene información sobre las llamadas producidas desde el teléfono del usuario, así como su duración.
- *Dinning*: contiene información sobre las compras alimenticias dentro del campus universitario de la Universidad de Dartmouth.

- *Education*: contiene información diversa de los diferentes cursos a los cuales los estudiantes asistieron a lo largo del estudio. Esto incluye los cursos a los cuales cada estudiante estaba inscripto, información detallada de cada curso, las fechas límites de cada estudiante, así como también el puntaje obtenido en cada asignatura.
- *EMA (Ecological momentary assessments)*: contiene información sobre diversas y pequeñas encuestas que han sido respondidas por los usuarios a lo largo de las 10 semanas que duró el estudio. Estas encuestas buscan evaluar diferentes cuestiones de orden psicológico, como el estado anímico, el nivel de estrés y el comportamiento en general. En promedio cada usuario respondió 8 EMAs por día.
- *Sensing*: contiene información recolectada a partir de los sensores de los smartphones portados por los usuarios. Este tipo de datos ha sido la principal fuente de datos para entrenar los modelos desarrollados en esta tesis, por lo que tiene su sección propia.
- *SMS*: similar al tipo de datos Call-log, pero conteniendo información sobre los mensajes de texto enviados y recibidos.
- *Survey*: contiene 8 diferentes encuestas que fueron realizadas antes y después de comenzar el estudio. Las encuestas son: *Flourishing Scale (Russell, 1996)*, *Loneliness Scale (Russell, 1996)*, *Panas (Watson et al., 1988)*, *Perceived Stress Scale* y *PHQ-9 (Kroenke et al., 2001)*.

4.2.2. Datos no disponibles

Gran parte del total de los datos recolectados durante el estudio *StudentLife* no se encuentran públicamente disponibles en el *dataset*, debido a dos razones. La primera, está relacionada a cuestiones de privacidad, como los mensajes de los SMS, los datos de audio o los nombres de los usuarios. Otros datos que no están disponibles fueron utilizados solo para procesar otros datos que sí fueron incluidos en el *dataset* publicado. De esta forma, el total de los datos recolectados a partir de los diferentes sensores del celular es de 52,6 GB, mientras que en el *dataset* disponible públicamente solo son 2,6 GB aproximadamente.

4.2.3. Datos del acelerómetro

Los únicos datos sin procesar que se encuentran disponibles son los del acelerómetro, pero no han podido ser utilizados por problemas técnicos. Los datos del acelerómetro para cada usuario fueron puestos a disposición de tal forma que cada usuario tiene asociado un archivo de base de datos de tipo *SQLite*. Cada base de datos está compuesta por solo una tabla con dos columnas. La primera columna es llamada *timestamp* y representa el momento en el que se registró la información del acelerómetro. La segunda columna es llamada *feat_data*, que debiera contener la información del registro del acelerómetro. El problema hallado es que esta última columna es de tipo BLOB, que es utilizado para almacenar cadenas de longitud variable de bytes binarios. No se logró encontrar la forma correcta de extraer los datos del acelerómetro de tipo BLOB, por lo que dichos datos no han sido utilizados. Es importante señalar que los datos del acelerómetro podrían haber representado una importante mejoría para los modelos implementados, ya que podrían haberles aportado información relevante acerca de la actividad física del

usuario. Esto es así debido a que los datos de la actividad provistos por el *dataset StudentLife* tienen una granularidad muy pobre (solo 3 tipos).

4.2.4. Datos de sensado

Cada uno de estos tipos de datos de sensado son provistos para cada usuario en formato csv (*comma separated values*, valores separados por una coma). Por lo tanto, a cada tipo de dato le corresponden 48 archivos csv. Cada uno de estos datos está acompañado de un *timestamp* que indica el horario y la fecha del registro. Estos *timestamp* tienen el formato de Unix, por lo que se representan mediante un número entero positivo que indica la cantidad de segundos pasados desde el primero de enero de 1970. Por ejemplo el *timestamp* 1364357009 equivale a 03/27/2013 4:03 AM. Por lo tanto, cada registro se compone de un *timestamp* y diversos datos asociados al tipo de dato, pudiendo ser uno solo (como en el caso de *activity*) o varios, como en el caso de GPS. Aunque este es el caso para la mayoría de tipos de datos de sensado continuo, algunos presentan otro formato, en el cual existen dos *timestamp* que indican el momento de inicio y finalización de un evento. Al detallar cada tipo de dato de sensado, se clasificará a estos dos tipos de datos como Tipo de Sensado Discreto (TSD) y Tipo de Sensado por Intervalo (TSI).

Los investigadores utilizaron la aplicación *BeWell* (Lane et al., 2012) para proveer un framework para el sensado automático en el estudio *StudentLife*. Los investigadores aclaran que el estudio *StudentLife* fue llevado a cabo antes que Google anunciara su servicio de reconocimiento de actividad para teléfonos Android, que fue el utilizado en el estudio. Debido a esto, se utilizó el motor de sensado Jigsaw (Lu et al., 2010), desarrollado por los investigadores en un trabajo previo, que permite recolectar y clasificar datos del acelerómetro, el GPS y audio de forma eficiente.

A continuación, se listan los diferentes datos de sensado continuo incluidos en el *dataset StudentLife*.

- Tipo de Sensado Discreto
 - *Activity*: contiene registros de la actividad física del usuario. Este tipo de dato ha sido recolectado aproximadamente cada 2-3 segundos, dependiendo de la frecuencia de muestreo del acelerómetro del smartphone. El clasificador de actividad del motor de sensado Jigsaw alcanzó un 94% de precisión. Los registros de actividad están clasificados en 4 tipos diferentes y se representan mediante un código en los archivos csv. Por ejemplo, [1364356899, 0] representa un registro de actividad física, que significa que los datos del acelerómetro que fueron recolectados el 27/03/2013 a las 4:01:39 fueron inferidos como “estacionario” por el clasificador de actividad de Jigsaw. A continuación, se listan los tipos de actividad física junto con el código con el cual están representados en el *dataset*:
 - *Stationary* (Cód. 0): el usuario se encuentra en estado estacionario.
 - *Walking* (Cód. 1): el usuario se encuentra caminando;
 - *Running* (Cód. 2): el usuario se encuentra corriendo;
 - *Unknown* (Cód. 3): se desconoce el estado de actividad del usuario¹¹.
 - *Audio*: Contiene registros de audio del usuario. El clasificador de audio de Jigsaw funciona realizando inferencias de audio durante 1 minuto y luego se pausa por 3

¹¹ En un intercambio de mails con Rui Wang, el investigador explicó que estos registros se encuentran principalmente al principio del estudio *StudentLife*, por lo que no deberían representar un inconveniente para realizar experimentos utilizando el *dataset*.

minutos, a no ser que el clasificador de conversación detecte que hay una conversación en curso, en cuyo caso el clasificador de audio no se detiene. Este tipo de dato ha sido recolectado aproximadamente cada 2-3 segundos. A continuación, se listan los tipos de audio junto con el código con el cual están representados en el *dataset*:

- *Silence* (Cod. 0): el clasificador de audio infiere silencio.
 - *Voice* (Cod. 1): el clasificador de audio infiere voz (posiblemente una conversación).
 - *Noise* (Cod. 2): el clasificador de audio infiere ruido.
 - *Unknown* (Cod. 3): el clasificador de audio no reconoce el tipo de audio.
 - *Bluetooth*: contiene información sobre escaneos del dispositivo de bluetooth perteneciente al smartphone de cada usuario. En este caso, y a diferencia de otros tipos TSD, varios registros pueden compartir el mismo *timestamp*. Esto es normal, ya que cada escaneo reporta información sobre las redes bluetooth cercanas halladas, que pueden ser más de una. Para cada red bluetooth escaneada, se obtiene la dirección MAC y la intensidad de la señal. La frecuencia de muestreo para este tipo de sensor es de 1 cada 10 minutos.
 - *GPS*: contiene información sobre el GPS. Dicha información incluye la proveniencia de la señal (GPS o Network), la latitud, la longitud y la altitud. La frecuencia de muestreo para este tipo de sensor es de 1 cada 10 minutos.
 - *Wifi* : Similar al tipo de dato Bluetooth, estos registros arrojan información sobre las redes de tipo Wifi escaneadas, como la dirección MAC del router, la frecuencia del canal por donde se realiza la conexión y la intensidad de la señal. Al igual que en Bluetooth, pueden existir varios timestamp idénticos.
 - *Wifi location*: los autores del *dataset StudentLife* dividieron el Campus de la ciudad de Dartmouth en diferentes zonas. En los registros de este tipo de sensor, podemos hallar la ubicación aproximada de un usuario en un determinado momento a partir de la red Wifi a la que está conectado. Las zonas se clasifican en In[x] (dentro del edificio x) y Near[x] (dentro del edificio x).
- Tipos de sensado por intervalo
 - *Conversation*: intervalo de tiempo en el que el usuario llevó a cabo una conversación.
 - *Phone charge*: intervalo de tiempo en el que el smartphone está siendo cargado.
 - *Phone lock*: intervalo de tiempo en el que el smartphone se encuentra bloqueado.
 - *Light*: intervalo de tiempo en el que el smartphone se encuentra en la oscuridad.

4.3. Implementación (Evaluación Experimental)

Para la implementación de los modelos se utilizó la librería Keras¹² de Python 3. Los scripts implementados para el procesamiento del *dataset* y el entrenamiento de los modelos se dejan disponibles públicamente¹³.

4.3.1. Análisis de datos

4.3.1.1. Tamaño de buckets

Como ya ha sido mostrado, los tipos de datos que provee el *dataset StudentLife* son un conjunto heterogéneo, es decir, existen muchos tipos diferentes de datos. Por lo tanto, si se quiere utilizar este *dataset* para generar modelos de *Machine Learning* con el fin de llevar a cabo la tarea de PCSF, debe llevarse a cabo un proceso previo que permita que dicho *dataset* sea apto para los diferentes modelos. Para esto, a partir del *dataset StudentLife* se debe generar un nuevo *dataset* que esté compuesto por casos, donde cada caso esté asociado a un valor de la variable objetivo. Cada caso de estudio estará compuesto por características que serán utilizadas para predecir la variable objetivo. Por su parte, la variable objetivo representará el nivel de sedentarismo expresado a través del valor de MET.

El problema más grande que trae la gran heterogeneidad de los datos es su disponibilidad. Podemos distinguir 3 tipos de datos presentes en el *dataset* según su disponibilidad:

- Las encuestas que se hicieron antes y después del estudio StudentLife,
- los datos EMA, y
- los datos de sensores y derivados de ellos.

De estos 3 tipos, solo en el tercer caso existen datos presentes uniformemente a lo largo de todo el estudio -incluyendo los datos de actividad física-, lo que permite generar una gran cantidad de casos o ejemplo de entrenamiento para los modelos, que es crucial para lograr entrenar modelos de *Machine Learning* que alcancen un buen desempeño. Por esta razón, los *datasets* para entrenar y testear los modelos serán generados a partir de los datos de sensores y derivados de ellos. Los otros dos tipos de datos no serán utilizados debido a su baja disponibilidad. Es decir, si fuesen utilizados, sólo podrían generarse casos de entrenamiento para aquellos momentos en los que hay datos disponibles, que son muy pocos en comparación a los datos de sensado.

Dentro de los datos de sensores, puede observarse también una disponibilidad variable. Es decir, cada tipo de datos de sensor tiene una cantidad promedio de registros por unidad de tiempo diferente (esta cantidad de registros está dada en la descripción del *dataset*, pero aun así es calculada junto con otros estadísticos en la siguiente sección a modo de comprobación). Por esta razón, para generar casos es necesario procesar los datos de sensores de forma tal que cada caso pueda incluir información de todos los sensores. Esto último se logra discretizando el tiempo en intervalos, llamados *buckets*. La longitud de estos intervalos debe definirse teniendo en cuenta la frecuencia que tiene cada uno de los datos. Es decir, cada *bucket* debe contener por lo menos un registro de cada tipo de dato que se quiere usar, ya que la gran mayoría de los modelos de *Machine Learning*, y en especial los que serán utilizados en esta tesis, no

¹² <https://keras.io/>

¹³ <https://github.com/martinscooper/tesis-python>

aceptan características sin valor. Así mismo, es válido discutir un número mínimo de registros necesarios para que un tipo de dato en un intervalo de tiempo específico sea válido. Es decir, es necesario responder a la pregunta: ¿A partir de qué cantidad de registros se considera válido un tipo específico de dato? A modo de ejemplo, ¿es válido un *bucket* de 1 hora de longitud donde se cuenta con solo un registro de actividad física? ¿Puede considerarse que ese registro representa a todo el intervalo (teniendo en cuenta que según la descripción del *dataset* los registros de actividad física se han registrado cada 3 segundos, es decir, que se esperan 1200)? ¿Si no, cuál debería ser el número mínimo? ¿Es conveniente descartar estos *buckets* sabiendo que reducen el número de datos de entrenamiento y, en consecuencia, el desempeño de los modelos?

Una vez que el *dataset* se discretiza en *buckets*, diferentes características pueden ser calculadas a partir de los registros de cada tipo de dato incluido en cada uno de los *buckets*. Por ejemplo, para un Tipo de Sensor Discreto en el que se cuentan con x registros en un intervalo de tiempo t , correspondiente al tamaño de *bucket*, se generan características a partir de la información disponible. Dichas características pueden ser estadísticos que expliquen los datos disponibles u otros cálculos ad-hoc. Es válido tanto que varios tipos de datos puedan ser combinados para generar una sola característica, así como que varias características sean generadas a partir de un solo tipo de dato.

De acuerdo a las explicaciones anteriores, el tamaño de *bucket* va a definir cuántos registros están presentes en cada uno de ellos, así como también la cantidad de *buckets* resultantes. De esta manera, se da la siguiente relación: cuanto mayor sea el tamaño de *bucket*, mayor será la cantidad de datos disponibles en cada uno de ellos, a la vez que la agrupación del *dataset* en *buckets* dará como resultado una menor cantidad total de ellos. Esta relación sirve como punto de partida para analizar y definir cuál debería ser el tamaño de *bucket*. Por un lado, que haya más *buckets* en total es beneficioso para los modelos ya que se parte de un *dataset* más grande del cual aprender y la granularidad a la hora de predecir el comportamiento sedentario es mayor, por lo que un tamaño de *bucket* pequeño es deseado. Por otro lado, hay tipos de datos cuya frecuencia de muestreo es muy baja, por lo que si el tamaño de *bucket* es demasiado pequeño, puede que no haya datos disponibles para ese tipo de sensor. A partir del análisis anterior, se decidió realizar los experimentos utilizando dos tamaños de *buckets* diferentes: de 1 hora y de 30 minutos.

Existen diferentes razones por las cuales se define el tamaño de *bucket* de 1 hora. Por un lado, se toma el tamaño de *bucket* de 1 hora debido a que es el utilizado en todos los trabajos relacionados a la tarea de PCSF que usan el *dataset StudentLife* (He & Agu, 2017, 2016a, 2016b, 2016c). Por otro lado, los intervalos de 1 hora son lo suficientemente grandes como para incluir en promedio siempre más de un registro por *bucket* para generar las características. Asimismo, la división entre *buckets* de media hora fue incluida para comparar el impacto en el desempeño de los modelos de aprendizaje de doblar aproximadamente la cantidad de casos generados a partir del *dataset* original. La desventaja de esto es que habrá más *buckets* que deberán ser descartados por no presentar registros de alguno de los tipos de datos. Como consecuencia, los modelos que sean entrenados a partir de la utilización de varios *buckets* consecutivos (por ejemplo: CNN y RNN) se verán afectados porque se darán más casos en los cuales una secuencia de *buckets* deberá ser descartada por la ausencia de uno o más *buckets*.

4.3.1.2. Disponibilidad de los datos

La frecuencia de cada uno de los tipos de datos de sensado continuo fue listada en la [Sección 4.2.4](#). Como ya ha sido discutido, para entrenar modelos de *Machine Learning*, es preferible utilizar los tipos de datos para los cuales haya más información disponible. Es por esta razón que se seleccionan del *dataset StudentLife* los tipos de datos de sensado continuo, y se descartan aquellos relacionados a test psicológicos o encuestas EMA.

En esta sección se presenta un análisis sobre la disponibilidad de cada tipo de datos de sensado continuo con el objetivo de verificar la información sobre la frecuencia de cada tipo de dato listada en la descripción del *dataset*. El análisis de la disponibilidad de cada tipo de dato difiere según si el tipo es TSD o TSI. Para realizar un cálculo de la disponibilidad de cada tipo de dato se discretiza el tiempo en dos tipos de *buckets*: de 1 hora y de 30 minutos.

Para los TSD, se calcula:

- la cantidad de registros disponibles,
- la cantidad de *buckets* totales resultantes luego de la discretización,
- los *buckets* para los cuales se tiene información -que serán siempre menores o iguales a los *buckets* totales-,
- los siguientes estadísticos: media, desviación estándar, mínimo, máximo de registros por *buckets*.

Los resultados pueden verse en la Tabla 4.1. En el caso específico del sensor *Activity*, existen muchos valores nulos, por lo que se incluye una fila con los resultados de eliminar los registros clasificados como *unknown*.

Como puede verse en la Tabla 4.1 la cantidad de registros disponibles varía mucho entre los diferentes TSD, yendo desde poco más de 200 mil registros disponibles para el GPS, hasta casi 100 millones en el caso de los datos de audio. A pesar de esto, la cantidad de *buckets* para los cuales existe información no se ve tan afectada, tanto en el caso de *buckets* de una hora como para los de 30 minutos. Para los casos de los *buckets* de una hora la cantidad mínima de *buckets* corresponde al sensor Bluetooth, con 39.693 *buckets*, mientras que la cantidad máxima al sensor *Activity*, con 62.807 *buckets*. Sin embargo, para la gran mayoría de los sensores de este tipo la cantidad de *buckets* ronda los 60.000. El mismo resultado se da en el caso de *buckets* de 30 minutos, con la diferencia de que las cantidades son de aproximadamente el doble en cada caso.

La columna “Cantidad de *buckets*” se calcula como la cantidad de *buckets* totales existentes desde el primer registro al último, es decir, teniendo en cuenta también los *buckets* para los cuales no existen datos para el sensor. Puede observarse que son iguales para todos los TSD y en ambos tamaños de *buckets*. La fecha del primer registro data del 2013-03-27 a las 05:00:00, mientras que la última del 2013-06-01 a las 05:59:59.

	Cantidad total de registros	Cantidad de <i>buckets</i> para los cuales hay datos		Cantidad de <i>buckets</i>		Media y desviación estándar de registros por <i>bucket</i>		Maximo y minimo de registros por <i>bucket</i>	
		1h	30min	1h	30min	1h	30min	1h	30min
Activity	22.842,191	62.807	125.009	77.665	155.330	363,68/193,29	182,72/100,09	1 / 4.820	1 / 3.538
Activity (sin <i>unknowns</i>)	22.233,593	62.767	124.887	77.665	155.330	354,22/186,45	178,02/96,42	1 / 4.367	1 / 3.537
Audio	99.298,223	62.792	124.968	77.665	155.330	1.581,38/973,7	794,58/514,07	5 / 45.294	1 / 22.692
GPS	202.877	59.223	115.896	77.665	155.330	3.42/1,3	1,75/0,77	1 / 16	1 / 9
Wifi	19.244.309	61.750	122.526	77.665	155.330	31,64/522,3	157,06/273,09	1 / 6.144	1 / 3.500
Wifi location	1.893.838	50.747	99.407	77.665	155.330	37,31/59,74	19,05/30,63	1 / 357	1 / 179
Bluetooth	1.288.526	39.693	70.075	77.665	15.5330	32,46/146,35	18,38/79,55	1 / 4.987	1 / 2.676
Calendar	1.687	563	563	65.660	131.292	2,99/2,31	2,99/2,31	1 / 20	1 / 20

Tabla 4.1. Estadísticos calculados a partir de los TSD

Las últimas dos columnas muestran diferentes estadísticos que permiten comprender mejor cómo se distribuye la totalidad de los registros en cada *bucket*. De la media y la desviación estándar se puede observar que en general la cantidad de registros por *bucket* difiere mucho, tanto que en algunos casos la desviación estándar es mayor que la media, de lo que se puede deducir que hay periodos de tiempo en los que la recolección de datos es muy alta y otra en las que fue casi nula. Además, muestra que en ningún caso la frecuencia de muestreo es igual a la reportada en la descripción del *dataset* para cada tipo de dato de sensado continuo. Por ejemplo, la frecuencia reportada para el sensor *Activity* es de 3 segundos, por lo que se esperarían 1200 registros por hora, mientras que la media de registros por *buckets* es de 360. Además, si se analizan los máximos y mínimos de cada sensor se pueden observar más inconsistencias, ya que todos los sensores poseen por lo menos un para el cual se dispone del número mínimo de registros, es decir uno. Mientras que el número máximo representa picos, también en todos los sensores, de manera anormal ya que en muchos casos se alejan mucho de la media, siendo el caso más extremo para el sensor *Audio*, para el cual existe un *bucket* de una hora con 45.294 registros. Este análisis no representa en sí un problema a la hora de procesar el *dataset StudentLife*, pero si da la pauta de que el *dataset* posee inconsistencias con respecto a su documentación.

Con respecto a la diferencia que existe entre la cantidad de *buckets* con información disponible para cada sensor, debe realizarse un análisis más detallado. Si bien para que un *bucket* sea apto para ser utilizado en el *dataset* de casos debe poseer información sobre todos los sensores que sean utilizados para generar las características, esto no significa que la falta de registros en un *bucket* equivalga a la falta de información. Esto es, en algunos casos, puede interpretarse la ausencia de datos como información útil que pueda ser calculada para dicho *bucket*. Por ejemplo, en el caso del sensor Bluetooth, que contiene registros correspondientes a las redes Bluetooth cercanas, la falta de registros en un *bucket* determinado puede interpretarse como que no se han detectado redes Bluetooth cercanas. Por lo tanto, si una característica para cada *bucket* es el número de redes Bluetooth detectadas, los *buckets* para los cuales no se tengan registros Bluetooth tendrán un número 0 de redes Bluetooth detectadas y no deberán ser descartados del *dataset*.

En el caso de los sensores que sean usados y no sea posible interpretar los *buckets* para los cuales no se tengan registros disponibles como información útil, deberán ser descartados. Por lo tanto, en estos casos es importante analizar si es mejor no utilizar los datos de ese sensor en pos de tener más *buckets* disponibles o mantenerlos, aumentando así la información de contexto que pueden utilizar los modelos, pero reduciendo la cantidad de casos de entrenamiento disponibles. Como la cantidad de *buckets* en todos los casos es similar, esta decisión desembocará siempre en utilizar el sensor con *buckets* vacíos en lugar de descartarlos. Otro punto importante a destacar es que el número total de casos resultantes no será el número de *buckets* del sensor con menos *buckets* disponibles con información, sino que será la cardinalidad de la unión de todos los *buckets* con información útil disponible para cada uno de los sensores utilizados. Se hace esta aclaración ya que los *buckets* sin información de un sensor pueden no coincidir con los *buckets* sin información de otro sensor.

Para los TSI, se calculan:

- La cantidad total de registros,
- El promedio y la desviación estándar del tamaño de los intervalos,
- El máximo y mínimo del tamaño de los intervalos, y

- El porcentaje de cobertura total.

En la tabla Tabla 4.2 pueden observarse los datos calculados. Al analizar la primera columna puede notarse que la cantidad total de registros del sensor *Conversation* posee de 8 a 24 veces más registros que los demás sensores. Esto se da porque el sensor *Conversation* es el único TSI para el cual no se limitó el tiempo mínimo de duración. Para los TSI *Dark*, *PhoneLock* y *PhoneCharge* los únicos registros que se incluyeron en el *dataset* son aquellos cuya duración supera una hora. Esto puede corroborarse gracias a la columna donde se muestran los mínimos, donde puede observarse que el mínimo de estos 3 sensores es de una hora. Es comprensible la decisión de no limitar la mínima duración de las conversaciones ya que tienden a ser más cortas que los eventos que describen los otros sensores. Por ejemplo: los dispositivos requieren más de una hora para ser cargados. Aunque se entiende que la duración mínima de las conversaciones también ha sido limitada, ya que la conversación más corta registrada es de 36 segundos. Con respecto a la duración máxima de los registros, aunque no se informa al respecto en la descripción del *dataset*, parece haber sido truncada a 10 horas.

La mayor granularidad del sensor *Conversation* permite idear diferentes características que pueden ser calculadas para el resto de los TSI. Principalmente, a un determinado *bucket* pueden pertenecer más de un registro del TSI *Conversation*, por lo que puede calcularse la cantidad de conversaciones pertenecientes a cada *bucket* o el porcentaje del *bucket* en el cual el usuario mantuvo una conversación, lo cual enriquece el contexto a partir del cual los modelos realizarán las predicciones. En el caso de los otros TSI, la cantidad de registros que pertenezcan a un determinado *bucket* de 30 minutos o una hora siempre va a ser 1 o 0.

El porcentaje de cobertura total se calculó como la división entre la suma de la duración de todos los registros y el tiempo entre el comienzo del primer registro y el final del último. Este porcentaje fue calculado suponiendo que no hay intersecciones temporales entre los registros por lo que los valores mostrados son aproximados. El porcentaje de cobertura total puede variar entre los diferentes usuarios. Por ejemplo, aquellos que sean más sociables tendrán un mayor porcentaje de cobertura total para el sensor *Conversation*. Como puede observarse, el sensor *PhoneLock* resultó ser aquel con el mayor porcentaje de cobertura total, mientras que el sensor *Conversation* resultó ser aquel con el mayor porcentaje de cobertura total.

	Cantidad total de registros	Promedio y desviación estándar del tamaño de los intervalos (horas)	Maximo y minimo del tamaño de los intervalos (horas)	Porcentaje de cobertura total (%)
<i>Dark</i>	7.269	3,246/2,521	1/9,997	33,7
<i>PhoneLock</i>	9.275	3,121/2,37	1/10	40,5
<i>PhoneCharge</i>	3.318	3,95/2,722	1/9,999	18,9
<i>Conversation</i>	79.023	0,17 / 0,27	0,006/6,872	18,8

Tabla 4.2. Estadísticos calculados a partir de los TSI

4.3.1.3. Registros desconocidos

Existen dos razones por las cuales es importante realizar un análisis específicamente sobre los datos del sensor *Activity*. Por un lado, este es el único sensor que presenta registros clasificados como desconocidos, por lo que es importante llevar a cabo un análisis que permita estimar las consecuencias de la presencia de estos registros faltantes en el *dataset StudentLife*. Por otro lado, los datos de este sensor son los únicos utilizados para generar la variable objetivo, es decir, la variable a predecir. El nivel de sedentarismo del usuario en un *bucket* dado se calcula a partir de todos los registros de actividad física disponibles en ese *bucket*. Los registros de actividad física clasificados como *unknown* no pueden ser utilizados ya que se desconoce cuál es la actividad física que el usuario estaba llevando a cabo en ese momento.

En esta sección se analizan los registros clasificados como *unknown* para entender su distribución a lo largo del estudio *StudentLife*, así como también cómo varía su distribución para cada uno de los usuarios. Esto permitirá evaluar si es necesario tomar medidas con respecto al manejo de estos registros desconocidos y que problemas puede causar.

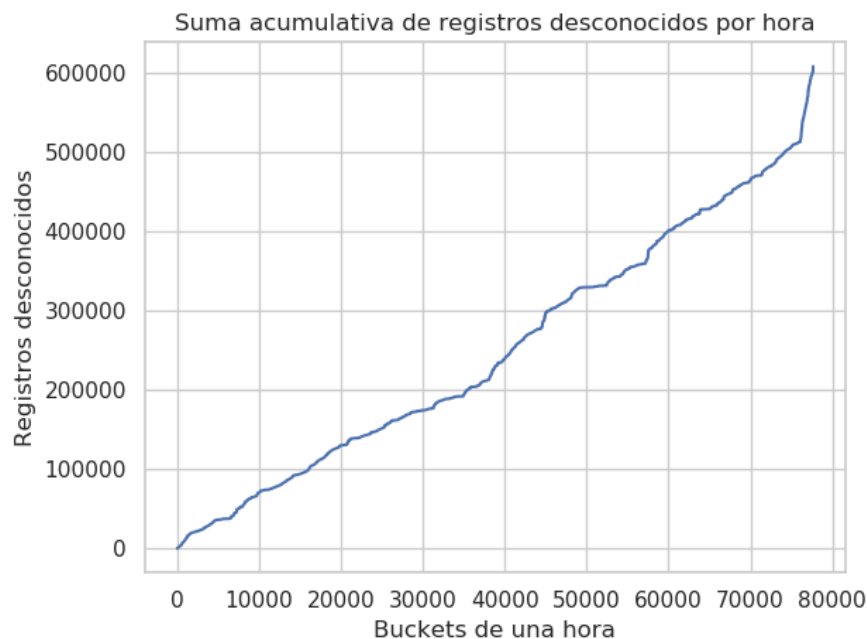


Figura 4.1. Suma acumulativa de registros desconocidos

Los registros para los cuales no se tiene información sobre la actividad física son codificados con el número 3. Sabiendo esto, resulta sencillo generar gráficos que muestren claramente la distribución de dichos registros. Cabe aclarar que todos los gráficos que se muestran en esta sección han sido generados a partir de los *buckets* de una 1 hora. No se presenta el mismo análisis para ambos tipos de *buckets* ya que arrojaron resultados similares.

En la Figura 4.1 se muestra la suma acumulada de registros desconocidos para todas las horas disponibles. En dicha figura, el eje x representa a cada *bucket* que es posible generar a partir de los datos del sensor *Sensing*, sin importar a qué usuario pertenezcan. Mientras que el eje y representa la suma acumulada de registros clasificados como *unknown*, es decir que el valor de y_i es la suma desde el primer *bucket* hasta el *bucket* i .

En la figura, puede observarse que la cantidad de registros crece de forma aproximadamente lineal con respecto a la cantidad de *buckets* a lo largo de todo el *dataset*. Esto es una buena señal, ya que significa que, a priori, los *buckets* tendrían una cantidad similar de registros a partir de los cuales generar las características de actividad y que los registros *unknown* no están agrupados en una sección específica del *dataset*. Esto quiere decir que, al momento de calcular el nivel de sedentarismo, cada *bucket* tendrá un soporte similar para calcular dicho nivel.

Al final del eje x , se puede observar un pico en la cantidad acumulada de registros *unknown*. Estos registros pertenecen al último usuario, que presenta algunas diferencias con los demás usuarios, que podrán ser mejor visualizadas en el siguiente gráfico. En la Figura 4.2, se muestran dos gráficos de barras con la media y la desviación estándar de la cantidad de registros desconocidos para cada hora de un determinado usuario. Es decir, si para un usuario se disponen 1.000 horas, se calcula la media y la desviación estándar de la cantidad de registros desconocidos disponibles para cada hora.

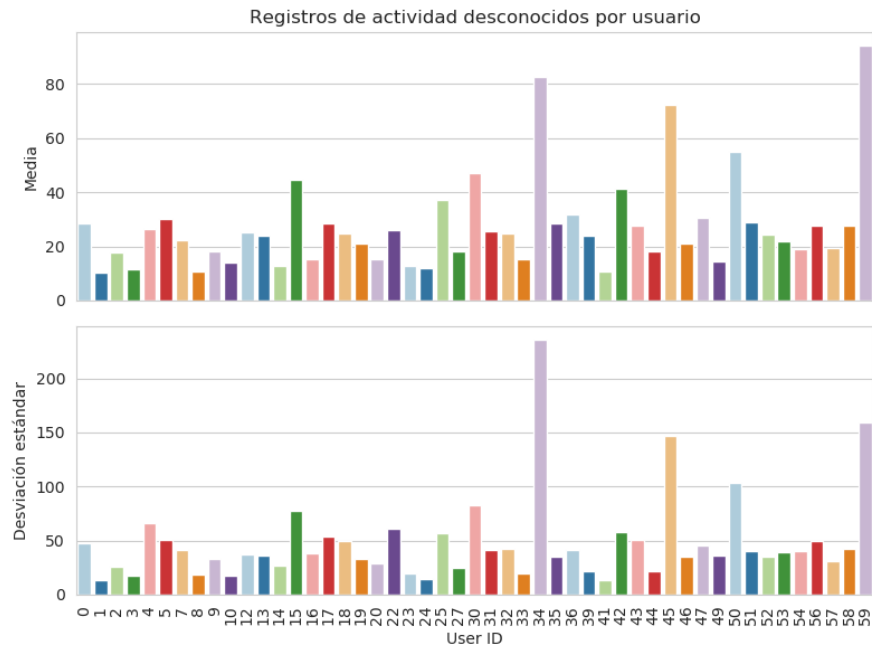


Figura 4.2. Media y la desviación estándar de registros desconocidos

Varias observaciones pueden hacerse a partir de estas figuras. Con respecto a los valores faltantes, puede verse que la mayoría de los usuarios presenta entre 20 y 40 registros desconocidos por hora, lo que explica el carácter lineal de la función graficada en la Figura 4.1. Sin embargo, existen ciertas excepciones, como los usuarios 34, 45 y 59, donde se puede observar una media mayor de registros

faltantes, asimismo como una desviación estándar mayor. El usuario 59 es el que mayor cantidad de registros desconocidos posee, lo que explica el pico observado al final de la Figura 4,1.

Por su parte, la Figura 4.3 muestra los mismos estadísticos, pero en lugar de ser generados a partir de los registros desconocidos, éstos son generados a partir de los registros no desconocidos (con un código de actividad 0, 1 o 2), es decir, a partir de los cuales se tiene información certera. En el primer gráfico, donde se muestra la media, puede verse que todos los usuarios presentan casi la misma cantidad de registros por *bucket* (aproximadamente 350). Sin embargo, los mismos usuarios que poseen muchos registros desconocidos (34, 54, 59) también presentan una alta desviación estándar en la cantidad de registros no desconocidos por hora, hechos que podrían estar relacionados. Es posible observar, también, que el usuario 59 posee aproximadamente el cuádruple de registros por *buckets*. Más adelante se discutirá la posibilidad de tratar a estos usuarios que presentan una representación diferente en el *dataset* como *outliers*.

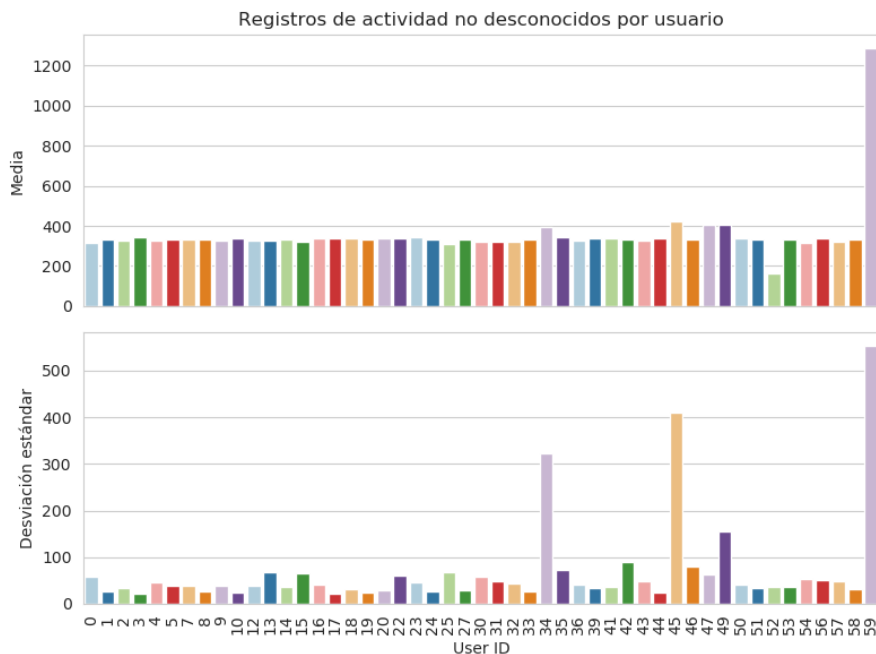


Figura 4.3. Media y la desviación estándar de registros no desconocidos

4.3.1.4. Depuración de buckets

Una vez que el *dataset StudentLife* ha sido discretizado, el siguiente paso del preprocesamiento es eliminar los *buckets* para los cuales no se posee información sobre alguna de las variables. Es preciso notar que solo se eliminan los *buckets* para los cuales existe al menos alguna característica para la cual no exista ningún registro que pueda ser utilizado para realizar el cálculo de dicha característica. Por ejemplo, para los *buckets* en los que no hubo registros de audio, puede interpretarse que hubo silencio, por lo que

las características diseñadas pueden aprovechar esta interpretabilidad y tener un valor para esos *buckets*. En este caso, el valor para *AudioMajor* será 0 o “en silencio”.

En la Figura 4.4 se muestra la cantidad de *buckets* resultantes disponibles para cada usuario y la cantidad de *buckets* descartados por la falta de registros para alguna variable, luego de depurar el *dataset* preprocesado. Como puede observarse, existe una gran diversidad con respecto a la cantidad de *buckets* para cada usuario. El usuario con menor cantidad de *buckets* disponibles es el 39, con 479 *buckets*, mientras que el usuario con más *buckets* es el 51, con 1579.

Como ya ha sido discutido en el marco teórico, la cantidad de casos de entrenamiento para un algoritmo de aprendizaje puede determinar que dicho algoritmo alcance un buen desempeño o no. Es necesario recordar en este momento que se comparan dos tipos de modelos de acuerdo a de donde provengan los datos de entrenamiento: los modelos personales y los impersonales. Con respecto a la división anterior, es posible hipotetizar que para los usuarios con muy pocos *buckets* disponibles, como es el caso del usuario 39, los modelos impersonales obtendrán un mayor desempeño que los personales. La explicación de la hipótesis anterior se da porque puede que la baja cantidad de *buckets* no llegue a darle la información al algoritmo de aprendizaje sobre las idiosincrasias de estos usuarios en particular, que es la ventaja que presentan los modelos personales por sobre los impersonales.

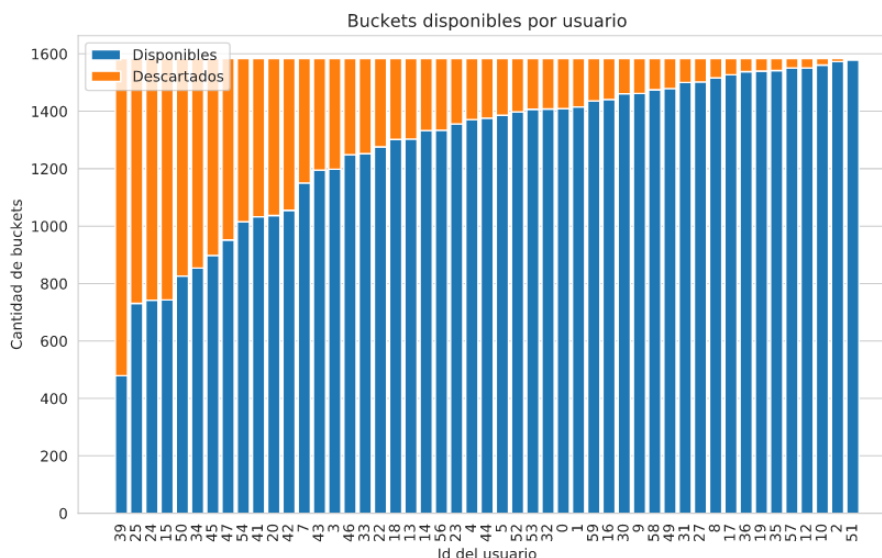


Figura 4.4. Cantidad de *buckets* disponibles y descartados por cada usuario

4.3.1.5. Nivel de MET

Los registros de actividad de cada usuario están en el *dataset* clasificados como estacionario, caminando o corriendo. A partir de estos registros se calcula el valor del gasto energético para cada intervalo. Tal como fue explicado en la [Sección 4.1](#), el gasto energético es comúnmente medido en términos de MET. El *dataset StudentLife* no posee información explícita que determine el nivel de MET, por lo que se tomó la decisión de diseñar un marco en el cual fuese posible calcularlo a partir de los datos disponibles. A cada tipo de actividad disponible en el *dataset* se le asigna un valor estático aproximado de MET de acuerdo al

Compendio de Actividades Físicas (Ainsworth et al., 2011)¹⁴. El compendio de Actividades Físicas posee 21 categorías, donde cada una de ellas está conformada por tipos de actividades. A su vez, cada actividad está acompañada por el valor de MET asociado a esa actividad. Para cada una de las actividades presentes en el Compendio existe evidencia publicada que soporta la asociación entre dicha actividad y su valor de MET.

Una vez analizado el compendio de actividades físicas, se seleccionaron los tipos de actividades más similares a aquellas presentes en el *dataset StudentLife*. A las actividades registradas como estacionarias, se les otorgó un valor de MET de 1,3, el cual corresponde a actividades como “sentado en calma”, “acostado en calma”, “no haciendo nada”, “acostado despierto en la cama”, “escuchando musica”. A las actividades registradas como “caminando”, se les otorgó un valor de MET de 5,0, el cual corresponde a actividades como “cargando aproximadamente 6 kilos (por ejemplo, una mochila), en terreno llano o bajando escaleras”. A las actividades registradas como “corriendo” se les otorgó un valor de MET de 8,3, el cual corresponde a actividades como “corriendo, 8 km/h”.

Una vez que se definió a qué valores de MET asociar cada tipo de actividad, se transformó la información disponible sobre la actividad (variable categórica ordinal) en una variable numérica. Luego, fue necesario definir cómo iba a ser resumida la información de la actividad física -expresada en METs- para cada *bucket*, de tal forma que el resultado fuese un número real. Es decir, cómo todos los datos disponibles son agrupados por *buckets*, debe calcularse un valor de MET representativo para cada uno de ellos. Para ello, se define al nivel de MET de un *bucket* como la media del valor de MET de cada uno de los registros de actividad perteneciente a dicho *bucket*. Por lo tanto, el nivel de MET de un *bucket* b está dado por la fórmula:

$$MET(b) = \frac{\sum_i^n MET(r_i)}{n},$$

donde $MET(x)$ es el valor de MET asociado a la actividad x , r_i el registro de actividad número i y n la cantidad de registros de actividad disponibles para el *bucket*. Por ejemplo, si un *bucket* contiene 100 registros de tipo “estacionario”, 10 registros de tipo “caminando” y 10 registros de tipo “corriendo”, el nivel de MET ese *bucket* se calcula como:

$$(100 * 1,3 + 10 * 5 + 10 * 8,3)/120 = 2,2$$

Como ya ha sido explicado con anterioridad, un comportamiento es considerado como sedentario si supera el valor de 1,5 METs. La anterior afirmación es adaptada en esta tesis de la siguiente forma: el comportamiento llevado a cabo en un *bucket* es considerado como sedentario si el nivel de MET para dicho *bucket* es superior a 1,5.

Es necesario recordar en este punto que la baja granularidad en el tipo de actividad física especificadas en el *dataset StudentLife* representa una limitación, ya que el tipo de actividad del Compendio de Actividades Físicas asociadas a cada una de las actividades físicas especificadas en el *dataset StudentLife* puede no ser certero y distar de la verdadera actividad que el usuario estaba llevando a cabo al momento de ser registrada. Además, los valores de MET asociados a cada actividad en el Compendio no estan destinados a ser utilizados de manera individual, ya que según se explica “Los

¹⁴ <https://sites.google.com/site/compendiumofphysicalactivities/home>

valores en el Compendio no estiman el gasto de energía de la actividad física en una forma en la cual se tengan en cuenta diferencias en masa corporal, adiposidad, edad, sexo, eficiencia en el movimiento, condiciones geográficas y ambientales en las cuales las actividades son llevadas a cabo. Por lo tanto, las diferencias de gasto energético para la misma actividad pueden ser grandes y el gasto energético real para un individuo puede o no ser cercano al promedio establecido de MET presentado en el Compendio (Ainsworth et al., 2011)”.

4.3.1.6. Análisis del nivel de MET

En esta sección se realiza un análisis de la actividad física, con el fin de comprender cómo se distribuye a lo largo de todo el *dataset StudentLife*. Este análisis permitirá conocer cuáles son las actividades que más se llevan a cabo y en qué medida. Además, a través de los gráficos podrán observarse los patrones de comportamiento de los diferentes usuarios.

La Figura 4.5 muestra, de manera similar a las figuras mostradas en la sección anterior, dos gráficos de barras en los que se representa el promedio y la desviación estándar del nivel de MET, ordenados de acuerdo al promedio del nivel MET. Ambos estadísticos son computados a partir del nivel de MET de todos los *buckets* disponibles para cada usuario. Como puede observarse, el promedio de MET de cada usuario ronda los 1,5, es decir, el nivel de MET que delimita lo que se considera un comportamiento sedentario o no.

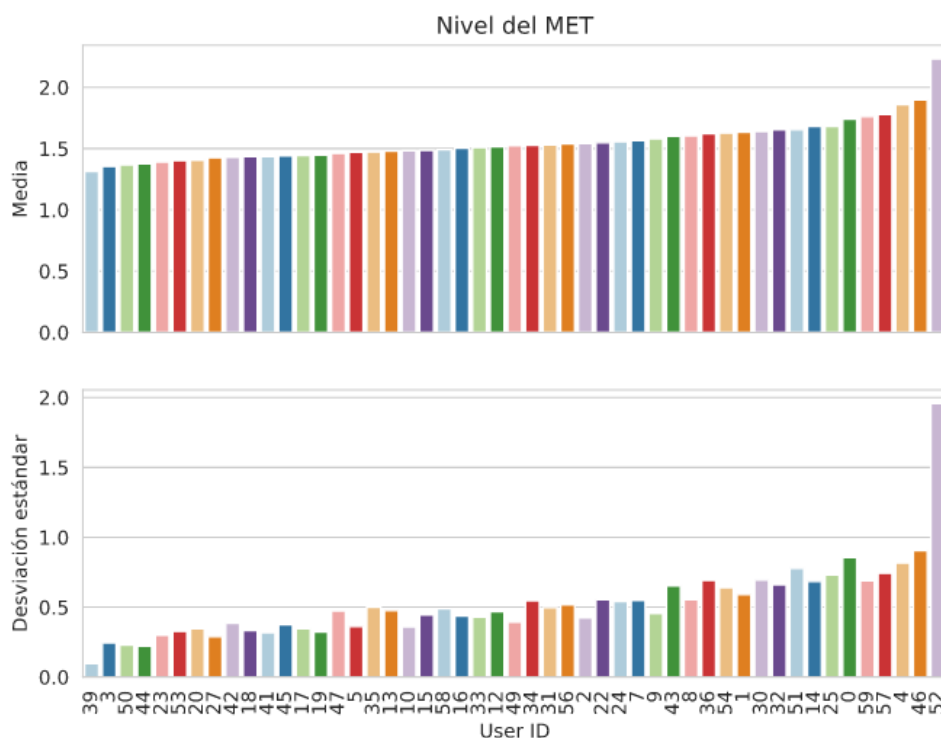


Figura 4.5. Promedio y desviación estándar del nivel de MET

Si se observa el gráfico de barras que muestra la desviación estándar, podemos ver que crece aproximadamente proporcionalmente a medida que crece el promedio de MET. Esto puede deberse a que estos usuarios presentan más *buckets* en los cuales llevaron a cabo actividades físicas. Por lo tanto, esto explicaría la diferencia en la desviación estándar entre los diferentes usuarios, donde los que practican menos actividad física son aquellos que tienen el menor valor para este estadístico.

También, puede interpretarse este crecimiento en la desviación estándar proporcional a la media del gasto energético con respecto a cuan rutinario es cada usuario. Por lo tanto, estos gráficos parecen mostrar que cuanto más activo es un usuario, menos rutinario es. Esta hipótesis puede comprenderse mejor si se muestra cuán activo es un usuario a lo largo de las horas del día y los días de la semana. Para ello, se generaron mapas de calor con el promedio y la desviación estándar del nivel de MET para todos los usuarios de acuerdo al día de la semana y las horas del día. Para llevar a cabo esto, se agruparon los *buckets* por día de la semana (eje *y*) y la hora del día (eje *x*). Los mapas de calor que muestran el promedio del nivel de comportamiento sedentario ya habían sido usados en otros trabajos pero sin tener en cuenta el nivel de MET. En esos trabajos relacionados (He & Agu, 2016a, He & Agu, 2016b, He & Agu, 2016c) los mapas de calor fueron generados a partir del porcentaje de actividades clasificadas como estacionarias en cada bucket de 1 hora. Los mapas de calor generados a partir de la desviación estándar de cada bucket de 1 hora no han sido aún analizados en otros trabajos relacionados.

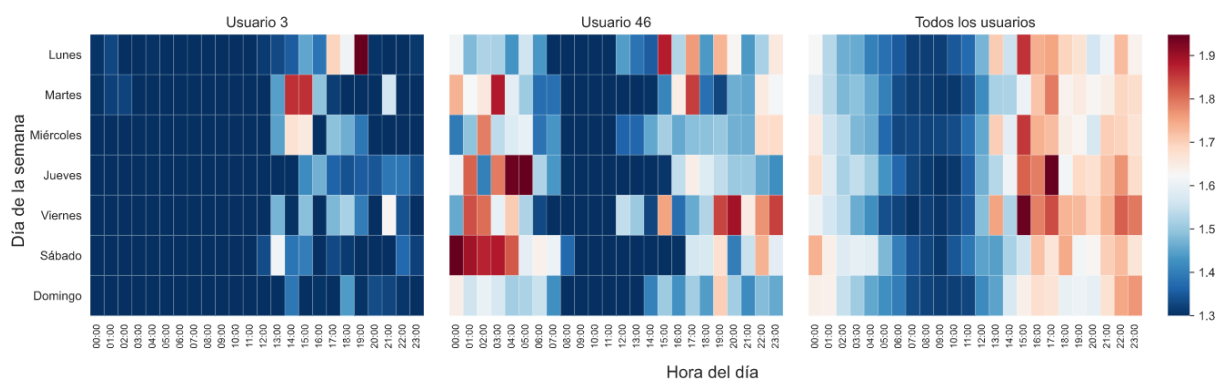


Figura 4.6. Mapa de calor del promedio del nivel de MET

Tanto en la Figura 4.6 como en la Figura 4.7, se muestran 3 mapas de calor, correspondientes a la media y a la desviación estándar respectivamente. El primer par de mapas corresponden al usuario 3, que es el segundo con menor MET promedio (después del usuario 39, que no fue utilizado para realizar los mapas de calor debido a la poca cantidad de *buckets* disponible). El segundo usuario seleccionado fue el 46, que es el segundo con mayor MET promedio. El usuario con mayor MET promedio fue el usuario 52 y no fue utilizado para realizar los mapas de calor debido a las inconsistencias halladas en los datos de actividad reportados, ya que se identificaron periodos de alto gasto energético consecutivo durante más de 3 días. El último par de mapas de calor corresponde a todos los usuarios. Es decir, para generarlos se utilizan todos los *buckets* disponible de todos los usuarios. Puede observarse que el usuario 3 tiene un comportamiento sedentario todos los días de la semana antes de las 13:00 horas. Mientras que el usuario 46 muestra un alto nivel de MET para las primeras horas del día, sobre todo la madrugada del sábado.

Esto puede deberse a que dicho usuario asistiera a eventos sociales en esas horas o que realizaba ejercicio nocturno. Además puede comprobarse que estas actividades no eran llevadas a cabo todas las semanas ya que presentan una alta desviación estándar.

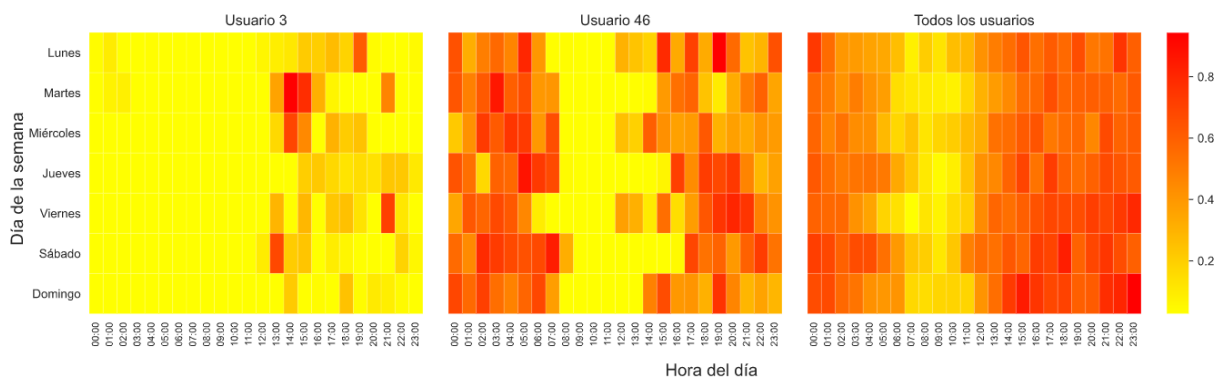


Figura 4.7. Mapa de calor de la desviación estándar del nivel de MET

Como puede observarse, las horas del día con el mayor comportamiento sedentario (eso es, niveles de MET menores a 1,5) se corresponden, también, con una desviación estándar baja. Por su parte, las horas del día con el menor comportamiento sedentario (eso es, niveles de MET mayores a 1,5) se corresponden, también, con una desviación estándar alta. Este fenómeno puede notarse tanto en los usuarios 3 y 46, como en el caso general para todos los usuarios. Una baja desviación estándar sugiere que el comportamiento sedentario observado en estos mapas de calor corresponde a un comportamiento de rutina, mientras que el comportamiento no sedentario no forma parte de la rutina de los usuarios debido a la alta desviación estándar.

Para sostener la hipótesis anterior, se calculó la correlación de *Pearson* entre la media y la desviación estándar para todos los usuarios y para todas las posibles combinaciones hora/día de la semana. Luego, se calculó el promedio de la correlación y resultó ser de 0,87, lo que sirve de soporte a la observación sobre que el comportamiento sedentario está correlacionado positivamente con el comportamiento rutinario. Dado que los usuarios que participaron en los experimentos son estudiantes, esta correlación puede ser explicada por el hecho de que dos posibles comportamientos rutinarios que tienen un bajo nivel de gasto energético son las horas de sueño y los horarios de los cursos. La información sobre las clases a las cuales los usuarios asistieron forma parte del *dataset*, por lo que se verificó que los horarios de cursada de los estudiantes se corresponden con los horarios de bajo gasto energético. En resumen, los usuarios tienden a ser más sedentarios en actividades rutinarias que en actividades no rutinarias. Esta observación deja lugar a la hipótesis de que el comportamiento sedentario es más predecible que el comportamiento no sedentario en términos de variables de tiempo.

4.3.2. Inconsistencias encontradas

Puede verse que el usuario 52 se diferencia de los demás en ambos gráficos, ya que posee un promedio de más de 2 METs y una desviación estándar mucho más elevada que la de los demás, cercana a 2 METs. Esto quiere decir que, en promedio, el gasto energético de cada hora de este usuario se aleja en 2 METs de la media. Por lo tanto, puede deducirse que este usuario tiene muchas horas en las que presenta

un alto gasto energético. Si se analizan gráficos en los que se muestre qué actividad llevó a cabo este usuario a lo largo del tiempo pueden apreciarse inconsistencias. Por ejemplo, en la Figura 4.8, se muestra el porcentaje de cada actividad (como suma acumulativa) en el periodo de 2013-05-22 al 2013-05-28. Puede observarse que, según los datos disponibles para este usuario en ese periodo de tiempo, el usuario 52 estuvo corriendo por 4 días consecutivos lo cual se deba posiblemente a un error en la aplicación de sensado Jigsaw. Además, en la figura, se muestran líneas verticales para cada hora en la que no se posee información de actividad del usuario. Como puede observarse, a continuación de la inconsistencia anterior no se dispone de ningún *bucket* del día lunes.

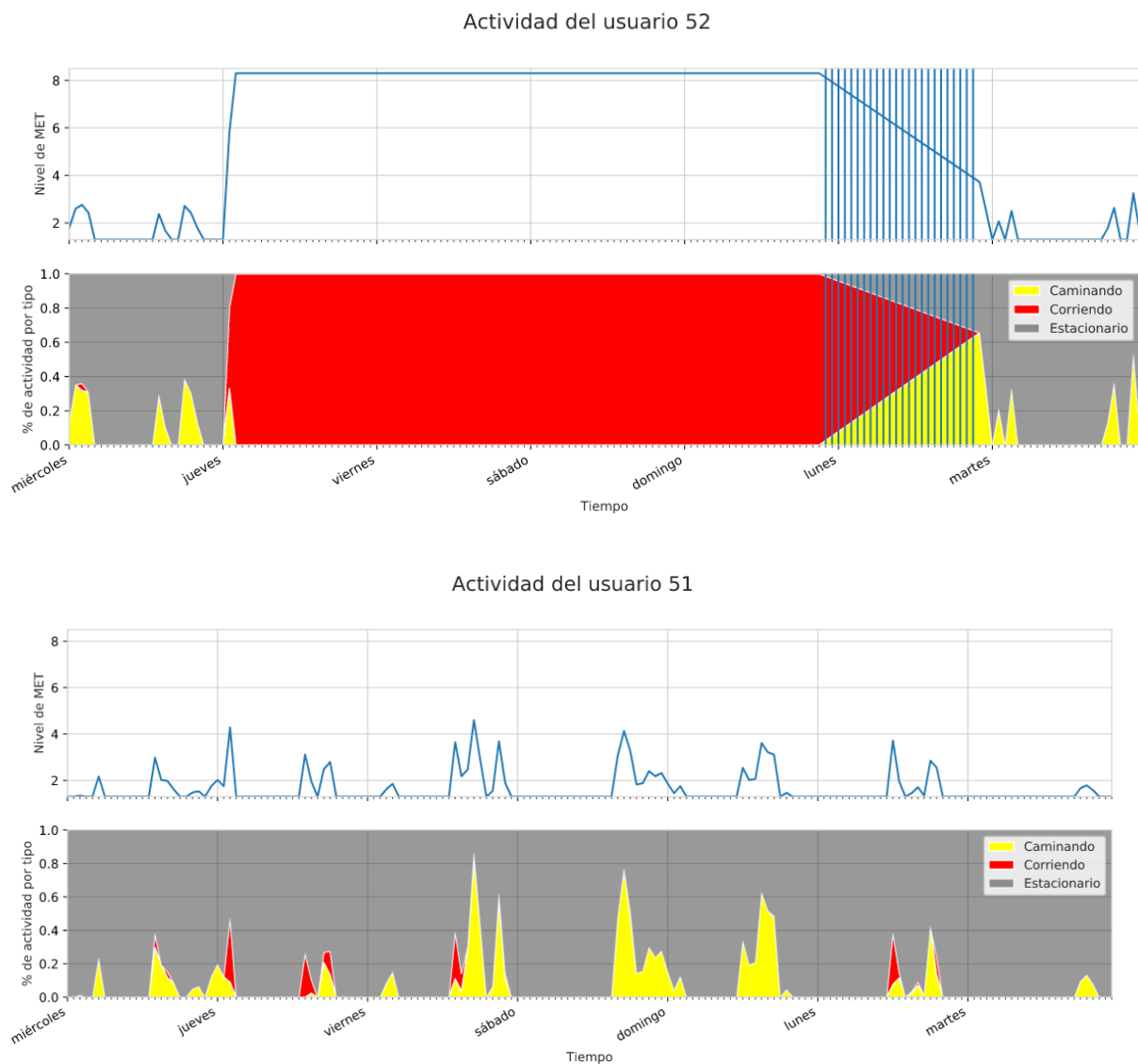


Figura 4.9. Actividad física del usuario 51

Las inconsistencias detalladas en el párrafo anterior se repiten en diferentes porciones temporales de los datos disponibles del usuario 52. Por esta razón, se decidió eliminarlo y no utilizarlo para entrenar los modelos propuestos. Esta decisión se toma en base a que los algoritmos que utilicen los datos de este usuario para realizar predicciones están aprendiendo a partir de una distribución que no corresponde ni al del usuario 52, ni al de ningún otro usuario, por lo que empeora el desempeño de los modelos.

A modo de comparación, en la Figura 4.9 se muestra un ejemplo de la actividad llevada a cabo en una semana por el usuario 51. Puede observarse que hay muchos espacios temporales donde el usuario se encuentra totalmente estacionario, lo que puede deberse tanto a horas de sueño como horas de ocio o asistencia a cursos. De hecho, en la figura se observa que las primeras horas del día son, en general, completamente estacionarias, con la excepción del sábado donde el usuario pudo haber llevado a cabo alguna actividad social. Este tipo de patrón es el que está presente en la mayoría de los usuarios.

4.3.3. Procesamiento del *dataset*

En esta sección se describe el proceso llevado a cabo a partir del *dataset StudentLife* con la finalidad de ser utilizado por los diferentes modelos planteados para resolver el problema de PCSF. Como se ha mencionado con anterioridad, cada registro del *dataset StudentLife* posee un *timestamp* asociado. Por lo tanto, todas las características que se generaron corresponden a una combinación de usuario/*bucket* en particular y representan cálculos que resumen ciertos aspectos de los datos asociados a dicha combinación. Por ejemplo, un intervalo específico puede corresponder al usuario 10 y el *bucket* 2013-04-24 19: 00–20: 00.

A continuación, se describe la serie de etapas que atraviesa el procesamiento del *dataset StudentLife*. En la Figura 4.10 se muestran las etapas junto con sus dimensiones para un *dataset* de ejemplo con 4 *lags*. Las etapas son: generación de características, tratamiento de variables categóricas, generación de *datasets* con *lags*, ajustes en la dimensionalidad de los casos de entrenamiento, separación entre casos de entrenamiento y testeo y, por último, separación entre características y variable objetivo.

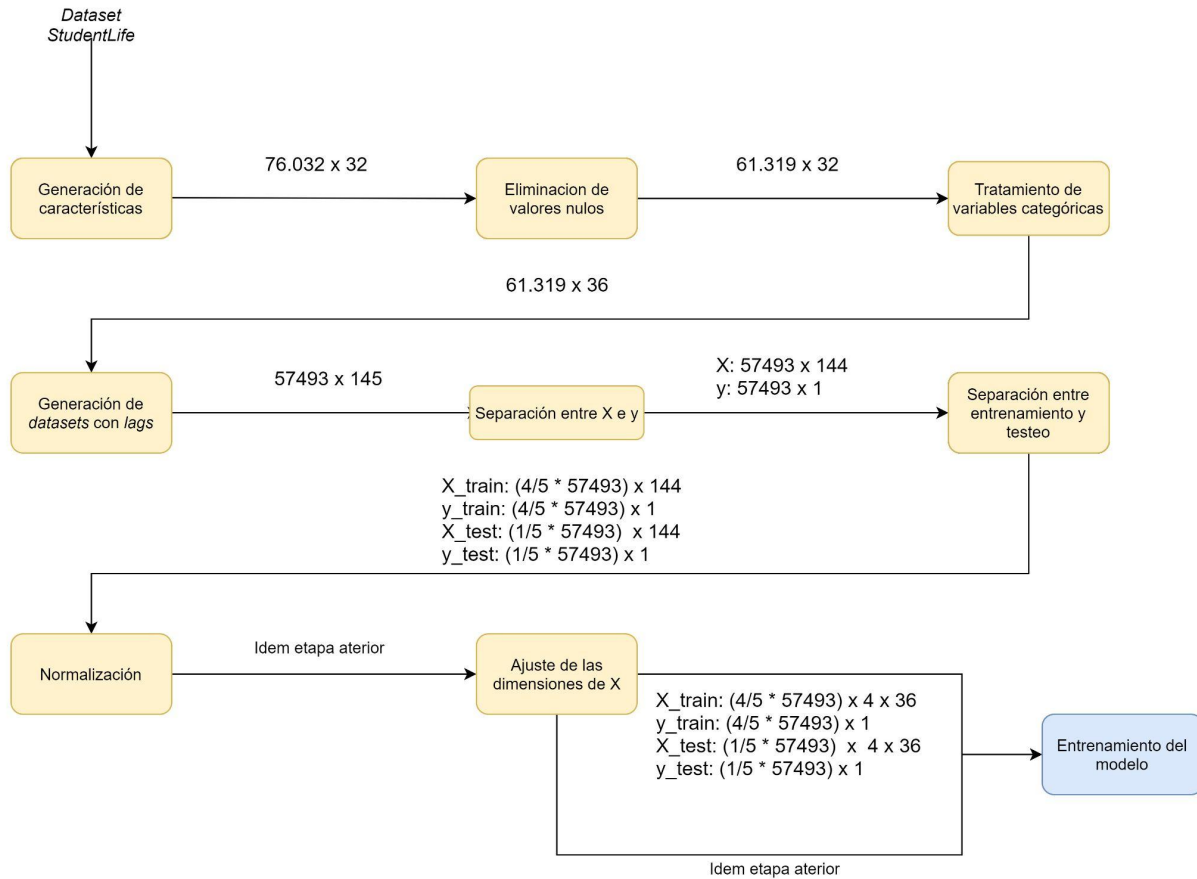


Figura 4.10. Etapas del procesamiento

4.3.3.1. Selección y cómputo de características

En esta sección, se definen las características que se generarán a partir del *dataset StudentLife*. Cada una de las características fue seleccionada considerando los siguientes aspectos:

- Características generadas en base a investigaciones previas (“Activity Prediction,” 2015; He & Agu, 2016b)
- Características generadas en base al análisis realizado en la sección anterior.
- Características que no incluyen información directamente ligada a la vida estudiantil, con la finalidad de que los resultados puedan ser más universales y no únicamente ligados a los estudiantes.

A su vez, se definen dos tipos de características: aquellas que fueron generadas a partir de los sensores de tipo TSD (Tipo de sensado discreto) y aquellas que fueron generadas a partir de sensores TSI (Tipo de sensado por intervalo). Estos dos tipos de características fueron llamadas Características Discretas y Características por Intervalo, respectivamente. La información utilizada para computar cada

una de las siguientes características está ligada al tamaño de bucket que se esté utilizando (ej.: 30 min, 1h., etc.).

4.3.3.1.1. Características Discretas

- Características obtenidas a partir del GPS (Saeb et al., 2015, 2016):
 - *location_variance*: utilizado para medir la variabilidad en la ubicación GPS de un participante en cada *bucket*.

$$\text{Varianza de la ubicación} = \log(\sigma_{lat}^2 + \sigma_{long}^2)$$

- *location_mean*: similar a la característica anterior, pero con respecto al promedio.

$$\text{Promedio de la ubicación} = \log(\text{mean}_{lat} + \text{mean}_{long})$$

- *speed_mean*: promedio de la velocidad instantánea, cómo se computa en (Saeb et al., 2015).

$$V = \frac{1}{n} \sum \left(\sqrt{\left(\frac{lat_i - lat_{i-1}}{t_i - t_{i-1}} \right)^2 + \left(\frac{long_i - long_{i-1}}{t_i - t_{i-1}} \right)^2} \right)$$

- *speed_variance*: varianza de la velocidad instantánea, cómo se computa en (Saeb et al., 2015).
- *total_distance*: desplazamiento geográfico total.

$$\text{Distancia total} = \sum \sqrt{(lat_i - lat_{i-1})^2 + (long_i - long_{i-1})^2}$$

Para el cálculo de las últimas 3 características se tomaron una serie de decisiones importantes, que se detallan a continuación. Primero, en todos los casos en los que es necesario calcular las diferencias entre dos registros consecutivos (por ejemplo, entre lat_i y lat_{i-1}), hay casos en que el registro anterior al actual no existe. Esto se da en los casos en que el registro i es el primero de un usuario y cuando es el primero del *bucket*. En el caso del primer registro del usuario, los mismos fueron eliminados. Para el caso en el que el registro sea el primero del *bucket*, se decidió no descartarlos, ya que es muy baja la cantidad de registros disponibles para el sensor GPS. El cálculo para dichos registros utiliza el último registro del *bucket* anterior, lo que por un lado puede parecer erróneo, ya que se está mezclando la información de dos *buckets* diferentes, pero por otro lado estos dos registros (el primero del *bucket* i y el último del *bucket* $i - 1$) están estrechamente relacionados ya que se da en muchos casos en los que se aporta información complementaria al *bucket* en cuestión. Tomando esta última decisión, se descartan 49 registros (uno por usuario), en lugar de 59.223 (una por cada *bucket* disponible para el sensor de GPS, como se muestra en la Tabla 4.1).

Segundo, para el cálculo de *speed_variance*, cuando se calcula la varianza, $\sigma^2 = \frac{\sum(x_i - x)^2}{N-1}$, si el *bucket* contiene solo un registro y la velocidad espontánea para ese registro es 0 (posiblemente porque el usuario permanece siempre en el mismo sitio) se da que $\sigma^2 = \frac{0}{0}$, que da como resultado indeterminado. La forma de expresar computacionalmente este valor es *NaN* (*not a number*, no es un número). Obviamente, estos valores no pueden ser almacenados en el *dataset* generado, por lo que se tomó la decisión de reemplazar estos valores *NaN* por 0 (cero).

Por último, como sucede con muchos de los sensores, hay *buckets* para los cuales no hay ningún registro disponible. Como ya ha sido discutido, en algunos casos es posible interpretar la ausencia de datos de manera que esos *buckets* no deban ser descartados. Según la descripción del motor de sensado Jigsaw (Lu et al., 2010), con el objetivo de utilizar la menor cantidad de batería, al mismo tiempo que minimizando el error del GPS, este sensor puede permanecer inactivo hasta que se detecte que el usuario está en movimiento a partir del acelerómetro. Por lo tanto, puede interpretarse que para los *buckets* para los cuales no existen registros del sensor GPS, el usuario permanece en el mismo sitio que la hora anterior. De esta forma, las características generadas a partir del sensor GPS para los cuales no existen registros disponibles se completa de la siguiente manera:

- *location_variance*: se completa con 0.
- *location_mean*: se completa con el valor de esta característica para la hora anterior.
- *speed_mean*: se completa con 0.
- *speed_variance*: se completa con 0.
- *total_distance*: se completa con 0.

- Características basadas en el tiempo:

Como se mostró en el análisis del *dataset*, las características temporales pueden llegar a tener un rol muy importante al momento de predecir el comportamiento sedentario de un usuario, debido a que, por lo general, estos siguen patrones temporales. Estos patrones ya fueron explorados en otros trabajos relacionados y es posible que las redes neuronales puedan sacar provecho de eso.

- *past_minutes*: el número de minutos transcurridos desde el comienzo del día.
- *remaining_minutes*: el número de minutos que quedan para terminar el día.
- *second_sine*: transformación de seno de la cantidad de segundos pasados a partir del comienzo del día.

$$segundos_{seno} = seno\left(\frac{2\pi segundos}{24*60*60}\right)$$

- *second_cosine*: transformación de coseno de la cantidad de segundos pasados a partir del comienzo del día.

$$segundos_{\text{coseno}} = \text{coseno}\left(\frac{2\pi \text{segundos}}{24*60*60}\right)$$

- *weekday_sin*: transformación de seno del día de la semana.

$$weekday_{\text{sin}} = \text{seno}\left(\frac{2\pi \text{dia}}{7}\right)$$

- *weekday_cos*: transformación de coseno del día de la semana.

$$weekday_{\text{cos}} = \text{coseno}\left(\frac{2\pi \text{dia}}{7}\right)$$

Estas últimas 4 características tienen como objetivo darle a los modelos de *Machine Learning* la posibilidad de comprender la naturaleza cíclica de las variables relacionadas al tiempo. Sin realizar esta transformación, los modelos de *Machine Learning* no tienen la información necesaria para comprender que entre el primer valor posible y el último existe la misma distancia que entre cualesquiera otros dos valores consecutivos. Es decir, en el caso de los días de la semana, la diferencia entre martes y miércoles, es la misma que entre domingo y lunes. El problema subyace en que, en informática, los días de la semana se expresan como números naturales -los cuales no son cíclicos- donde el número 0 representa al domingo y el número 6 al sábado (esto puede variar dependiendo de las implementaciones).

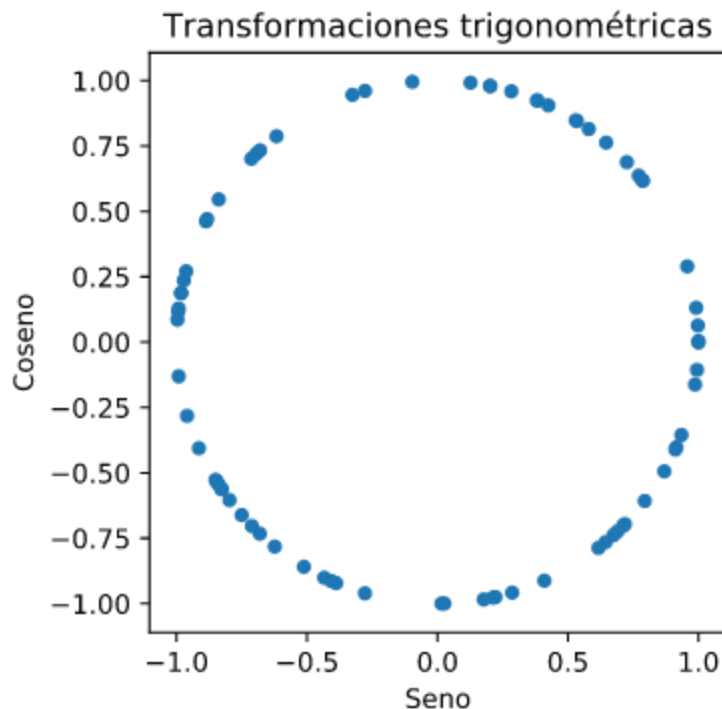


Figura 4.11. Ejemplo de transformación de seno y coseno

La transformación llevada a cabo consiste en aplicar por un lado el seno, y por otro el coseno, a la variable cíclica estandarizada (con valores entre 0 y 1) multiplicada por 2 radianes (2π). Es decir, primero se transforma a la variable categórica como un ángulo entre 0 y 2π , y luego se le aplican las dos funciones trigonométricas. De esta forma, observando el seno como el coseno, el primer y el último valor van a ser similares, ya que ambas funciones trigonométricas son periódicas. Aunque con una de las dos funciones alcanzaría para plasmar el carácter cíclico de las variables de tiempo, se da el fenómeno de que hay diferentes partes del día que no se diferencian entre sí. Por ejemplo, con respecto al tiempo pasado desde el comienzo del día, si se utiliza el seno, las 20 horas tienen el mismo valor que las 0 horas. El mismo fenómeno se da para el coseno y las demás variables. Por lo tanto, es importante usar ambas transformaciones para completar la información y permitir a los modelos diferenciar todos los diferentes valores de las características basadas en el tiempo, a su vez que permitir deducir que la distancia entre cada par de valores consecutivos es la misma, incluyendo al primer y último valor. En la Figura 4.11 se muestra un gráfico de dos dimensiones, donde el eje x representa el valor de transformar la cantidad de segundos pasados desde el comienzo del día a partir del coseno, mientras que el eje y muestra la misma transformación, pero a partir del seno. Como puede observarse los puntos toman la forma del círculo o de un reloj, denotando así su carácter periódico.

- Características de actividad física:

- *stationary_count*: la cantidad de instancias de actividad física clasificadas como "estacionario/a" en cada *bucket*.
- *walking_count*: la cantidad de instancias de actividad física clasificadas como "caminando" en cada *bucket*.
- *running_count*: la cantidad de instancias de actividad física clasificadas como de "corriendo" en cada *bucket*.
- *total_activity_count*: la cantidad de instancias en cada *bucket*.
- *activity_major*: el tipo de actividad física con más instancias en cada intervalo.

- Características de audio:

- *silence_count*: la cantidad de instancias de audio clasificadas como "silencio" en cada intervalo.
- *voice_count*: la cantidad de instancias de audio clasificadas como "voz" en cada intervalo.
- *noise_count*: la cantidad de instancias de audio clasificadas como "ruido" en cada intervalo.
- *total_audio_count*: la cantidad de instancias de actividad en cada *bucket*.
- *number_of_conversations*: el número de conversaciones que ese estudiante tuvo en cada intervalo.

- Otras características

- *wifi_changes*: el número de cambio de conexiones wifi en cada intervalo.

4.3.3.1.2. Características por intervalo

- *is_Locked*: si el *smartphone* está bloqueado;
- *Locked_ratio*: representa la proporción del *bucket* en la cual el *smartphone* estuvo bloqueado.
- *is_in_dark*: si el *smartphone* estaba en la oscuridad;
- *dark_ratio*: representa la proporción del *bucket* en la cual el *smartphone* estaba en un ambiente oscuro (por ejemplo, en un bolsillo).
- *is_in_conversation*: si el usuario mantuvo o no una conversación.
- *conversation_ratio*: representa la proporción del *bucket* en la cual el usuario se encontraba manteniendo una conversación.
- *conversation_number*: cantidad de conversaciones diferentes mantenidas en el *bucket*.
- *is_charging*: si el *smartphone* se estaba cargando.
- *charging_ratio*: representa la proporción del *bucket* en la cual el *smartphone* se estaba cargando.

Para realizar el cálculo de las proporciones -es decir, aquellas cuyos nombres finalizan en *ratio*- se divide iterativamente cada intervalo que supere el tamaño de *bucket* en nuevos intervalos más pequeños, hasta que todos los intervalos tengan un tamaño menor o igual al tamaño de *bucket*. Una vez hecho esto, para cada intervalo se calcula la proporción que ocupa del *bucket* al que pertenece. Luego, se dividen todos los intervalos en grupos de acuerdo al *id* del usuario y al *bucket* al que pertenecen (por ejemplo: si los *buckets* son de una hora y el intervalo comienza en 14:34:20, se utiliza la función piso para obtener el *bucket* correspondiente, que da como resultado 14:00:00). Finalmente, se suman las proporciones de cada intervalo perteneciente a cada grupo en particular.

Al calcular esta categoría se descubrieron nuevas inconsistencias en los datos (particularmente en el caso del usuario 49), ya que al realizar los cálculos existían *buckets* para los cuales la proporción en la cual el *smartphone* se estaba cargando era mayor a 1. Realizando un análisis se descubrió que había intervalos de tiempo superpuestos. Por ejemplo, para el usuario 49 existe un intervalo que va desde 00:34 hasta 08:53, mientras que otro intervalo va desde 00:13 hasta 03:22.

En la descripción del *dataset StudentLife* se afirma que todos los intervalos tienen una duración mayor a 1 hora. Si se analiza el caso de los *buckets* de 1 hora, al momento de la agrupación debería existir solo un intervalo por grupo. Sin embargo, se hallaron en total 355 grupos con más de un intervalo, lo que quiere decir que existen 355 *buckets* de una hora que presentan inconsistencias debido a la superposición de intervalos. La cantidad de *buckets* que superan la proporción de 1 son 45, por lo que la mayoría de *buckets* con inconsistencias no superan la proporción total, aunque los valores calculados resultan ser igualmente erróneos. No es posible decidir cuál de los intervalos superpuestos es el correcto, por lo que la única forma de solucionar este problema sería eliminando los intervalos inconsistentes. Esta acción implicaría eliminar los *buckets* involucrados, lo cual reduciría el tamaño del *dataset*. Por lo tanto, se decidió no descartar estos *buckets* inconsistentes, y en lugar de ello, reemplazar por 1 (ocupación total del *bucket*) aquellos *buckets* cuya proporción era superior a 1, y dejando igual los demás *buckets* inconsistentes.

4.3.3.1.3. Variable objetivo

- *sLevel*: el valor de MET para el *bucket*.

Esta variable es llamada variable objetivo. Es decir, es la variable que los modelos de *Deep Learning* tratarán de predecir a partir de todas las demás características. A pesar de que *sLevel* es la variable objetivo, también es usada como característica. Es decir, como la tarea de predicción que se busca llevar a cabo en esta tesis es la de predecir el comportamiento sedentario futuro, el nivel de sedentarismo del momento actual y pasado es ya conocido, por lo que puede formar parte de los *datasets* de entrenamiento para ser usado por los modelos para aprender.

Salida de esta etapa de preprocesamiento: Como salida de esta etapa del preprocesamiento se tiene un *dataset* producido a partir del *dataset StudentLife*. El *dataset* está compuesto por registros. Cada registro representa a una combinación usuario/bucket única. A su vez, cada registro está compuesto por 32 campos (uno por cada una de las características listadas en esta sección, incluyendo la variable objetivo).

Representación matricial de la salida: 76.032 filas y 32 columnas (76.032×32) para una granularidad de una hora y 152.064 filas y 32 columnas (152.064×32) para una granularidad de 30 minutos.

4.3.3.2. Eliminación de valores nulos

En esta etapa de preprocesamiento se buscan y eliminan aquellos registros que presenten por lo menos un valor nulo en alguno de sus campos. Esto es necesario ya que los modelos utilizados en esta tesis necesitarán la presencia de todas las características para entrenarse y realizar predicciones. Es importante notar que la cantidad de registros que se tendrán para una granularidad de 1 hora será siempre mayor o igual a la mitad de la cantidad de registros que se tendrán para una granularidad de 30 minutos. Esto se debe a que la cantidad de registros que se pierdan al llevar a cabo la eliminación de valores nulos siempre será mayor o igual en una granularidad de 30 minutos. Esto se debe a que, dado un *bucket* válido de una hora, si se lo separa en dos *buckets* de 30 minutos, existe la posibilidad de que uno de ellos no sea válido y deba descartarse.

Salida de esta etapa de preprocesamiento: Como salida de esta etapa del preprocesamiento se tiene un *dataset* con una estructura similar a aquella de la salida de la etapa anterior, solo que con menos registros.

Representación matricial de la salida: 61.319 filas y 32 columnas (61.319×32) para una granularidad de una hora y 122.015 filas y 32 columnas (122.015×32) para una granularidad de 30 minutos. Obsérvese que $61.319 \times 2 \geq 122.015$ y $61.319 \times 2 - 122.015 = 619$.

4.3.3.3. Tratamiento de variables categóricas

Como las redes neuronales no pueden recibir como datos de entrada variables de tipo categórica (sobre todo si estas no están representadas en forma numérica), es necesario utilizar un método para convertirlas o codificarlas en un tipo de variable que sea aceptable por el modelo. Además, no solo debe ser aceptable por el modelo, sino que tiene que representar de alguna manera la mayor información posible de manera que el modelo pueda utilizarla para realizar las predicciones. Una vez generadas las características, el *dataset* resultante posee dos características de tipo categóricas: *day_of_week* y *activity_major*. Es importante notar que dichas variables son nominales y no ordinales, ya que no es posible determinar un orden entre ellas. Si estas variables fuesen nominales sería más fácil convertirlas, ya que compartirían una característica con el resto de las variables numéricas, un orden entre ellas.

Hay muchos métodos que permiten codificar variables categóricas. Algunos de los más comunes son: *Label Encoding*, *One-Hot Encoding*, *Dummy Encoding*, *Target Encoding*, entre otras. La utilización de uno por sobre otro depende del *dataset* que se esté utilizando. En esta tesis se decidió aplicar *Dummy Encoding* sobre las 2 variables categóricas presentes en el *dataset* procesado. *Dummy Encoding* consiste en generar $i - 1$ variables, siendo i la cantidad de categorías diferentes que posea la variable categórica en cuestión. Las nuevas $i - 1$ variables serán vectores pertenecientes $\mathbb{R}^{n \times 1}$ donde el componente j - con $0 \leq j \leq n-1$ del vector tomará el valor de 1 si la variable categórica tiene el valor i para el caso j . Mientras que la última categoría estará representada por el vector donde todos sus componentes son 0.

Finalmente, con esta transformación, se añadieron 6 nuevas características y se descartaron 2. Esta es la última transformación que se le hace a cada *bucket* en particular.

Salida de esta etapa de preprocesamiento: Como salida de esta etapa del preprocesamiento se tiene un *dataset* donde cada registro está compuesto por $32 + 6 - 2 = 36$ campos. La cantidad de registros no se ve modificada.

Representación matricial de la salida: 61.319 filas y 36 columnas (61.319×36) para una granularidad de una hora y 122.015 filas y 36 columnas (122.015×36) para una granularidad de 30 minutos.

4.3.3.4. Generación de *datasets* con *lags*

El siguiente paso del procesamiento consiste en generar los *datasets* con *lags*. En esta etapa del procesamiento se prepara el *dataset* para que pueda ser utilizado para entrenar y testear los modelos. Dos parámetros intervienen en este paso: la cantidad de *lags* y el valor de los periodos. Cuando se realicen los experimentos, diferentes valores para estos parámetros serán evaluados en pos de estudiar cuál de ellos es el que alcanza un mejor desempeño y cuáles funcionan mejor con cada arquitectura.

Para llevar a cabo esta etapa del procesamiento, se toma cada uno de los *buckets* b disponibles como si representara el momento presente. A continuación, se prepara la lista de *buckets* anteriores a b que serán utilizados a partir de su ubicación temporal relativa de b . Para preparar dicha lista, se tienen en cuenta la cantidad de *lags* y el valor del periodo. Esta lista va a definir de qué información se valdrán los modelos para predecir el nivel de MET del momento presente, es decir, el valor de MET de b .

A continuación, se concatenan los *buckets* seleccionados al *bucket* actual, al mismo tiempo que se eliminan todas las características del *bucket* actual salvo la variable objetivo. La cantidad de unidades de tiempo que cada registro del *dataset* abarca es de $\text{granularidad} \times \#lags$. También es posible calcular el intervalo de tiempo que abarca cada registro de un *dataset* determinado. Dicho intervalo está dado por $\text{granularidad} \times \#lags \times \text{período}$. Por lo tanto, el intervalo de tiempo que abarca cada registro es mayor o igual a la cantidad de unidades de tiempo de la cual posee información. Otro punto importante a notar es que la cantidad de característica que posee cada registro del *dataset* es $\#características \times \#lags$, por lo que la cantidad de características crece directamente proporcional al número de *lags*. Esto puede traer problemas si la cantidad de casos de entrenamiento es baja ya que puede presentarse el problema conocido como *curse of dimensionality* o la maldición de la dimensionalidad, donde el espacio dimensional es tan grande que no se dispone de la cantidad suficiente de casos de entrenamiento como para que se posean ejemplos regularmente espaciados de forma que dicho espacio dimensional quede relativamente ocupado. Gracias a la introducción del período, los modelos pueden obtener información de un pasado lejano sin aumentar el número de características.



Figura 4.12. Ejemplos de datasets con lags

En la Figura 4.12 pueden observarse 3 ejemplos de registros de diferentes *datasets* con *lags*. Las celdas verdes representan *buckets* utilizados por el modelo, mientras que el *bucket* rojo es el que representa el momento presente y del cual el modelo va a intentar predecir el nivel de MET. En a) el *dataset* tiene 8 lags y un periodo de 1. En b) el *dataset* tiene 2 lags y un periodo de 2. En c) el *dataset*

tiene 4 *lags* y un periodo de 4. Suponiendo una granularidad de 1 hora, puede apreciarse el efecto de la utilización de diferentes valores del periodo. Por ejemplo, aunque en a) el *dataset* tiene el doble de *lags* que en c), el intervalo de tiempo que es abarcado en c) es de 16 horas, la mitad que en a). Los *buckets* seleccionados en cada caso son concatenados y dados de entrada al modelo para realizar una predicción del MET b_p , llámese $\hat{y}$. Luego, el MET de b_p , llámese y , es utilizado para estudiar cuán acertada fue la predicción $\hat{y}$. Esto se lleva a cabo a partir de una función de error, como MSE.

Es importante recordar que muchos de los *buckets* se encuentran ausentes ya que una o más de las características no están presentes. De esta forma, si alguno de los *buckets* que forman parte de la lista de *buckets* a concatenar no está disponible, el registro entero se descarta. Por lo tanto, cuanto mayor sea la cantidad de *buckets* pertenecientes a dicha lista, mayor la probabilidad de que la lista entera deba ser descartada por la ausencia de uno de los *buckets* pertenecientes a ella.

En la práctica, el proceso de generar los *datasets* con *lags* se lleva a cabo como la traslación del *dataset* completo en una sola operación y no de manera iterativa, ya que de esta manera el proceso es paralelizable y mucho más rápido. Entonces, se genera una lista con los tiempos -por ejemplo: [16, 12, 8, 4] en el caso de c) de la Figura 4.12- relativos a partir del número de *lags* y el valor del periodo. Luego, para cada elemento de esa lista se realiza la traslación del *dataset*. Luego, todos los *datasets* son concatenados.

Salida de esta etapa de preprocesamiento: Como salida de esta etapa del pre-procesamiento se obtiene un *dataset* donde cada registro está compuesto por la variable objetivo que representa al momento presente y las características de uno o más *buckets* del pasado.

Representación matricial de la salida: En este punto del preprocesamiento, la cantidad de filas variará de acuerdo al número de *lags* y del valor del periodo. Diferentes combinaciones tendrán diferencias en cuanto a la cantidad de filas disponibles. Además, el número de *lags* determinará la cantidad de columnas de la matriz. En este punto, el *dataset* está representado por una matriz de dimensiones $n \times (36 \times \#lags) + 1$, donde n es el número de filas. Por ejemplo, si el número de *lags* es de 4 y el periodo es 2 se tiene una matriz de 57493×145 para una granularidad de una hora y de 117.295×145 para una granularidad de 30 minutos. Por dar otro ejemplo, si el número de *lags* es de 2 y el periodo es 1 se tiene una matriz de 60.227×73 para una granularidad de una hora y de 12.0692×73 para una granularidad de 30 minutos. Nótese cómo al aumentar el número de *lags* disminuye la cantidad de filas y aumenta la cantidad de columnas.

4.3.3.5. Separación entre X e y

En este punto, se procede a separar al *dataset* entre las características -que serán utilizadas por los modelos para predecir- y la variable objetivo. Respetando la convención en el área de *Machine Learning*, se llama X al conjunto resultante de características e y al vector cuyos componentes representan el valor de la función objetivo para cada caso.

Salida de esta etapa de preprocesamiento: Como salida de esta etapa del preprocesamiento se tiene un *dataset* de características X y un vector de la variable objetivo y .

Representación matricial de la salida: X tendrá una dimensión de $n \times (36 \times \#lags)$ e y tendrá una dimensión de $n \times 1$.

4.3.3.6. Separación entre entrenamiento y testeo

Esta etapa del preprocesamiento consiste en separar el *dataset* en 2 conjuntos. A diferencia de la etapa anterior, donde se separó al *dataset* en su dimensión horizontal (es decir, a nivel de columnas), en esta etapa se lo separa en su dimensión vertical (a nivel de filas). El primero de los conjuntos es llamado conjunto de entrenamiento y, el segundo, conjunto de testeo. El primero de ellos será utilizado para entrenar al modelo de *Deep Learning*, mientras que el segundo será utilizado para evaluarlo, es decir, para medir el desempeño alcanzado en términos de su capacidad de generalización. De esta forma, el *dataset* original queda dividido en 4 partes, ya que tanto el conjunto de características, conocido como X , y el vector de la variable objetivo, conocido como y , son divididos en conjuntos de entrenamiento y de testeo. Estos 4 conjuntos serán llamados X_{train} , y_{train} , X_{test} e y_{test} .

Como será explicado en la [Sección 4.3.4](#), la evaluación de los modelos de *Deep Learning* fue realizada mediante una técnica especialmente implementada para el problema que se aborda en esta tesis. Esto se debió a que, por un lado, se tuvo que tener en cuenta que el *dataset* es de tipo *timeseries*, y por otro, que se probaran modelos personales e impersonales. Estos dos últimos puntos impactan en la decisión correcta sobre cómo evaluar los modelos. De esta forma, la evaluación de los modelos que se llevó a cabo en esta tesis escapa de la simple separación entre casos de entrenamiento y de testeo. En la [Sección 4.3.4](#) se verá que los *datasets* de entrenamiento y de testeo pueden provenir de conjuntos de datos diferentes.

Salida de esta etapa de preprocesamiento: Como salida de esta etapa del preprocesamiento se tiene un total de 4 conjuntos de datos diferentes: un *dataset* de características X_{train} que posee los casos de entrenamiento, un *dataset* de características X_{test} que posee los casos de testeo, un vector y_{train} con los valores de la variable objetivo para los casos de entrenamiento y un vector y_{test} con los valores de la variable objetivo para los casos de testeo.

Representación matricial de la salida: Las dimensiones de las matrices serán las mismas que en la sección anterior.

4.3.3.7. Normalización

Esta etapa del preprocesamiento consiste en normalizar los datos de los *datasets* que contienen los casos de entrenamiento y testeo -esto es, X_{train} y X_{test} -. El proceso de normalización tiene la finalidad última de agilizar el proceso de optimización de los parámetros de los modelos. Esto se logra

normalizando los valores de cada una de las características, es decir, haciendo que todas las características numéricas se encuentren entre el mismo rango de valores.

Para este paso del preprocesamiento se utilizan normalizadores provistos por la librería Scikit-learn⁵. El proceso de normalización se basa en que un normalizador aprenda la transformación a realizar a partir de los datos de entrenamiento y, luego, con la transformación aprendida, aplicarla a los *datasets* de entrenamiento y testeo. El motivo por el cual se aprende la transformación de los datos sólo a partir del conjunto de entrenamiento es evitar la fuga de datos (*data leakage*). El fenómeno *data leakage* debe ser siempre tenido en cuenta para evaluar la capacidad de generalización de los predictores. En esta tesis se usa un normalizador conocido como *StandardScaler*¹⁵. *StandardScaler* consiste en calcular la media y la desviación estándar de cada característica. Luego, a todos los valores de una característica dada se le resta la media y se lo divide por la desviación estándar.

Salida de esta etapa de preprocesamiento: Como salida de esta etapa del preprocesamiento se obtienen los *datasets* X_{train} e X_{test} normalizados. Los vectores de la variable objetivo no son utilizados en esta etapa.

Representación matricial de la salida: Las dimensiones de las matrices serán las mismas que en la sección anterior.

4.3.3.8. Ajuste de las dimensiones de X

Esta etapa del preprocesamiento es aplicada sobre los *datasets* X_{train} e X_{test} solo en el caso en que ciertos modelos de *Deep Learning* los vayan a utilizar. Algunos modelos esperan que su entrada se encuentre en la forma de vectores y otros en matrices. Dicho de otra forma, en el primer caso, el *dataset* deberá estar representado por una matriz, mientras que en el segundo, el *dataset* deberá estar representado por un tensor. Un tensor es la extensión del concepto de matriz a un espacio de N dimensiones. Por diversas características, esta representación de un alto número de dimensiones puede beneficiar a algunos tipos de modelos, ya que estos pueden aprovechar mejor la estructura espacial de los datos de entrada.

En el caso de las redes Perceptrón Multicapa, se espera como entrada un vector, por lo que el *dataset* estará representado por una matriz donde las filas representan los casos y las columnas las características. Esta última representación es la que se ha mantenido a lo largo de todo el preprocesamiento. Otros tipos de modelos, como las CNN o las RNN pueden recibir como entrada matrices o incluso tensores. En lo que respecta a esta tesis, los modelos que no sean MLP esperan como entrada una matriz, por lo que el *dataset* deberá estar representado por un tensor de 3 dimensiones (una más que las matrices). Estas 3 dimensiones representan: los casos, las características y los *lags* de dichas características. Con esta organización de los datos, los modelos pueden aprovechar las características temporales. Esto es así porque gracias a esta disposición los datos de una misma característica son contiguos entre dos *lags* contiguos, hecho que no se da en la representación de matrices. De esta forma, es más fácil para los modelos -aquellos que puedan utilizar datos organizados de acuerdo a esta disposición- aprender características relacionadas a la dimensión temporal. Por ejemplo, la relación entre las características relacionadas a la actividad en dos o más *lags* consecutivos.

¹⁵ <https://scikit-learn.org/stable/modules/generated/sklearn.preprocessing.StandardScaler.html>

Salida de esta etapa de preprocesamiento: Como salida de esta etapa del preprocesamiento se obtiene una versión remodelada de los *datasets* X_{train} e X_{test} en el caso en que no vayan a ser utilizados por una arquitectura MLP. Los vectores de la variable objetivo no son utilizados en esta etapa.

Representación matricial de la salida: Las dimensiones de las matrices X_{train} e X_{test} serán $n \times (\#lags \times 36)$ o bien $n \times \#lags \times 36$.

4.3.4. Validación de los modelos

Como ya fue mostrado en el marco teórico, existen muchos métodos utilizados para validar modelos de *Machine Learning*. El objetivo de la validación es verificar el desempeño del modelo. Para validar los modelos debe discutirse cuál de los métodos existentes es el más adecuado para ser utilizado en el problema que motiva esta tesis. Como los datos que serán utilizados para entrenar y testear los modelos propuestos son de tipo *time-series*, no todos los métodos de validación son aplicables, ya que deben cumplir con ciertas características.

En la gran mayoría de los métodos de evaluación se debe dar por supuesto que las variables del *dataset* son independientes e idénticamente distribuidas, lo cual no puede asumirse en el caso de *time-series*, ya que no pueden intercambiarse casos entre sí sin modificar la distribución del *dataset*. Además, el método de validación a utilizar debe cumplir con la característica de adaptarse a la forma real en la cual los datos son registrados y puestos a disposición de los modelos. Es decir, todos los datos que se usan para entrenar un modelo deben ser anteriores a aquellos datos utilizados para testearlo. De esta forma, se simula una situación real en la cual los datos del futuro no están aún disponibles.

Por último, el método a utilizar debe adaptarse para los diferentes experimentos propuestos en esta tesis. Sobre todo, debe poder adaptarse tanto a los modelos personales como a los impersonales. Esto es necesario ya que los datos utilizados por los modelos para ser entrenados y testeados pueden o no pertenecer al mismo *dataset*.

Normalmente, para evaluar modelos entrenados a partir de *datasets* de tipo *time-series* se utiliza un método de validación cruzada para series de tiempo llamado *TimeSeriesSplit*, disponible en la librería *Scikit-Learn*. *TimeSeriesSplit* consiste en dividir el *dataset* en k particiones, respetando el orden temporal, donde cada partición tiene la misma cantidad de casos. Luego, se llevan a cabo $k - 1$ iteraciones en las cuales se re-entrena el modelo, con la particularidad de que en cada iteración el conjunto de casos de entrenamiento es un superconjunto del conjunto de casos de la iteración anterior. Más precisamente, el conjunto de casos de entrenamiento de la iteración i es igual a la unión entre los conjuntos de entrenamiento y de testeo de la iteración $i - 1$. Entonces, en la iteración i se utilizan las i primeras particiones para entrenar al modelo y la partición $i + 1$ para testearlo. El resultado final de este método de validación es la media del desempeño alcanzado por el modelo sobre el *dataset* de testeo en cada iteración. En algunos casos, la desviación estándar puede ser un estadístico útil.

El método de validación descrito en el párrafo anterior no es adecuado para ser utilizado en esta tesis por dos razones:

- Da por supuesto que los casos son observaciones en intervalos fijos de tiempo, lo cual no es aplicable a los *datasets* obtenidos a partir del *pipeline* de preprocesamiento, donde en dos intervalos de tiempo de igual longitud puede contener una cantidad diferente de casos.
- Para incluir tanto modelos personales como impersonales, es necesario que el método de evaluación pueda realizar el entrenamiento y testeo de los modelos a partir de dos *datasets* diferentes. Este requerimiento se da en el caso de los modelos impersonales, donde el *dataset* de entrenamiento pertenece a un *dataset* que incluye datos sobre muchos usuarios, mientras que el *dataset* de testeo contiene datos sobre un único usuario.

Por lo tanto, se propone un nuevo método basado en *TimeSeriesSplit* que resuelve los problemas descritos. El método de evaluación propuesto difiere en lo siguiente de método *TimeSeriesSplit*:

- El criterio para generar las particiones es que estas deben representar el mismo intervalo de tiempo y no la misma cantidad de casos. Para esto, se calculan las fechas mínimas y máximas del *dataset*. Luego, se calcula la longitud del intervalo temporal comprendido entre las dos fechas calculadas (por ejemplo 65 días y 4 horas) y se lo divide por la cantidad de iteraciones k , obteniendo así la longitud de tiempo que aborda cada partición. Una vez obtenidas estas particiones, se prosigue de la misma manera que en *TimeSeriesSplit*.
- Para permitir que el método se adapte a los modelos personales e impersonales, éste recibe como parámetro dos *datasets* diferentes, uno para el entrenamiento y otro de testeo. De esta forma, en el caso de los modelos personales, los *datasets* de entrenamiento y de testeo pertenecen al mismo usuario, mientras que en el caso de los modelos impersonales el *dataset* de entrenamiento contiene los datos de todos los usuarios salvo el del usuario objetivo mientras que el *dataset* de testeo contiene los datos del usuario objetivo.

Los pasos a seguir por el algoritmo de validación anteriormente enunciado varían de acuerdo a si el modelo es personal o impersonal, ya que esto determina de qué usuarios provienen los datos de entrenamiento y de testeo. En la Figura 4.13 se muestra el caso general de validación para un modelo impersonal. En este, se parte del *dataset* entero, que posee los datos de todos los usuarios. El primer paso consiste en seleccionar los *datasets* de entrenamiento y de testeo, donde el segundo corresponde a los datos de un usuario, y el primero a los datos de todos los demás usuarios. En este caso, el *dataset* de testeo corresponde a los datos del usuario n y el de entrenamiento a los datos del resto de los usuarios. A continuación, se generan k particiones de cada uno de los *datasets*. Las particiones con el mismo índice de ambos *datasets* corresponden al mismo intervalo temporal. Por último, se determinan los *datasets* de entrenamiento y de testeo para la iteración i . El *dataset* de entrenamiento de la iteración i estará compuesto por la unión de todas las particiones con índice menor o igual a i , de la forma $\bigcup_{t=1}^i \text{Partición}_t$, del *dataset* de entrenamiento completo. El *dataset* de testeo de la iteración i estará compuesto por la partición $i + 1$ del *dataset* de testeo completo.

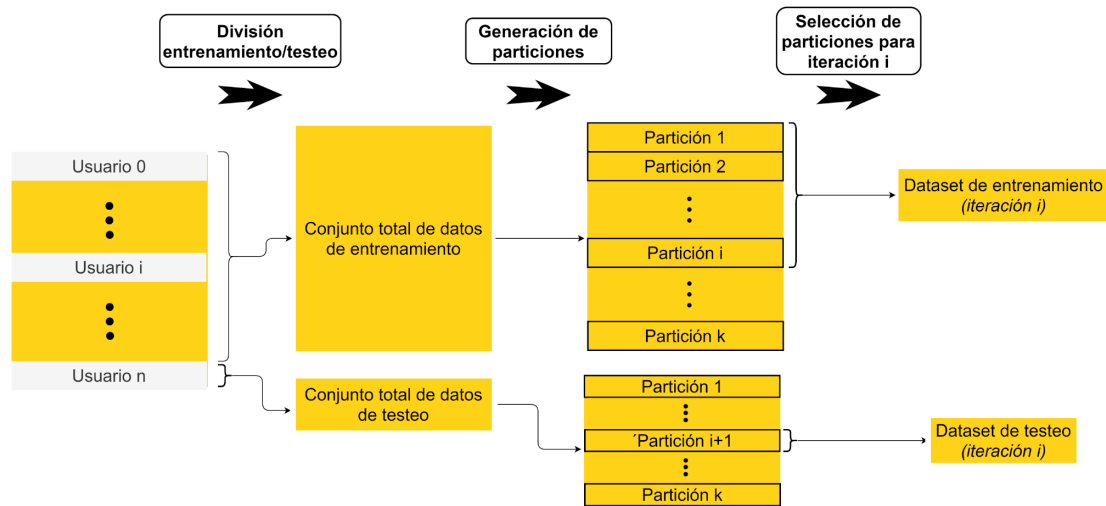


Figura 4.13. Etapas de la validación de modelos impersonales

Por su parte, en la Figura 4.14 se muestra el caso general de validación para un modelo personal. En este, al igual que en los modelos impersonales, se parte del *dataset* entero. Luego, se selecciona el conjunto del *dataset* del usuario en cuestión que corresponderá al mismo tiempo al *dataset* de entrenamiento y de testeo. He aquí la mayor diferencia entre ambos tipos de modelos, ya que en el caso de los modelos personales tanto el *dataset* de entrenamiento como el de testeo forman parte del mismo conjunto de datos. A continuación, se generan las k particiones. Para la iteración i se seleccionan las primeras i particiones para entrenar el modelo y la particiones $i + 1$ para entrenar.

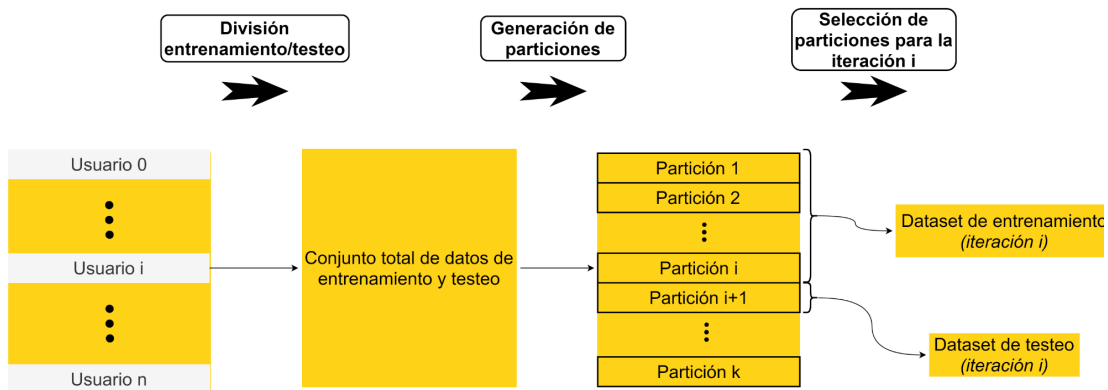


Figura 4.14. Etapas de la validación de modelos impersonales

4.3.4.1. Métricas de evaluación

La métrica utilizada para evaluar el desempeño de los modelos propuestos es el Error Cuadrático Medio o MSE (*Mean Squared Error*). El MSE es una medida de desempeño cuantitativa utilizada comúnmente para medir el error que hay entre el valor real y el valor estimado. En este contexto MSE es igual a la sumatoria de los errores cuadráticos. En comparación con el Error Medio Absoluto o MAE, MSE amplifica y penaliza con mayor fuerza aquellos errores de mayor magnitud. donde y_t es el resultado en el tiempo t y p_t es el pronóstico de valor en el tiempo t .

Es interesante remarcar que la métrica seleccionada para evaluar los modelos coincide con la función de pérdida que es optimizada al entrenar dichos modelos.

4.3.5. Tuning de los modelos

En el entrenamiento de un modelo de *Deep Learning* existen diferentes hiperparámetro que deben ser ajustados para lograr que el modelo alcance el mayor desempeño posible. Para esto, se deben probar diferentes valores para cada hiperparámetro, comparando el desempeño obtenido entrenando el modelo con cada uno de ellos. Existen diferentes formas de realizar la comparación de los diferentes valores de los hiperparámetro. Para realizar el *tuning* o “búsqueda de los hiperparámetro óptimos” en los modelos que serán comparados en la tarea de PCSF a partir del *dataset StudentLife* se utilizó la Optimización de Bayes (Agnihotri & Batra, 2020).

4.3.5.1. Optimización de Bayes

La optimización de Bayes es utilizada para buscar el máximo global en funciones analizadas como “cajas negras”, es decir aquellas que no tienen una forma especial y en las que no es posible utilizar la optimización por el gradiente. Las Redes Neuronales Profundas son un caso claro de este tipo de funciones, en las que por muchos años la mejor forma de encontrar el valor adecuado de los hiperparámetro de un modelo de *Deep Learning* fue el de utilizar una búsqueda aleatoria o simplemente evaluar todos los hiperparámetro posibles. El problema con este enfoque es que si el tiempo que se tarda en entrenar un modelo es elevado es inviable realizar una búsqueda exhaustiva de los mejores hiperparámetro.

La Optimización de Bayes puede solucionar este problema decidiendo de forma inteligente qué valores de los hiperparámetro evaluar en la siguiente iteración, encontrando un punto medio entre los siguientes dos objetivos:

- Obtener información sobre la función f con el fin de tener más información a la hora de realizar las siguientes evaluaciones. Esto se logra evaluando en los puntos para los cuales la incertidumbre es mayor (en el caso de usar un Proceso Gausiano como modelo sustituto, donde la varianza sea la mayor).
- Encontrar el máximo de la función f basándose en la información ya conocida por el modelo sustituto.

Más formalmente, la Optimización de Bayes consiste en hallar $x \in \mathcal{R}^d$ correspondiente al maximo o minimo global de una función $f: \mathcal{R}^d \rightarrow \mathcal{R}$. La Optimización de Bayes tiene las siguientes restricciones:

- El conjunto factible de f es simple.
- f es continuo, pero no es fácil de optimizar.
- La evaluación de f no da información sobre el gradiente.
- f es costosa de evaluar.
- f es ruidosa.

Teniendo en cuenta estas restricciones, la Optimización de Bayes sigue los siguientes pasos:

1. Se elige un modelo sustituto para modelar la verdadera función f y definir sus probabilidades a priori.
2. A partir de las evaluaciones de f , se utiliza la regla de Bayes (de ahí el nombre del proceso de optimización) para obtener probabilidades a posteriori.
3. Se utiliza una función de adquisición $\alpha(x)$, que es una función de la probabilidad a posteriori, para decidir cuál será la siguiente evaluación t de f . t se selecciona de tal forma que $x_t = \underset{x}{\operatorname{argmax}} \alpha(x)$.
4. Añadir la nueva observación al conjunto de observaciones e ir al paso 2. Repetir hasta que haya convergencia o hasta que se acabe el presupuesto.

En el caso donde la función f representa un modelo de *Deep Learning*, cada evaluación se basa en obtener el desempeño del modelo para un conjunto de hiperparámetro determinado, y en base al valor obtenido, actualizar el modelo sustituto para luego seleccionar otro valor para los hiperparámetro basándose en la función de adquisición.

En esta tesis, se utiliza un Proceso Gaussiano (Görtler et al., 2019) como modelo sustituto. Además, se decidió utilizar la función de adquisición llamada Probabilidad de Mejora. Se utiliza la implementación de la Optimización de Bayes provista por la librería *Scikit-Optimize*¹⁶. A cada proceso de tuning se le dan hasta 100 evaluaciones de diferentes configuraciones de hiperparámetro. Estos procesos pueden finalizar antes de las 100 evaluaciones en caso de que el modelo sustituto converja. Finalmente, el resultado del proceso de *tuning* será la configuración de hiperparámetro que haya alcanzado el mejor desempeño.

4.3.5.2. Selección de experimentos modelo

Resultaría ideal que se llevase a cabo el proceso de *tuning* para cada *dataset* diferente, lo cual resulta inviable debido a la alta cantidad de *datasets* que serán evaluados. Por esta razón, se decidió tomar ciertos experimentos como representantes de los demás. Las decisiones que se tomaron son las siguientes:

- Todos los *datasets* utilizados en el proceso de *tuning* comparten las siguientes características: granularidad de 1 hora, 4 *lags* y períodos de valor 1.

¹⁶ <https://scikit-optimize.github.io/stable/>

- Se exploran tanto modelos personales como impersonales, debido a que son la categoría que mayor diferencia implica en los *datasets*.
- Se realiza el proceso de ajuste de hiperparámetro para cada tipo de arquitectura propuesta. Esto es: MLP, CNN, TCN y RNN.
- También, se seleccionan dos usuarios representativos. Esta selección es más compleja que las demás por lo que se le dedica su propia sección ([4.3.5.2.1](#)).

De esta forma, la cantidad de arquitecturas totales resultantes del proceso de *tuning* será de $2 * 4 * 2 = 16$. Se decidió darle 100 iteraciones a cada uno de los procesos de tuning. Una vez realizadas las 100 iteraciones se toma el conjunto de hiperparámetro para los cuales el modelo alcanzó el mayor desempeño. La función que el algoritmo de optimización de bayes intenta minimizar en este caso es el error del modelo para un experimento (ver pipeline del modelo). Es decir, se lleva a cabo el pipeline completo de un experimento normal y se toma la media del desempeño alcanzado en cada una de sus iteraciones.

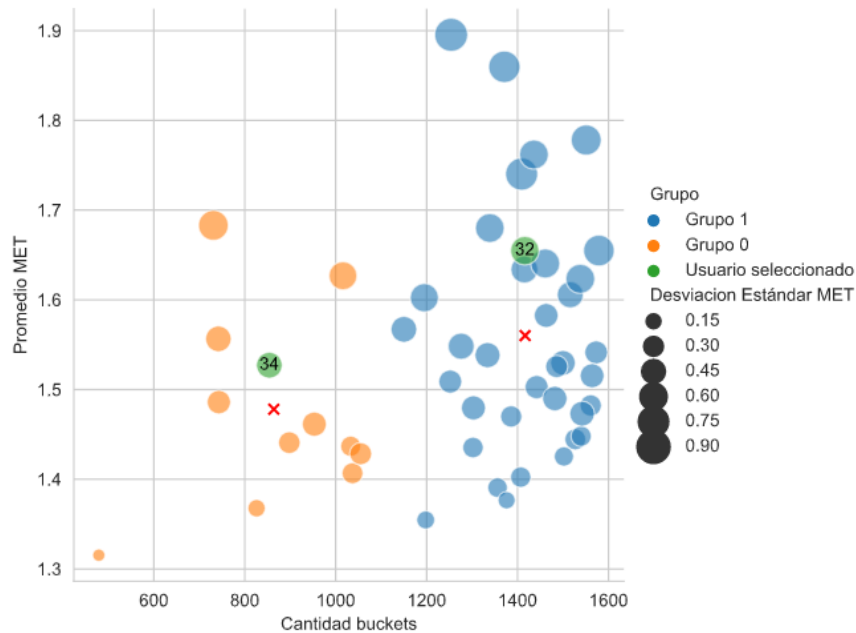


Figura 4.15. Grupos de usuarios y usuarios modelo

4.3.5.2.1. Selección de usuarios modelo

Para decidir cuáles serán los usuarios representativos se utiliza el método de *Machine Learning* no supervisado *K-Means*, que se encarga de encontrar centroides a partir de ciertas características calculadas para cada usuario. Dichas características son: promedio de MET, desviación estándar de MET, cantidad de *buckets* de 1 hora disponibles. El procedimiento funciona generando dos grupos de usuario a partir de

las características de cada uno de ellos. Se toma como el usuario modelo aquel más cercano al centroide calculado por el algoritmo.

En la Figura 4.15 se muestra un eje cartesiano donde el eje x corresponde a la cantidad de *buckets* de una hora disponibles, mientras que el eje y corresponde al promedio de MET. Además se introduce la tercera característica utilizada por el algoritmo *K-Means* representada por el radio de cada punto del eje cartesiano. Cuanto más grande el punto, mayor es la desviación estándar del nivel de MET. Se muestran en verde los usuarios seleccionados: el 34, que representa a los usuarios con bajo MET promedio y baja cantidad de *buckets* disponibles, y el 32, que representa a los usuarios con alta cantidad de *buckets* y alto MET promedio. Por último, las cruces rojas muestran la ubicación exacta de los centroides determinados por *K-Means*. Cabe aclarar que solo se muestran las primeras 2 dimensiones de dichos centroides, de manera que parece que los usuarios 32 y 34 no son los más cercanos a ellos, cuando se está obviando la última dimensión.

La idea detrás de seleccionar usuarios modelo es la de representar patrones de gasto energético diferentes a la hora de encontrar los mejores hiperparámetro para las diferentes arquitecturas y *datasets*, a la vez que se disminuye enormemente el tiempo requerido por el proceso de *tuning*.

4.3.6. Diseño de los experimentos

Como se introdujo en la [Sección 4.1](#), en esta tesis se proponen diferentes aspectos a estudiar a la hora de abordar la tarea de PCSF. Estos son: el tipo de red neuronal, las características del *dataset* (número de *lags*, granularidad y valor del periodo) y la naturaleza de los modelos (personal/impersonal). En esta sección, se presentan los experimentos que serán llevados a cabo a partir de la evaluación de diferentes valores para cada uno de los aspectos anteriores, conformando así diferentes categorías en las cuales los experimentos pueden ser clasificados. Para ello se propone una lista de valores para cada aspecto. Cada uno de los valores será puesto a prueba para cada uno de los usuarios. La Tabla 4.3 muestra todas las categorías en las que pueden ser clasificados los experimentos, así como también se muestran los posibles valores que puede tomar cada una de esas características.

Categoría	Algoritmo de aprendizaje	Naturaleza del modelo	Características del <i>dataset</i>			Usuario
			Cantidad de <i>lags</i>	Períodos	Granularidad del <i>dataset</i>	
Posibles valores	MLP, RNN, CNN, TCN	Personal o Impersonal	1,2,4,8	1,2,4	30 minutos, 1 hora	Conjunto de usuarios

Tabla 4.3. Categorías de los experimentos y sus valores

Una vez descritos los posibles valores para cada una de las categorías que caracterizan a los experimentos, es posible calcular la cantidad de experimentos a llevar a cabo. Dicha cantidad queda definida por la cardinalidad del producto cartesiano de los conjuntos de posibles valores de cada categoría en la que pueden clasificarse los experimentos. De esta forma, la cantidad de experimento a realizar es:

$$\begin{aligned}
\text{Nr. de experimentos} &= |\text{arquitecturas}| \times |\text{poi}| \times |\text{nro. lags}| \times |\text{períodos}| \times |\text{granularidad}| \times |\text{usuarios}| \\
&= |\text{arquitecturas}| \cdot |\text{poi}| \cdot |\text{nro. lags}| \cdot |\text{períodos}| \cdot |\text{granularidad}| \cdot |\text{usuarios}| \\
&= 4 \cdot 2 \cdot 4 \cdot 3 \cdot 2 \cdot 48 = 9.216
\end{aligned}$$

Por lo tanto, deben llevarse a cabo 9.216 experimentos diferentes. Puede observarse aquí porque se tomaron las medidas de la sección de Selección de los Experimentos Modelos ([Sección 4.3.5.2.1](#)): la cantidad de experimentos a realizar es muy grande como para llevar a cabo el proceso de *tuning* para cada uno de ellos. Además, también puede notarse como el aumento en la cantidad de valores de alguna categoría aumenta considerablemente la cantidad de experimentos a realizar. Por ejemplo, si se agregase un valor al conjunto granularidad la cantidad de experimentos a realizar aumentaría a 13.824.

Resumen

En este capítulo, se expusieron las cuestiones más importantes sobre cómo se aborda el problema de PCSF en esta tesis. Primero, en la [Sección 4.1](#) se presentó el enfoque sobre cómo se aborda el problema de PCSF. Con este fin, se define formalmente el problema de PCSF. Además, se exponen los modelos que serán desarrollados en base a las redes neuronales que serán utilizadas y los datos que tendrán dichos modelos para ser entrenados. Luego, en la [Sección 4.2](#) se presentó el *dataset* utilizado. Para ello se mostraron los datos disponibles en el, con especial detalle en los datos de sensado. Por último, en la [Sección 4.3](#) describieron exhaustivamente las cuestiones relacionadas a la implementación de los experimentos. Esto incluye el análisis de los datos, el preprocesamiento del dataset con la finalidad de obtener datos aptos para entrenar a los modelos, la técnica de validación de los modelos y el tuning de los mismos.

Capítulo 5: Resultados

En este capítulo se procede a presentar los resultados obtenidos. Por un lado, en la [Sección 5.1](#) se muestran los resultados del proceso de *tuning* que se lleva a cabo utilizando la optimización de Bayes. Por otro lado, en la [Sección 5.2](#) se muestran los resultados de los experimentos llevados a cabo a partir del diseño pautado en la [Sección 4.3.6](#). En esta última sección, se discuten diferentes maneras de procesar los resultados y particularmente se eligen los aspectos más importantes de los resultados.

5.1. Resultados del proceso de *tuning*

En esta sección se muestran los resultados obtenidos en el proceso de *tuning*. A modo de recapitulación: el proceso de *tuning* consistió en realizar ciertos experimentos seleccionados repetidas veces para poder encontrar las mejores combinaciones de hiperparámetros para cada una de las arquitecturas propuestas. Este proceso fue llevado a cabo a través de la Optimización de Bayes, que utiliza un Proceso Gaussiano para decidir cuál combinación de los hiperparámetro es conveniente ajustar en cada iteración.

Cada una de las arquitecturas propuestas posee diferentes hiperparámetros a ajustar, por lo que se incluye una sección para cada una de ellas. Igualmente, existen ciertos hiperparámetros comunes a todas ellas: número de épocas y el tamaño de bache, ya que no están relacionados a la arquitectura del modelo sino a la cantidad de veces que se itera sobre el *dataset* de entrenamiento y cada cuántos casos de entrenamiento se actualizan los parámetros del modelo. En todos los casos se utilizó *Adam* como función de optimización y *MSE* como función de pérdida. En total, fueron seleccionados 16 experimentos modelo. En esta sección, se los presenta categorizados por el tipo de red neuronal, mientras que para cada tipo de red neuronal se muestran los resultados de los modelos asociados. Las otras categorías que varían son la naturaleza del modelo (personal o impersonal) y el usuario modelo (usuario 32 o 34).

5.1.1. MLP

Las arquitecturas MLP son las más simples de todas, por lo que comparten algunos hiperparámetros con otras arquitecturas. Los hiperparámetros que se ajustaron para las MLP fueron: el número de capas densas, el número de unidades por capa, si usar o no *Batch Normalization* y el valor del *dropout*. En la Tabla 5.1 se muestran los resultados del proceso de *tuning* asociados a las MLP.

Usuario	Naturaleza	Nro. nodos	Nro. capas	Usa Batch Normalization	Dropout	Nro. épocas	Tam. bache	MSE
34	per	256	16	Si	0,8	64	8	0,252
32	per	4	4	No	0,368	64	8	0,361
34	imp	32	2	No	0,743	64	128	0,211
32	imp	32	2	Si	0,374	64	128	0,308

Tabla 5.1. Resultado del proceso de *tuning* para las MLP

5.1.2. CNN

En el caso de las arquitecturas CNN los hiperparámetro que se ajustaron fueron: el número de filtros, el tamaño del *kernel* y el *dropout* aplicado a la capa de convolución. Por lo general, esta arquitectura es utilizada en combinación con una o más capas densas. Por lo tanto, se decidió agregar una capa densa opcional como la última capa de esta arquitectura, por lo que el número de unidades densas también debió ser tuneado, así como también el valor del *dropout* para esta capa. En la Tabla 5.2 se muestran los resultados del proceso de tuning asociados a las CNN.

Usuario	Naturaleza	Nr. filtros	Tam. kernel	Dropout (conv)	Nr. nodos densos	Dropout (denso)	Nr. épocas	Tam. bache	MSE
34	per	256	4	0	16	0,8	64	8	0,269
32	per	128	8	0,041	8	0,589	64	8	0,383
34	imp	256	16	0,8	64	0,306	64	256	0,21
32	imp	4	16	0	64	0,8	32	256	0,305

Tabla 5.2. Resultado del proceso de tuning para las CNN

5.1.3. TCN

Como las TCN son similares a las CNN, comparten ciertos hiperparámetros. En esta arquitectura, los hiperparámetros que se ajustaron fueron: el número de filtros, el tamaño del kernel, el valor del *dropout*, si usar o no *Batch Normalization* y si usar o no la omisión de conexiones (*skip connections*). En la Tabla 5.3 se muestran los resultados del proceso de *tuning* asociados a las TCN.

Usuario	Naturaleza	Nr. filtros	Tam. kernel	Dropout	Omitir conexiones	Usa Batch Normalization	Nr. épocas	Tam. bache	MSE
34	per	16	16	0,8	Si	Si	256	8	0,258
32	per	4	4	0,8	No	Si	256	8	0,384
34	imp	64	4	0,8	No	Si	64	128	0,216
32	imp	4	16	0,506	No	Si	256	128	0,307

Tabla 5.3. Resultado del proceso de tuning para las TCN

Esta arquitectura presenta una diferencia importante respecto a las otras. Esto es, es necesario que el campo receptivo sea igual o mayor a la cantidad de *lags* que posea el *dataset* que se está utilizando, ya que de esta forma es seguro que el modelo esté utilizando efectivamente todos los *lags*. Por lo tanto, y como se decidió que el proceso de *tuning* sea llevado a cabo sólo para algunos casos -más particularmente, para 4 *lags*-, el hiperparámetro "número de dilaciones" no fue tuneado por el proceso de tuning. En cambio, este hiperparámetro es ajustado en cada experimento a partir del número de *lags* y el

tamaño de los kernels. Como reportan (Bai et al., 2018), las arquitecturas no cambian considerablemente su desempeño siempre y cuando su campo receptivo haya sido determinado correctamente. La fórmula que determina el número de dilaciones es:

$$\#dilaciones = \lfloor \log_2 \left(\frac{\#lags}{tam. kernel} + 1 \right) \rfloor$$

Al utilizar la fórmula anterior, se asegura que la cantidad de dilaciones sea determinada de forma que el campo receptivo siempre abarque la cantidad variable de *lags*. Cabe aclarar que aunque las TCN permiten la concatenación de más de un bloque residual, en esta tesis se utiliza uno solo. Esta decisión fue tomada teniendo en cuenta que no se justifica la utilización de más de un bloque residual en una experimentación donde la mayor cantidad de *lags* es 8.

5.1.4. RNN

Por último, en las arquitecturas RNN se ajustaron los siguientes hiperparámetros: número de capas recurrentes, cantidad de unidades LSTM por capa y el valor del *dropout* aplicado en cada capa. Al igual que en las arquitecturas CNN, se permitió que la última capa sea una capa densa opcional, por lo que también deben ser tuneados la cantidad de unidades y el valor del *dropout* de esta última capa. En la Tabla 5.4 se muestran los resultados del proceso de tuning asociados a las RNN.

Usuario	Naturaleza	Nr. capas	Nr. unidades LSTM	Dropout (lstm)	Nr nodos densos	Dropout (denso)	Nr. épocas	Tam. bache	MSE
34	per	2	4	0	64	0,447	16	8	0,245
32	per	4	128	0	64	0,561	8	8	0,342
34	imp	4	4	0	1	0,8	32	8	0,21
32	imp	2	4	0	32	0	8	8	0,3

Tabla 5.4. Resultado del proceso de tuning para las RNN

Como conclusión, se puede observar en las tablas que muestran los resultados del proceso de *tuning* que no existe una clara diferencia entre los hiperparámetros de los modelos personales y los de los impersonales. Lo mismo es cierto con respecto a las diferencias entre los usuarios modelo.

Por ejemplo, podría suponerse que los modelos impersonales se beneficiarían de la utilización de arquitecturas más profundas o amplias, es decir, con un mayor número de unidades y de capas. En la Tabla 5.1 vemos que la arquitectura personal para el usuario modelo 32 está compuesta por 16 capas 256 unidades cada una, mientras que la arquitectura impersonal para el mismo usuario está compuesta por 2 capas de 32 neuronas cada una. En este caso, la diferencia entre la cantidad de hiperparámetros es abismal y anti-intuitiva. Por lo tanto, para la tarea abordada en esta tesis no se observa que a mayor cantidad de datos, mayor la complejidad de la red. De todas maneras, al ser una tarea tan poco abordada no pueden obtenerse conclusiones al respecto. Para tareas más conocidas como las que intentan modelar el lenguaje, sí puede observarse un crecimiento en la complejidad de los modelos a medida que aumenta la cantidad

de datos disponibles para entrenarlo -por ejemplo: las diferentes versiones de GPT¹⁷-.

En los resultados que muestran las tablas, puede observarse una única distinción entre los hiperparámetros de los modelos personales y los impersonales. Los modelos impersonales se benefician de un tamaño de bache más grande. Esto se da para todos los tipos de redes neuronales salvo para las RNN. Este fenómeno es esperable debido a la gran cantidad de datos disponibles en comparación con los modelos personales.

5.2. Resultados de los experimentos

Como ya ha sido mostrado en la sección de experimentos, son muchas las categorías por las cuales puede ser clasificado cada experimento. Dichas categorías son: la arquitectura utilizada, el usuario objetivo, aquellas relacionadas a la conformación del *dataset* -número de *lags*, periodo y granularidad- y por último, la naturaleza del modelo -es decir, personal o impersonal-. Además, se suman otras categorías a partir de las cuales los experimentos pueden ser comparados: el MSE obtenido en cada una de las iteraciones del algoritmo de evaluación, el MSE promedio y el tiempo requerido para el entrenamiento para cada una de las interacciones. Los modelos fueron entrenados utilizando un servidor equipado con un CPU AMD Ryzen 7 2700 y un GPU Titan XP. A continuación, se discuten las formas existentes de mostrar y comparar los resultados.

Una primera forma de mostrar los resultados de los experimentos es realizar *rankings* en los que se muestran los experimentos con mayor desempeño para cada una de las categorías o conjunto de ellas. Esto es, tomar los posibles valores de una categoría o conjunto de ellas y compararlos entre sí. Por ejemplo, dadas las categorías arquitectura y granularidad, realizar un *ranking* para comparar cuales obtuvieron el mayor desempeño. En este último caso, los experimentos a comparar son: TCN/1h, MLP/1h, CNN/1h, RNN/1h, TCN/30min, MLP/30min, CNN/30min y RNN/30min. Cada una de estas combinaciones de categorías representa un conjunto de experimentos. Por ejemplo, en el caso anterior cada una de las combinaciones representa a 1152 experimentos. Se obtiene un solo experimento en el único caso en que se determine el valor de cada una de las categorías. Es debido entonces discutir cómo comparar estas categorías si representan a más de un experimento. Una opción es aplicar alguna función de agregación para obtener un valor que represente a dicho conjunto de experimentos. Diferentes alternativas son posibles y se listan a continuación:

- Tomar del conjunto de experimentos el que mayor MSE obtuvo.
- Tomar del conjunto de experimentos el promedio del MSE de todos ellos.

Sin embargo, esta forma de comparación supone un problema: los diferentes usuarios pueden llegar a obtener valores muy diferentes entre sí debido a la idiosincrasia de cada uno de ellos. Esto puede afectar en cuan predecibles sean sus patrones a partir de los modelos entrenados. Por esta razón, es razonable estudiar a cada usuario en particular. En este contexto, se propone como una segunda forma de mostrar los resultados, aplicar la primera estrategia para cada uno de los usuarios. Por ejemplo, una tabla para cada uno de ellos donde se muestre cuál arquitectura alcanzó el mejor desempeño o una comparación

¹⁷ <https://openai.com/blog/better-language-models/>

por cualquier otra categoría o conjunto de ellas. La desventaja de este enfoque es que supondría tener diversas tablas para cada uno de los 48 usuarios.

Por último, se propone una tercera forma de mostrar los resultados que busca obtener las ventajas de las dos primeras. Esta estrategia consiste en realizar un *ranking* para cada uno de los usuarios, pero resumirlos en una única tabla. La generación de esta tabla puede llevarse a cabo siguiendo los siguientes pasos:

1. Seleccionar la/s categoría/s que se quieren comparar;
2. Obtener todos los posibles valores resultantes de la combinación de estas categorías;
3. Para cada usuario, aplicar una función de agregación a los experimentos que agrupa cada posible valor obtenido en el punto 2. y realizar un *ranking* a partir de ellos.
4. A partir de todos los *rankings* obtenidos, generar un *ranking* global. Esto es, para cada posible valor, calcular en cuantos *rankings* quedó primero, en cuantos quedó segundo, y así sucesivamente.

A pesar de que este proceso se mostró solo para realizar cada *ranking* individual sobre la categoría "usuario", podría ser llevado a cabo para cualquier otra categoría, siempre y cuando no sea una categoría seleccionada para hacer la comparación. Otra variable interesante a introducir es que los experimentos incluidos en el proceso anterior pueden ser personalizados. Es decir, la entrada al proceso de generación del *ranking* global puede no ser el conjunto total de experimentos, sino un subconjunto de ellos. De esta forma, puede realizarse un análisis con mayor granularidad, es decir, puede seleccionarse un valor específico para algunas de las categorías. Lo que motiva a tomar un subconjunto del total de los experimentos es que hay aspectos tanto cualitativos como cuantitativos que pueden ser tomados en cuenta al momento de comparar los experimentos y que no son el MSE obtenido. Estos aspectos pueden ser (pero no se limitan a): el tamaño del *dataset*, la cantidad de registros disponibles para cada *bucket*, el hecho de poder predecir el comportamiento sedentario futuro con una mayor o menor granularidad, el tiempo que llevó entrenar al modelo, su número de parámetros (a partir del cual puede aproximarse el tamaño ocupado por él). Por la existencia de estos aspectos, se justifica la utilidad de estudiar diferentes particiones del conjunto de experimentos por separado.

A modo de resumen, la estrategia para mostrar los resultados se compone de los siguientes aspectos:

La categoría del modelo que será comparada (por ejemplo, la red neuronal).

La categoría de la cual serán tomados los valores posibles con los cuales se realizará cada *ranking* individual (por ejemplo, el usuario).

La categoría del modelo que será utilizada para realizar cada *ranking* individual (por ejemplo, el MSE obtenido).

Función de agregación que será aplicada a cada grupo del punto 2. Aplicada sobre la categoría seleccionada en el punto 3.

Valores específicos para ciertas categorías de los modelos que funcionaran como un filtro sobre el total del conjunto de experimentos (por ejemplo, una determinada cantidad de *lags*).

De esta forma el algoritmo encargado de calcular el *ranking* recibe 4 o más argumentos, por lo que cada uno de los resultados puede ser resumido en una tupla del tipo [1., 2., 3., 4., 5. (opcional)].

A continuación se presentan diferentes configuraciones de la estrategia anteriormente detallada para mostrar los *rankings*. Para cada configuración se muestran dos valores para el aspecto 3 -es decir, dos funciones de agregación-: la mínima y la media. Con respecto al aspecto número 2, se decidió darle prioridad al MSE en el análisis ya que es el que mayor importancia tiene en cuanto al desempeño de los modelos, aunque la cantidad de parámetros del modelo y el tiempo requerido para el entrenamiento también serán abordados. Finalmente, en la última sección se muestran los resultados específicos para los usuarios modelos.

5.2.1. Comparación entre las redes neuronales a partir del MSE

Para comenzar con los resultados, en las Tablas 5.1 y 5.2 se muestra el *ranking* de los experimentos por tipo de red neuronal utilizada comparados por el MSE utilizando los usuarios para los *rankings* individuales sobre el conjunto total de los experimentos, utilizando como función de agregación la mínima y la media respectivamente.

	Puesto 1	Puesto 2	Puesto 3	Puesto 4
RNN	6	5	20	17
CNN	11	9	9	19
TCN	21	21	3	3
MLP	10	13	16	9

*Tabla 5.1. Comparación entre las diferentes redes neuronales
[red neuronal, usuario, MSE, mínima, -]*

	Puesto 1	Puesto 2	Puesto 3	Puesto 4
RNN	25	10	10	3
CNN	11	16	9	12
TCN	6	12	14	16
MLP	6	10	15	17

*Tabla 5.2. Comparación entre las diferentes redes neuronales
[red neuronal, usuario, MSE, media, -]*

En la Tabla anterior, cada celda muestra la cantidad de veces en las cuales un determinado tipo de red neuronal terminó en un determinado puesto del *ranking*. La suma de cada fila o columna es 48, el número total de usuarios, ya que sobre ellos se llevó a cabo cada *ranking* individual.

Como puede observarse en la Tabla 5.1, los modelos TCN obtuvieron el menor MSE para 21 de los 48 usuarios, mientras que las CNN y las MLP se desempeñaron mejor en 11 y 10 casos respectivamente. Por último, las RNN obtuvieron el menor MSE que las demás redes en solo 6 casos. Es interesante notar que las TCN obtuvieron el primer o segundo menor MSE para 42 de los 48 usuarios. En la Tabla 5.2 puede observarse, en cambio, la media del MSE. Se observan resultados muy diferentes a los de la Tabla 5.1, ya que en este caso las redes RNN obtuvieron el menor MSE promedio para 25 de los 48 usuarios, mientras que las CNN desempeñaron mejor en 11 y 10 casos respectivamente. Por último, las MLP y las CNN alcanzaron un menor MSE promedio en 6 casos cada una.

La diferencia expuesta entre los resultados de ambas tablas está relacionada a que se está tomando el conjunto total de experimentos. Se ha detectado que en los modelos impersonales se dio un crecimiento exponencial del error conocido como *exploding gradient*. Este hecho afectó particularmente a las TCN en los modelos personales. Este hecho será comentado con mayor detalle en la [Sección 5.5](#).

En las siguientes Tablas se muestra el mismo ranking que el de la tabla anterior con la diferencia de que se filtran por las diferentes granularidades posibles. Como puede observarse, los resultados no cambian en gran medida entre la granularidad de 1 hora y las de 30 minutos si se usa la mínima como función de agregación, donde la ventaja de las TCN se sigue manteniendo (Tablas 5.3 y 5.5). En cambio, si se utiliza la media como función de agregación la ventaja de las RNN se sigue manteniendo, aunque con menor notoriedad cuando la granularidad es de 1 hora (Tablas 5.4 y 5.6).

	Puesto 1	Puesto 2	Puesto 3	Puesto 4
RNN	6	6	18	18
CNN	11	8	11	18
TCN	22	20	3	3
MLP	9	14	16	9

*Tabla 5.3. Comparación entre las diferentes redes neuronales
[red neuronal, usuario, MSE, mínima, gran=1h]*

	Puesto 1	Puesto 2	Puesto 3	Puesto 4
RNN	19	11	8	10
CNN	11	12	11	14
TCN	14	13	12	9
MLP	4	12	17	15

*Tabla 5.4. Comparación entre las diferentes redes neuronales
[red neuronal, usuario, MSE, media, gran=1h]*

	Puesto 1	Puesto 2	Puesto 3	Puesto 4
RNN	5	5	19	19
CNN	10	12	11	15
TCN	22	15	6	5
MLP	11	16	12	9

*Tabla 5.5. Comparación entre las diferentes redes neuronales
[red neuronal, usuario, MSE, mínima, gran=30min.]*

	Puesto 1	Puesto 2	Puesto 3	Puesto 4
RNN	27	9	5	7
CNN	9	20	14	5
TCN	3	7	13	25
MLP	9	12	16	11

*Tabla 5.6. Comparación entre las diferentes redes neuronales
[red neuronal, usuario, MSE, media, gran=30min.]*

A continuación, se realizan otros rankings, pero esta vez filtrando a partir de los diferentes tipos de modelos según su naturaleza, esto es, modelos personales e impersonales por separado. En las primeras dos tablas (Tablas 5.7 y 5.8) se muestra el ranking de los modelos personales usando la mínima y la media respectivamente, mientras que en las últimas dos tablas (Tablas 5.9 y 5.10) se muestra el ranking de los modelos impersonales usando la mínima y la media respectivamente.

En estas tablas se observa una diferencia clara entre los modelos según su naturaleza. En los modelos personales son las RNN las que obtuvieron el mejor desempeño. Esto se da en 21 de los casos si se utiliza la mínima y 28 de los casos si se utiliza la media. En cambio, para los modelos impersonales son las TCN las que toman la delantera. Los motivos por los cuales se dio esta diferencia se abordan en la discusión ([Sección 5.3.3](#)).

	Puesto 1	Puesto 2	Puesto 3	Puesto 4
RNN	21	15	7	5
CNN	5	15	15	13
TCN	17	7	10	14
MLP	5	11	16	16

*Tabla 5.7. Comparación entre las diferentes redes neuronales
[red neuronal, usuario, MSE, mínima, naturaleza=personal]*

	Puesto 1	Puesto 2	Puesto 3	Puesto 4
RNN	28	13	5	2
CNN	9	21	10	8
TCN	7	6	10	25
MLP	4	8	23	13

*Tabla 5.8. Comparación entre las diferentes redes neuronales
[red neuronal, usuario, MSE, media, naturaleza=personal]*

	Puesto 1	Puesto 2	Puesto 3	Puesto 4
RNN	3	6	18	21
CNN	11	8	12	17
TCN	23	20	2	3
MLP	11	14	16	7

*Tabla 5.9. Comparación entre las diferentes redes neuronales
[red neuronal, usuario, MSE, mínima, naturaleza=impersonal]*

	Puesto 1	Puesto 2	Puesto 3	Puesto 4
RNN	5	5	15	23
CNN	13	14	7	14
TCN	24	11	7	6
MLP	6	18	19	5

*Tabla 5.10. Comparación entre las diferentes redes neuronales
[red neuronal, usuario, MSE, media, naturaleza=impersonal]*

5.2.2. Modelos Personales vs. Modelos Impersonales

En la Tablas 5.11 y 5.12 se muestra el *ranking* de los experimentos según la naturaleza del modelo comparados por el MSE utilizando los usuarios para los *rankings* individuales sobre el conjunto total de los experimentos, utilizando como función de agregación la mínima y la media respectivamente. A modo de ejemplo, la primera celda - columna: "Puesto 1", fila: "Personales"- de la Tabla 5.11 lleva el valor 5 ya que para 5 de los 48 usuarios sobre los cuales se llevaron a cabo los experimentos fue un modelo personal

el que alcanzó el mejor desempeño. Se observa, en ambas tablas, una clara predominancia de los modelos impersonales sobre los personales.

	Puesto 1	Puesto 2
Personales	5	43
Impersonales	43	5

Tabla 5.11. Comparación por la naturaleza del modelo
[naturaleza, usuario, MSE, mínima, -]

	Puesto 1	Puesto 2
Personales	0	48
Impersonales	48	0

Tabla 5.12. Comparación por la naturaleza del modelo
[naturaleza, usuario, MSE, media, -]

En las Tabla 5.13 y 5.14 se muestra un *ranking* similar, salvo que se muestra cada una de las iteraciones por separado. Se observa como los modelos personales van aumentando su precisión en comparación con los modelos impersonales cuantos más datos tienen para ser entrenados. Este fenómeno se ve con mayor notoriedad cuando la función de agregación es la mínima. Es debido aclarar que se toman las iteraciones individualmente y no acumulativamente. Por ejemplo, la Iteración 3 comprende únicamente el desempeño de los modelos en dicha iteración. Esto implica que los modelos que hayan alcanzado un alto desempeño en una iteración pueden no haberlo hecho en las demás.

	Iteración 1		Iteración 2		Iteración 3		Iteración 4		Iteración 5	
	Puesto 1	Puesto 2	Puesto 1	Puesto 2	Puesto 1	Puesto 2	Puesto 1	Puesto 2	Puesto 1	Puesto 2
Personales	8	40	14	34	18	30	19	29	23	25
Impersonales	40	8	34	14	30	18	29	19	25	23

Tabla 5.13. Comparación por la naturaleza del modelo para cada iteración.
[naturaleza, usuario, MSE, mínima, iteración=[1,2,3,4,5]]

	Iteración 1		Iteración 2		Iteración 3		Iteración 4		Iteración 5	
	Puesto 1	Puesto 2	Puesto 1	Puesto 2	Puesto 1	Puesto 2	Puesto 1	Puesto 2	Puesto 1	Puesto 2
Personales	1	47	5	43	4	44	10	38	16	32
Impersonales	47	1	43	5	44	4	38	10	32	16

Tabla 5.14. Comparación por la naturaleza del modelo para cada iteración.
[naturaleza, usuario, MSE, media, iteración=[1,2,3,4,5]]

5.2.3. Comparación entre las redes neuronales a partir del tiempo requerido para ser entrenadas

En las siguientes tablas se muestra el ranking de los experimentos según la red neuronal utilizada comparados por el tiempo requerido utilizando los usuarios para los rankings individuales, utilizando como función de agregación la mínima y la media respectivamente. En las Tablas 5.15 y 5.16 se realiza el ranking sobre el conjunto total de experimentos. En las Tablas 5.17 y 5.18 se filtra por los modelos impersonales, mientras que en las Tablas 5.19 y 5.20 se filtra por los modelos personales. A modo de ejemplo, la primera celda - columna: "Puesto 1", fila: "RNN"- de la Tabla A lleva el valor 48 ya que para todos los usuarios sobre los cuales se llevaron a cabo los experimentos fue un modelo personal el que fue entrenado en el menor tiempo. La razón por la cual se filtra por cada valor posible de la naturaleza del modelo es que, como es de esperar, es la categoría para la cual mayor diferencia existe en cuanto a tiempo requerido para entrenar los modelos.

	Puesto 1	Puesto 2	Puesto 3	Puesto 4
RNN	48	0	0	0
CNN	0	48	0	0
TCN	0	0	0	48
MLP	0	0	48	0

*Tabla 5.15. Comparación entre las diferentes redes neuronales
[red neuronal, usuario, tiempo, mínima]*

	Puesto 1	Puesto 2	Puesto 3	Puesto 4
RNN	0	0	36	12
CNN	48	0	0	0
TCN	0	0	12	36
MLP	0	48	0	0

*Tabla 5.16. Comparación entre las diferentes redes neuronales
[red neuronal, usuario, tiempo, media]*

	Puesto 1	Puesto 2	Puesto 3	Puesto 4
RNN	0	0	30	18
CNN	48	0	0	0
TCN	0	0	18	30
MLP	0	48	0	0

*Tabla 5.17. Comparación entre las diferentes redes neuronales
[red neuronal, usuario, tiempo, mínima, naturaleza=impersonal]*

	Puesto 1	Puesto 2	Puesto 3	Puesto 4
RNN	0	0	36	12
CNN	48	0	0	0
TCN	0	0	12	36
MLP	0	48	0	0

*Tabla 5.18. Comparación entre las diferentes redes neuronales
[red neuronal, usuario, tiempo, media, naturaleza=impersonal]*

	Puesto 1	Puesto 2	Puesto 3	Puesto 4
RNN	48	0	0	0
CNN	0	12	36	0
TCN	0	0	0	48
MLP	0	36	12	0

*Tabla 5.19. Comparación entre las diferentes redes neuronales
[red neuronal, usuario, tiempo, mínima, naturaleza=personal]*

	Puesto 1	Puesto 2	Puesto 3	Puesto 4
RNN	48	0	0	0
CNN	0	17	31	0
TCN	0	0	0	48
MLP	0	31	17	0

*Tabla 5.20. Comparación entre las diferentes redes neuronales
[red neuronal, usuario, tiempo, media, naturaleza=personal]*

Como puede observarse, en las Tablas 5.15 y 5.16 se da un fenómeno interesante. A pesar de que, para todos los usuarios, fue un modelo RNN el que fue entrenado en el menor tiempo, cuando se utiliza la media como función de agregación, las CNN son las que fueron entrenadas en el menor tiempo en promedio. Esto se da porque, para pocos datos de entrenamiento, las RNN pudieron ser entrenadas rápidamente $-0,13 \pm 0,035 \text{ minutos-}$ en comparación a las CNN $-0,285 \pm 0,08 \text{ minutos -}$ (Tablas 5.19 y 5.20), pero cuando la cantidad de datos de entrenamiento crece las RNN aumentan mucho el tiempo requerido para ser entrenadas $-4,895 \pm 5,24 \text{ minutos-}$, mientras que las CNN incluso requirieron un menor tiempo $-0,21 \pm 0,08 \text{ minutos -}$ gracias a su alta capacidad de paralelización (5.17 y 5.18). En todas las tablas se puede notar que las TCN se encuentra en el último puesto en cuanto al tiempo requerida para ser entrenada -global: $2,3 \pm 1,04 \text{ minutos}$, personal: $1,83 \pm 0,69 \text{ minutos}$, impersonal: $2,75 \pm 1,13 \text{ minutos-}$.

5.2.4. Resultados detallados de los usuarios modelo

En esta sección se muestran más detalladamente los resultados obtenidos para los usuarios modelos. El usuario 34 representa al grupo de usuarios con menor gasto energético y menor cantidad de *buckets* disponibles, mientras que el usuario 32 representa al grupo de usuarios con mayor gasto energético y mayor cantidad de *buckets* disponibles.

A continuación, se muestran las configuraciones de los experimentos que obtuvieron el menor MSE para cada uno de los usuarios modelo, así como también el MSE y el tiempo requerido para el entrenamiento para cada una de las iteraciones. Además, se muestran graficadas las predicciones realizadas.

En el caso del usuario 34, el modelo que menor MSE obtuvo fue de naturaleza impersonal, con una granularidad de 1 hora, usando 8 *lags*, y un periodo de 4. La arquitectura correspondió a una TCN. El promedio del MSE obtenido fue de 0,1328 mientras que el tiempo de entrenamiento promedio fue de aproximadamente 1,9 minutos. En la Tabla 5.21 se muestran los valores de MSE y tiempo de entrenamiento de todas las iteraciones.

	It. 1	It. 2	It. 3	It. 4	It. 5
MSE	0,195	0,044	0,14	0,107	0,178
Tiempo	0,713	1,393	2,029	2,614	2,844

Tabla 5.21. MSE y tiempo (en minutos) promedio de usuario 34

En la Figura 5.1 pueden observarse las predicciones realizadas por el modelo que menor desempeño obtuvo para el usuario 34. Las diferentes iteraciones están separadas por líneas horizontales intercaladas. Puede observarse la poca cantidad de *buckets* disponibles y la poca cantidad de registros con comportamiento no sedentario disponible. Debido a esto, el modelo es incapaz de identificar correctamente dónde el usuario tuvo un alto nivel de MET. En estos casos donde la gran mayoría de los *buckets* corresponden a un comportamiento sedentario se ve la importancia de la utilización de MET como función de error, ya que los *buckets* de comportamiento no sedentario deben ser fuertemente penalizados.

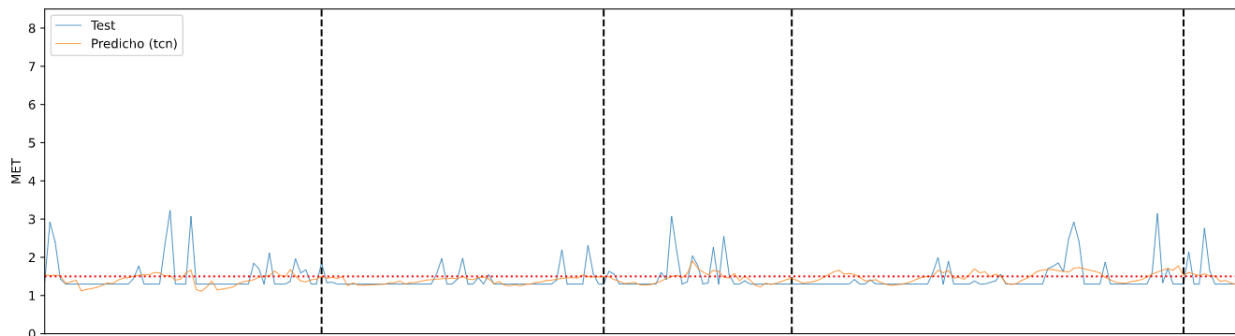


Figura X. Predicciones sobre el usuario 34

En el caso del usuario 32, el modelo que menor MSE obtuvo fue de naturaleza impersonal, con una granularidad de 1 hora, usando 8 *lags*, y un periodo de 1. La arquitectura correspondió a una TCN. El promedio del MSE obtenido fue de 0,3 mientras que el tiempo de entrenamiento promedio fue de aproximadamente 2,2 minutos. En la Tabla 5.22 se muestran los valores de MSE y tiempo de entrenamiento de todas las iteraciones.

	It. 1	It. 2	It. 3	It. 4	It. 5
MSE	0,11	0,221	0,186	0,569	0,414
Tiempo	0,938	0,973	2,934	2,391	3,892

Tabla 5.22. MSE y tiempo (en minutos) promedio de usuario 34

En la Figura 5.2 pueden observarse las predicciones realizadas por el modelo que menor desempeño obtuvo para el usuario 32. En comparación con la Figura X, en este caso pueden observarse muchos más *buckets* disponibles de comportamiento no sedentario, por lo que el modelo puede predecir mucho mejor cuando habrá un pico de alto gasto energético. A pesar de esto, el MSE obtenido es mayor al obtenido para el usuario 34, ya que para este último el modelo aprendió a predecir siempre un bajo nivel de MET y no ser penalizado haciéndolo.

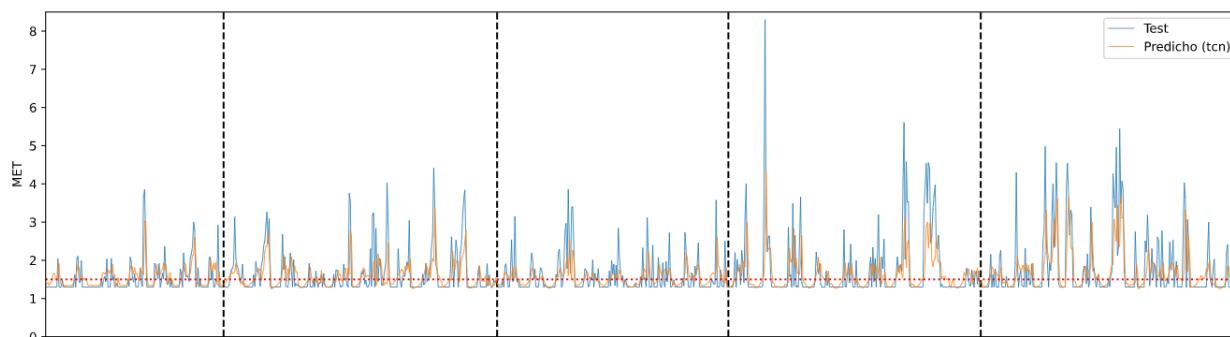


Figura 5.2. Predicciones sobre el usuario 32

Resumen

En este capítulo se presentaron los resultados obtenidos. Por un lado, en la [Sección 5.1](#) se mostraron los resultados del proceso de *tuning* que se lleva a cabo utilizando la optimización de Bayes. Para cada uno de los tipos de redes neuronales y experimentos modelos, se mostraron los valores resultantes de los hiperparámetros hallados. Por otro lado, en la [Sección 5.2](#) se mostraron los resultados de los experimentos llevados a cabo a partir del diseño pautado en la [Sección 4.3.6](#). Se presentó una discusión acerca de la forma de presentar los resultados de forma clara a la vez que permita mostrar resultados de una gran cantidad de experimentos. Para ello se diseñó una estrategia para generar un ranking global que resume una cantidad determinada de *rankings* individuales por usuario. A partir de dicha estrategia, se compararon diferentes aspectos de los resultados, como las redes neuronales, los modelos personales e

impersonales y el tiempo requerido para entrenar cada uno de ellos. Finalmente, se mostraron los resultados detallados de los usuarios modelos.

Capítulo 6: Discusión

En este capítulo se presenta la discusión llevada a cabo a partir de los resultados obtenidos. En la [Sección 6.1](#) se busca comprender el impacto de los usuarios modelo seleccionados. En la [Sección 6.2](#) se estudia la diferencia obtenida entre los modelos personales y los impersonales. En la [Sección 6.3](#) se compara el desempeño de las diferentes redes neuronales. En la [Sección 6.4](#) se compara el tiempo de entrenamiento para los diferentes modelos. En la [Sección 6.5](#) se analiza el impacto de las diferentes granularidades. En la [Sección 6.6](#) se estudian las implicancias de la utilización de diferentes números de *lags*. Por último, en la [Sección 6.7](#) se estudia el problema de *exploding gradient*, el cual sufrieron una gran cantidad de los modelos entrenados a partir de los diferentes experimentos.

6.1. Impacto de los usuarios modelos

Como se explicó en la [Sección 4.3.6](#), diferentes aspectos de los experimentos fueron tomados como modelos para llevar a cabo el proceso de *tuning*. Uno de esos aspectos fue el de los usuarios. Se tomaron dos usuarios modelos para los cuales llevar a cabo el proceso de *tuning*. Para ello, se utilizó el algoritmo *K-Means* con la finalidad de dividir al conjunto de usuarios en dos grupos teniendo en consideración 3 variables: el MET promedio, la desviación estándar del nivel de MET y la cantidad de *buckets* disponibles. Los usuarios que resultaron más cercanos al centroide de sus respectivos grupos fueron seleccionados como usuarios modelo. Para entrenar un modelo sobre un usuario no modelo, se tomó la arquitectura correspondiente al usuario modelo de su grupo. Esto puede haber impactado en el desempeño de los usuarios no modelo, ya que dichas arquitecturas no fueron ajustadas en base a ellos.

A continuación, se busca dilucidar el impacto de la estrategia utilizada y tomar en consideración, si fuese el caso, las ventajas que hubieran supuesto realizar el proceso de *tuning* para cada uno de los modelos. En la Figura 6.1 se muestra, para cada usuario, el MSE alcanzado. Se muestran los resultados de utilizar tanto la media como la mínima como función de agregación. Por ejemplo, en el primer gráfico, el radio del círculo del usuario 32 representa el promedio de MSE para los modelos realizados a partir de dicho usuario. En cambio, en el segundo gráfico, el radio del círculo del usuario 32 representa la mínima del MSE. Esta figura es similar a la Figura 6.2, que grafica la selección de los usuarios modelo.

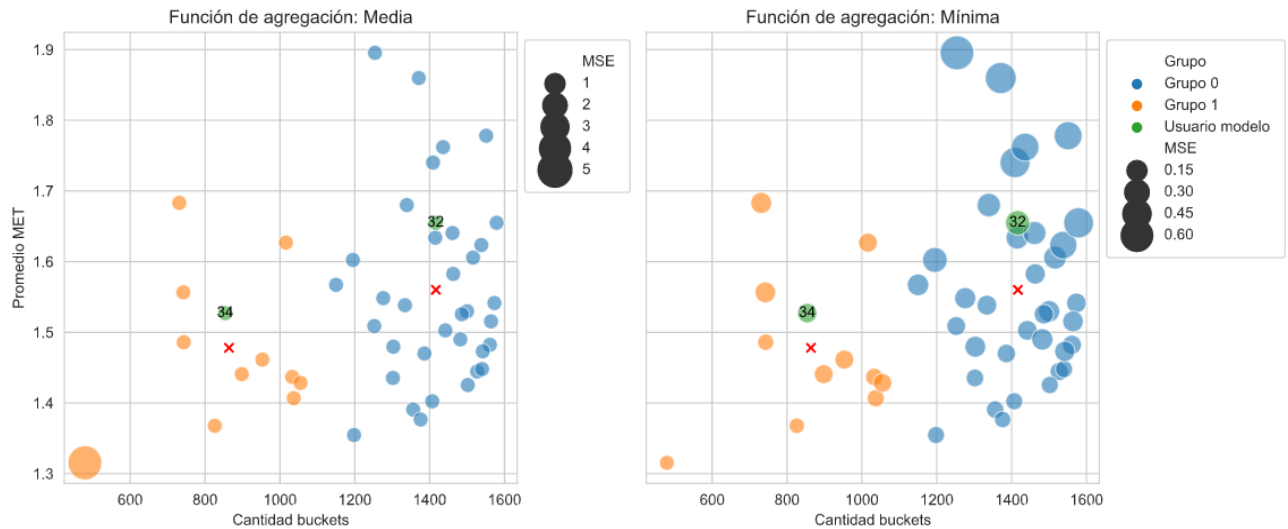


Figura 6.1. MSE obtenido para cada usuario

En el gráfico generado a partir de la media como función de agregación no se observa una gran diferencia entre los diferentes usuarios. Esto se debe principalmente a que hubo un usuario en particular cuyo promedio de MSE fue muy alto. En el segundo gráfico de la Figura 6.1 existe un patrón claro que diferencia a ambos grupos. Por un lado, en el grupo del usuario 34 (bajo promedio y desviación estándar de MET y poca cantidad de *buckets* disponibles) no se observa una gran diferencia entre los radios de los círculos que representan a los usuarios de este grupo. Por otro lado, en el grupo del usuario 32 (alto promedio y desviación estándar de MET y mayor cantidad de *buckets* disponibles) se da que: cuanto

menor el promedio de MET, menor el MSE obtenido. Esto puede deberse a la mayor facilidad del modelo para predecir el comportamiento sedentario ya que no hay muchos casos de comportamientos no sedentarios. Para los usuarios con mayor MET promedio, se observa claramente las dificultades que tuvieron los modelos (en comparación con los usuarios con bajo promedio de MET) para realizar las predicciones.

En síntesis, no se deduce de los resultados de los experimentos un patrón que demuestre que la elección de realizar el proceso de *tuning* a partir de ciertos usuarios modelos haya tenido consecuencias negativas para los usuarios que se encuentran más lejos de los centroides de cada grupo.

6.2. Modelos personales e impersonales

En esta sección se busca explorar las diferencias entre los modelos personales e impersonales, en términos de su desempeño general y a lo largo de las diferentes iteraciones. La diferencia principal entre ambos modelos es que en el caso de los impersonales se posee información sobre todos los usuarios menos sobre el que se quiere realizar predicciones. En el caso de los modelos personales, sucede lo contrario, se posee únicamente información del usuario sobre el que se quiere realizar predicciones. En la práctica, esto conlleva una diferencia muy importante. Los modelos impersonales poseen muchos más casos de estudio que los personales. De cuantos más casos de estudios dispondrán los modelos impersonales dependerá de la cantidad de usuario de los cuales se tenga información. En el caso que se aborda en esta tesis, la proporción es de aproximadamente 47 a 1. Aunque, al mismo tiempo, los modelos personales son los únicos que poseen datos que representan la idiosincrasia del usuario en cuestión. Nuevamente, cuánto provecho puede sacar este tipo de modelos de este hecho, dependerá de cuán similar sea el comportamiento sedentario entre los diferentes usuarios, ya que, si la mayoría de los usuarios tienen comportamientos similares, el modelo personal no debería poder obtener un mejor desempeño que el modelo impersonal. Entonces, es preciso analizar cómo impactó en los experimentos estas dos últimas consecuencias prácticas.

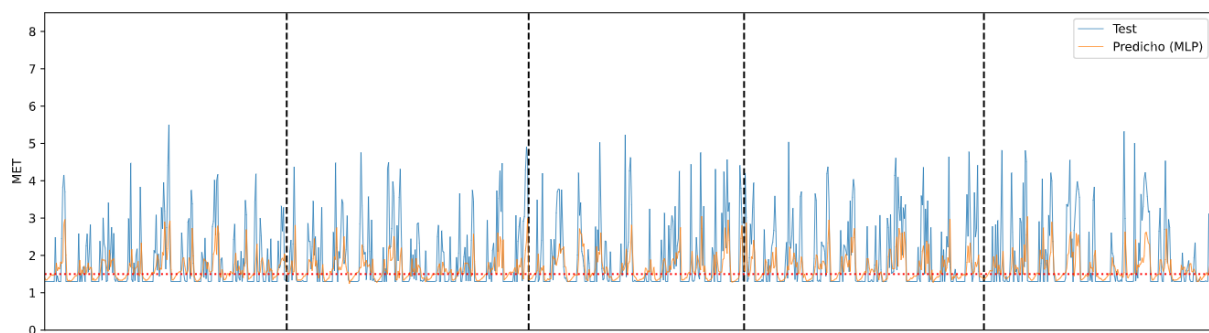


Figura 6.2. Predicciones sobre el usuario 46

Como se muestra en la Figura 6.1, el MET promedio de todo el conjunto de usuario varía ampliamente, desde poco más de 1,3 hasta 1,9. En la Figura 6.2 se muestran las predicciones sobre el usuario 46. El modelo que mayor desempeño alcanzó para el usuario 46 es un modelo impersonal, que

utiliza una MLP, con una granularidad de 60, 1 *lag* y un periodo de 1. En la figura, vemos que el usuario tiene muchos picos de gasto de energía. El modelo es capaz de predecir dichos picos pero no precisamente su magnitud. En la Figura 6.3 se muestra en detalle la primera iteración, donde se ve más claramente que los momentos sedentarios y los no sedentarios son predichos correctamente. Como el modelo que se muestra en las figuras es impersonal, el modelo fue entrenado a partir de los datos de todos los demás usuarios, no habiendo ninguno cuyo gasto energético sea tan alto (ya que el usuario 46 es el que mayor MET promedio tiene).

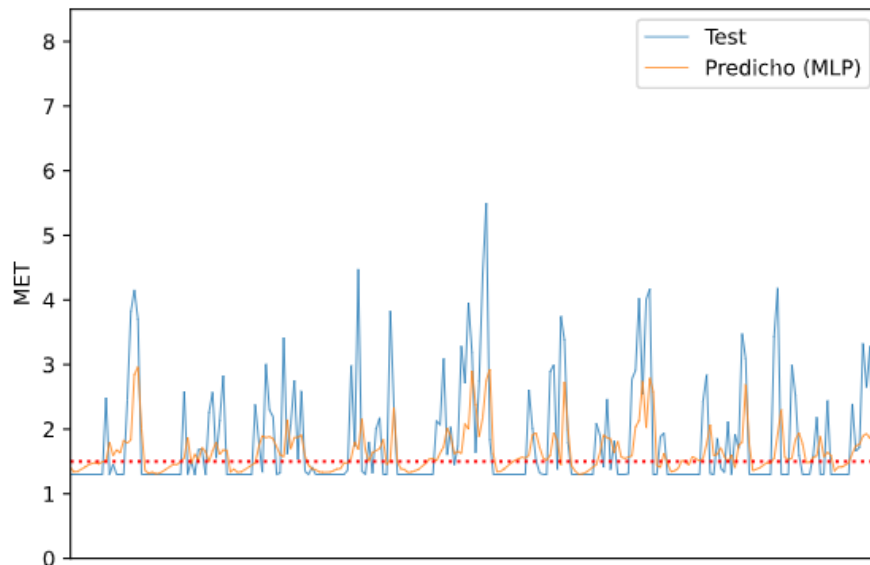


Figura 6.3. Predicciones sobre el usuario 46 (primera iteración)

Es lógico analizar, entonces, si los modelos personales para el usuario 46 son capaces de predecir con mayor precisión los picos de gasto energético. El modelo personal que mayor desempeño obtuvo se encuentra en el puesto 11. Este modelo utiliza una RNN, con una granularidad de 1 hora, 8 *lags* y un periodo de 1. En las Figuras 6.4 y 6.5 se muestran las predicciones para este modelo análogamente a como se hizo anteriormente con el modelo impersonal. En la Figura 6.4 puede observarse como para las primeras iteraciones las predicciones del modelo son altamente imprecisas, debido a la poca cantidad de casos de entrenamiento a partir de los cuales ha podido ser entrenada. Sin embargo, a partir de la iteración 3 el modelo comienza a realizar mejores predicciones. Se observa, además, en la segunda imagen de la Figura 6.5, que el modelo es capaz de predecir con mayor precisión los picos de gasto energético que en el caso del modelo impersonal, aunque el modelo erró en ciertos casos donde predice un MET alto cuando el usuario está llevando a cabo un comportamiento sedentario.

El MSE para las predicciones de las iteraciones del modelo impersonal fueron: 0,514, 0,727, 0,776, 0,774 y 0,799. Mientras tanto, los mismos valores para el modelo personal fueron: 0,733, 0,892, 0,874, 0,714, 0,606. Esto confirma lo que se mostró gráficamente en las figuras anteriores. El MSE para las primeras iteraciones es peor en el caso de los usuarios personales para las primeras iteraciones. Luego, en las últimas dos iteraciones, el modelo personal obtiene un mejor desempeño que el impersonal.

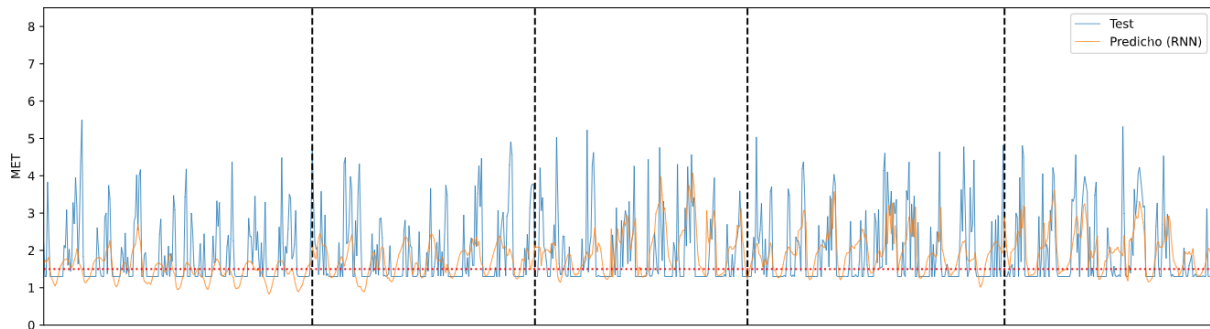


Figura 6.4. Predicciones para el usuario 46

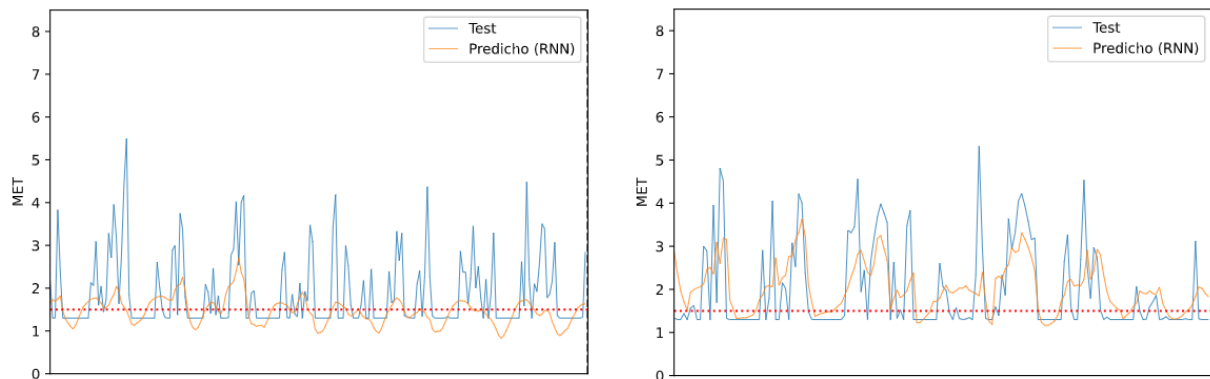


Figura 6.5. Predicciones sobre la primera y la última iteración del modelo personal para el usuario 46

El hecho de que el desempeño de los modelos personales mejore a medida que aumenta la cantidad de datos que poseen sobre el usuario -en este caso, a medida que aumentan las iteraciones- se da a nivel general, pudiendo deberse a que en iteraciones más tardías los modelos personales ya poseen datos suficientes a partir de los cuales pueden ser modeladas las idiosincrasias de cada usuario. En las Tablas 5.13 y 5.14, se muestra que a medida que avanzan las iteraciones, el desempeño de los modelos personales aumenta en gran medida en comparación con los modelos impersonales (en base a cuántos usuarios ha conseguido el mejor desempeño). Esto es un hecho importante, ya que a nivel general los modelos impersonales consiguieron un mayor desempeño que los personales para 43 de los 48 usuarios, reduciendo a 25 de los 48 usuarios en la última iteración. Quizás, el hecho de que a nivel general los

modelos impersonales se hayan impuesto por sobre los personales se debe al bajo desempeño que estos alcanzaron en las primeras iteraciones, debido a la baja cantidad de datos de entrenamiento. Es importante aclarar en esta última observación, que el análisis para comparar a los modelos personales y los impersonales se lleva a cabo a partir de las diferentes iteraciones tomadas individualmente. A modo de ejemplo, a pesar de que para la última iteración -para la cual los modelos poseen la mayor cantidad de datos disponibles para ser entrenados- los modelos personales alcanzaron el mejor desempeño casi para la misma cantidad de usuarios que los modelos impersonales, esto no quiere decir que esto se de a nivel global. Se da en muchos casos que un modelo personal supera a los modelos impersonales en una única iteración, y no es varias iteraciones consecutivas. Además, si se observa el *ranking* de los modelos, dentro de las primeras decenas la gran mayoría serán modelos impersonales, más allá de que se dé el caso en que el que obtuvo el primer puesto sea un modelo personal.

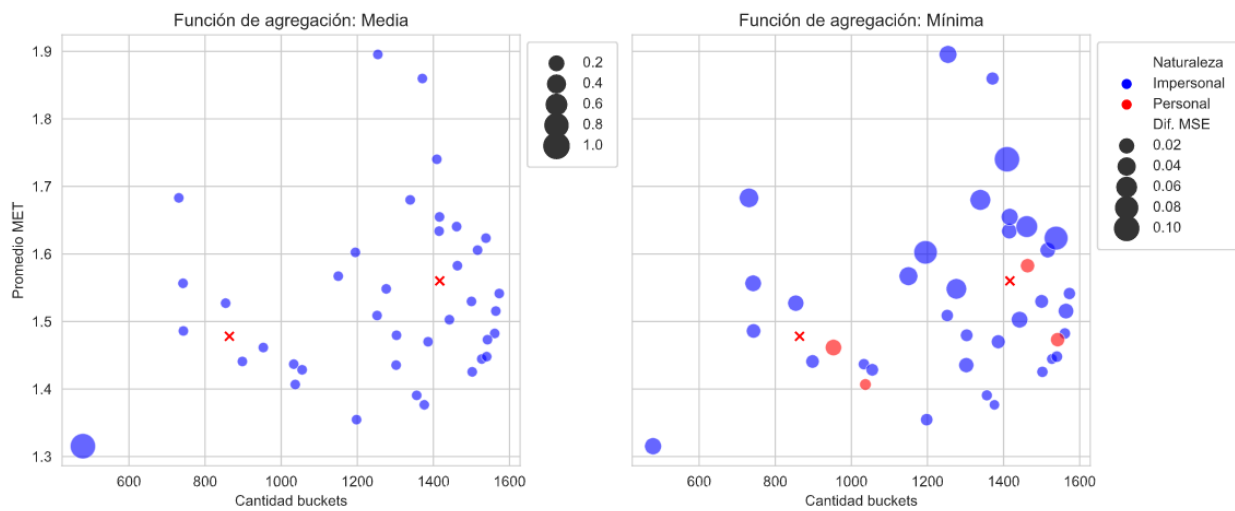


Figura 6.6. Diferencia de MSE entre los modelos personales e impersonales

En la Figura 6.6 se muestra, para cada usuario, el modelo que mayor desempeño alcanzó basado en la naturaleza del modelo. El radio de los puntos representa la diferencia entre el MSE del mejor modelo personal y el mejor modelo impersonal -en base a la media en el primer gráfico y en base a la mínima en el segundo-. En el primer gráfico, puede observarse que, para todos los usuarios, el mejor modelo fue una impersonal, es decir, aquel que obtuvo el menor promedio de MSE. Además, como se observa en el segundo gráfico, para todos los usuarios, salvo 4, los modelos impersonales obtuvieron un mayor desempeño que los modelos personales. Es interesante analizar la diferencia del MSE entre el mejor modelo personal y el mejor modelo impersonal en cada gráfico. Debido al problema del *exploding gradient* que sufrieron muchos modelos personales ([Sección 6.7](#)) las diferencias entre ambos cuando se analiza la primera imagen es inmensa (por más que no se puede apreciar en la imagen debido a la gran magnitud de la diferencia). En promedio, la diferencia de MSE usando la media como función de agregación fue de 20.854.079. El único tipo de red neuronal cuyos modelos personales no se vieron afectados por el *exploding gradient* fue la RNN. El promedio anterior descende a 0,04 si se filtran los modelos en los que se utilizó una RNN.

Si se analiza la segunda imagen, dicha diferencia entre ambos tipos de modelos ronda desde 0,0012 hasta 0,112, y se manifiesta similar en la mayoría de los casos. Sin embargo, esta diferencia

aumenta para los usuarios con mayor MET promedio y mayor cantidad de *buckets*. Más específicamente dicho fenómeno se da para los usuarios pertenecientes al lado superior del *cluster* asociado al usuario 32. Esto puede deberse a que los modelos impersonales para estos casos tengan suficientes casos de entrenamiento con usuarios con alto gasto energético de los cuales aprender, mientras que los modelos personales son los que más fallan en estos casos, especialmente en las primeras iteraciones.

6.3. Desempeño de las diferentes redes neuronales

En esta sección se analiza el desempeño de los diferentes tipos de redes neuronales, sobre todo en base a cómo se desempeñaron en los grupos que se crearon a partir de los usuarios modelo. Para comenzar, se presenta la Figura 6.7, que muestra, para cada usuario, la red neuronal que mejor desempeño alcanzó en base a la mínima y la media del MSE de los conjuntos de experimentos de cada usuario. De esta manera, en el primer gráfico, se muestra la red neuronal con el menor MSE promedio, mientras que en el segundo gráfico se muestra la red neuronal con el menor MSE alcanzado.

En el primer gráfico de la Figura 6.7 las RNN obtuvieron el menor MSE promedio para la gran mayoría de los usuarios. A pesar de esto, dicha mayoría no se mantiene en el grupo del usuario 34 (bajo promedio y desviación estándar de MET y poca cantidad de buckets disponibles). La ventaja de las RNN puede explicarse a partir de lo que se discute en la [Sección 6.7](#). Las RNN resultaron ser las más estables para los modelos personales.

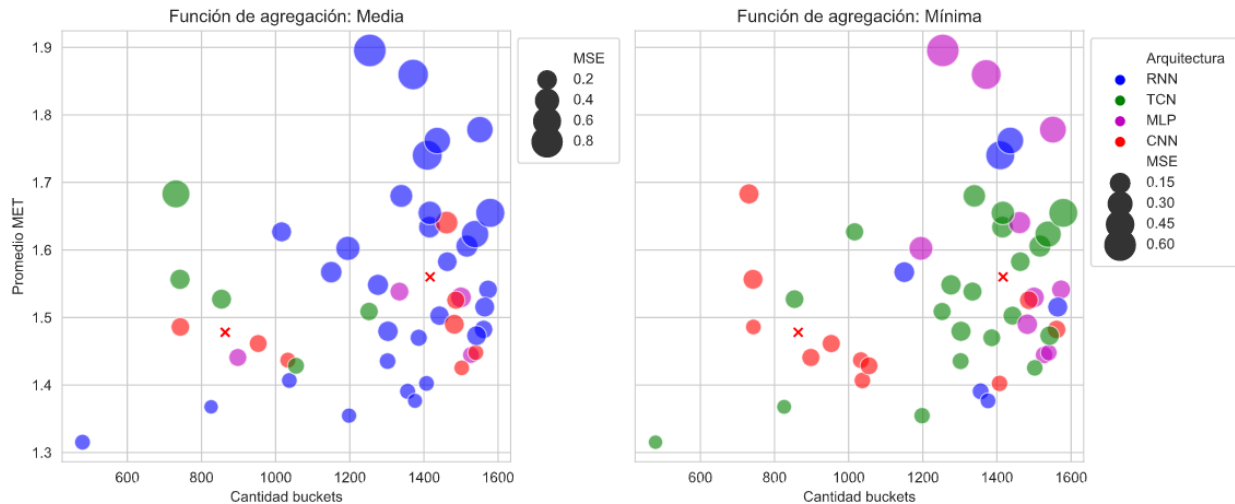


Figura 6.7. Mejores redes neuronales para cada usuario

Se llevó a cabo el Test de *Student* para comparar las diferentes redes neuronales en base a la diferencia estadística entre ellas. Como se desprende de lo analizado a partir de la Figura 6.6, para el grupo del usuario 32 existe una diferencia estadística menor a 0,05 entre las RNN y las demás redes neuronales. El valor de p para el Test de *Student* es de 0,004 cuando se comparan las RNN con las CNN, de 0,032 cuando se las compara con las MLP y de 0,0002 cuando se las compara con las TCN. No se halló una diferencia estadística significativa ($p < 0,05$) para el grupo del usuario 34.

Como puede observarse en el segundo gráfico de la Figura 6.7 existe un patrón que diferencia a ambos grupos. Por un lado, en el grupo del usuario 34 (bajo promedio y desviación estándar de MET y poca cantidad de *buckets* disponibles) los modelos que obtuvieron el mejor desempeño para 8 de los 12 usuarios fue a partir de CNNs. Por otro lado, en el grupo del usuario 32 (alto promedio y desviación estándar de MET y mayor cantidad de *buckets* disponibles) predomina en la gran mayoría de los usuarios las TCN como mejor red neuronal. La diferencia entre ambos gráficos se justifica por el hecho de que para generar las imágenes anteriores se utiliza el conjunto total de usuarios, donde se incluyen resultados para las cuales las TCN no tuvieron el problema del *exploding gradient*.

En la Figura 6.7 se utiliza el conjunto total de experimentos para generar los gráficos. A pesar de que, como ya ha sido mencionado, las RNN obtuvieron el mejor desempeño en la gran mayoría de los usuarios debido a que fue la red neuronal más estable, es interesante analizar qué redes neuronales tuvieron un mejor desempeño en cada caso para los modelos personales e impersonales por separado. Por lo tanto, a continuación se analiza cada caso en particular.

En la Figura 6.8 se muestran las mejores redes neuronales para cada usuario tomando en cuenta solo los modelos personales, utilizando la media y la mínima como funciones de agregación, tal como se hizo en las figuras anteriores. Para los usuarios pertenecientes al grupo de mayor gasto energético y mayor cantidad de *buckets* disponibles, las RNN son aquellas redes que tuvieron el mejor desempeño en la gran mayoría de los casos, aún con mayor predominancia que en el primer gráfico de la Figura 6.6. Esto indica su buena capacidad de predicción para *datasets* más heterogéneos con respecto a la mayor presencia de comportamiento de *buckets* correspondientes a un comportamiento no sedentario, al mismo tiempo que una poca cantidad de datos. Además, como ya se mencionó, no sufrieron el problema de *exploding gradient*. En cambio, para el grupo del usuario 34, con respecto al primer gráfico - media como función de agregación- en la mitad de los casos las TCN obtuvieron el mejor MSE promedio y en la otra mitad las RNN, mientras que en el segundo gráfico- mínima como función de agregación- las TCN obtuvieron el menor MSE para la gran mayoría de los usuarios.

Con respecto a la diferencia estadística entre las diferentes redes neuronales para los experimentos personales, se da en los mismos casos que en los detallados cuando se toma el conjunto total de experimentos.

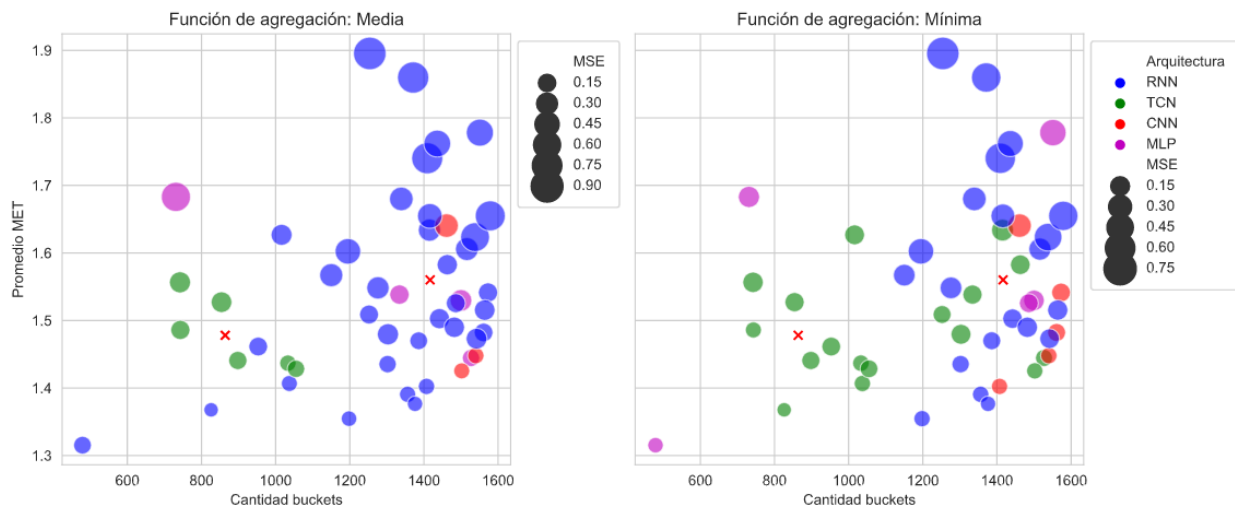


Figura 6.8. Mejores redes neuronales para los modelos personales

En la Figura 6.9 se muestran las mejores redes neuronales para cada usuario tomando en cuenta solo los modelos impersonales. En el primer gráfico se utilizó la media como función de agregación, mientras que se utilizó la mínima en el segundo. Al igual que en la Figura 6.8, ambos gráficos son muy similares. En este caso, los modelos que obtuvieron el menor MSE promedio también son aquellos que obtuvieron el menor MSE. En ambos gráficos, puede observarse que, para la mayoría de los usuarios, las redes neuronales que mayor desempeño obtuvieron en cada usuario al filtrar por los modelos impersonales son las TCN. Aunque fueron las CNN las que mejor desempeño obtuvieron para el grupo del usuario 34.

Se llevó a cabo el Test de *Student* para comparar las diferentes redes neuronales. Para el grupo del usuario 32 existe una diferencia estadística menor 0,05 entre las TCN y las CNN ($p=0,022$) y entre las TCN y las MLP ($p=0,034$). No existe diferencia estadística entre las TCN y las RNN para el conjunto de modelos impersonales. No se halla una diferencia estadística significativa ($p < 0,05$) para el grupo del usuario 34, por lo que, a pesar de ser las CNN las que obtuvieron el mejor desempeño en ese grupo, la diferencia no es amplia, sino que es similar a la de las otras redes neuronales.

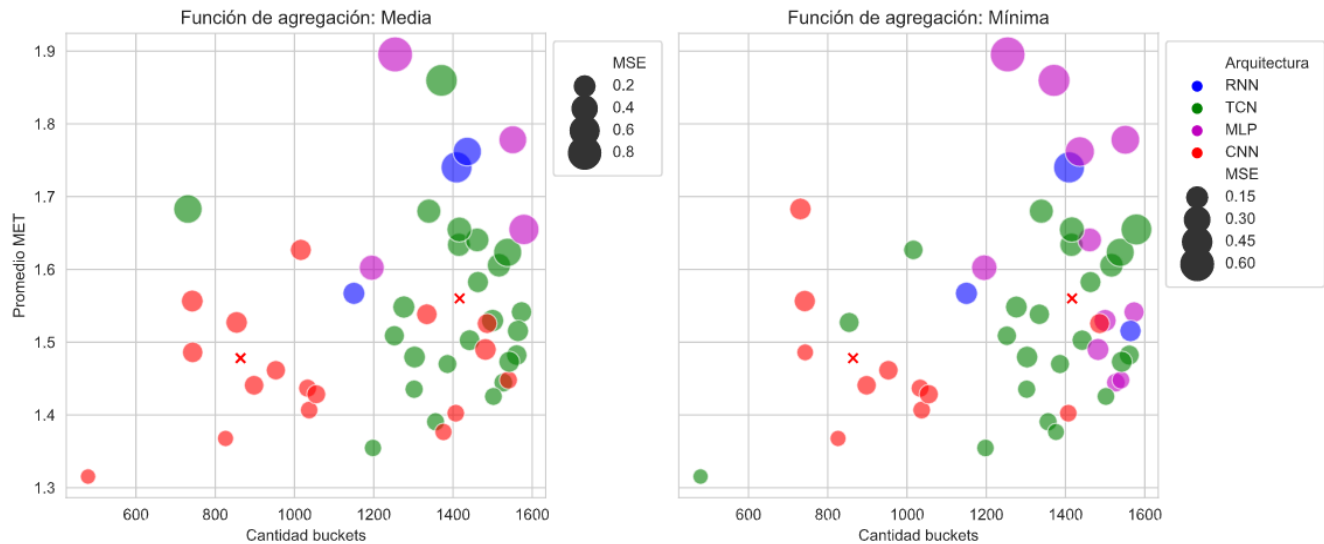


Figura 6.9. Mejores redes neuronales para los modelos impersonales

6.4. Tiempo de entrenamiento a lo largo de las iteraciones

En las discusiones anteriores se tomó al MSE como el aspecto más importante, ya que un menor MSE permite predecir con mayor precisión el comportamiento sedentario futuro. Sin embargo, también resulta de relevancia analizar los tiempos de ejecución. El tiempo en que el modelo es entrenado es también el tiempo que el usuario deberá esperar para poder obtener predicciones. Las diferentes características de los modelos tienen implicancias en cuanto a cuánto lleva a entrenar el modelo, así como también las tiene el ambiente de cómputo donde dicho modelo es entrenado.

Tras observar los resultados mostrados en las Tablas 5.15 a 5.20 puede notarse que existe una gran diferencia entre el tiempo requerido para entrenar los modelos dependiendo de la naturaleza del

mismo (personal o impersonal). El valor del p -value del Test de Student al comparar estos dos conjuntos es menor a 0,05 por lo que existe una diferencia estadística entre el tiempo requerido para entrenar un modelo personal y uno impersonal. Esto era esperable, ya que los modelos impersonales poseen muchos más datos de entrenamiento que los modelos personales, por lo que el algoritmo de aprendizaje requiere muchas más iteraciones para finalizar una época en el primer caso. En el caso de los modelos personales, la arquitectura RNN fue la que menor tiempo requiere para ser entrenada para cada uno de los usuarios, mientras que en el caso de los modelos impersonales, la arquitectura CNN fue la que menos tiempo requiere para ser entrenada para cada uno de los usuarios.

En las Figuras 6.10 y 6.11 pueden observarse el tiempo requerido para entrenar el modelo en cada uno de los experimentos personales e impersonales respectivamente. Para cada experimento, se grafican los 5 puntos correspondientes a cada una de las iteraciones y cada par de puntos consecutivos se une por una línea. Los experimentos de cada arquitectura poseen un color especial para que pueda observarse si existen patrones que diferencian a las distintas arquitecturas. A grandes rasgos, puede notarse un carácter lineal en el tiempo de entrenamiento. Sin embargo, la pendiente de dicha linealidad varía según las diferentes configuraciones de los modelos.

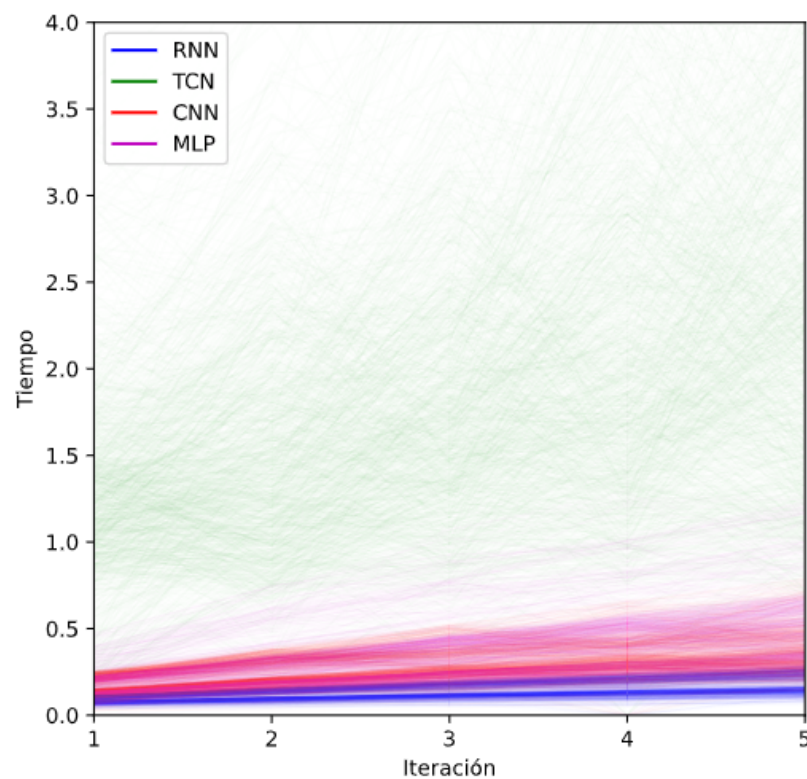


Figura 6.10. Tiempo para entrenar modelos personales por iteración

Con respecto a los modelos personales puede observarse que todas las arquitecturas menos las TCN se llevaron a cabo, en general, en menos de medio minuto, mientras que el tiempo requerido para entrenar las TCN varía mucho ya que puede observarse una bruma en la parte superior de la imagen. Esto puede deberse en parte a que las TCN fueron la única arquitectura cuyo conjunto de hiperparámetros no

fue completamente determinado al finalizar el proceso de *tuning*, ya que debió calcularse la cantidad de dilaciones para cada experimento en particular.

En el caso de los modelos impersonales, los modelos CNN y las MLP fueron entrenadas en una cantidad substancial de tiempo menor que las otras dos arquitecturas. Para los modelos TCN, puede observarse un fenómeno similar al de la figura anterior, donde no se identifica un claro patrón, sino que todas las líneas verdes parecen formar una nube, significando así que existe una gran diferencia en el tiempo requerido para entrenar este tipo de modelos. Por último, los modelos RNN pasan a ser, junto a las TCN, los más lentos para ser entrenados cuando la naturaleza del modelo es impersonal. En el caso de los RNN, esto se debe a su baja capacidad de paralelización, en comparación con otros modelos como los CNN que son altamente paralelizables. Otro fenómeno que puede observarse en el caso de los modelos RNN es la existencia de 4 líneas diferentes claramente distinguibles. Esto se debe, posiblemente, a las 4 configuraciones de hiperparámetros que fueron determinadas para los modelos RNN al finalizar el proceso de *tuning*. Este fenómeno seguramente se da en el caso de los CNN y MLP pero al ser tan bajo el tiempo de entrenamiento no llega a ser visible.

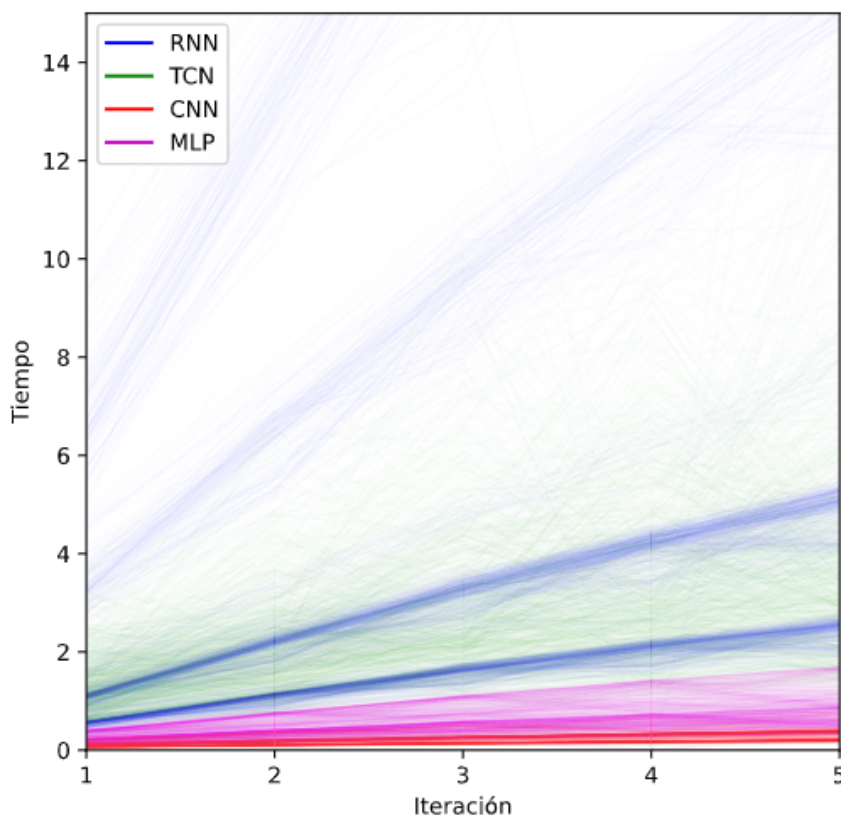


Figura 6.11. Tiempo para entrenar modelos impersonales por iteración

6.5. Granularidad

Una de las categorías que se definió al momento de diseñar los experimentos es la granularidad. La granularidad define el intervalo de tiempo a partir del cual se computan las características que representan a cada *bucket*. Los valores que se definieron para realizar la experimentación fueron de 30 minutos y una hora. Por lo tanto, como ya fue mencionado, los *datasets* a partir de los cuales fueron entrenados los experimentos con granularidad de 30 minutos poseen aproximadamente el doble casos de entrenamiento que aquellos con granularidad de 1 hora. Una menor granularidad permite una mejor precisión temporal a la hora de predecir el comportamiento sedentario futuro, mientras que las características utilizadas para entrenar al modelo poseen una menor cantidad de datos para realizar los cálculos.

Se llevó a cabo el test de *Student* con la finalidad de comparar el MSE de estos dos tipos de modelos (granularidad de 30 minutos y una hora) y no se halló diferencia estadística. Sin embargo, como puede observarse en la Figura 6.12, los modelos con granularidad de 1 hora poseen un MSE menor para la gran mayoría de los usuarios si se utiliza la media como función de agregación. Si se realiza el mismo test comparando el tiempo que requieren los modelos para ser entrenados sí se halla una diferencia estadística. Esto extiende lo ya visto en la [Sección 6.4](#). Aquellos valores de las características que varían entre los diferentes experimentos que aumentan la cantidad de registros de los *datasets* de entrenamiento se traducen en un aumento considerable del tiempo requerido para entrenar los modelos. Por lo tanto, es interesante notar que, dependiendo de qué característica sea la más importante minimizar, pueden ser tomados, preferiblemente, una granularidad o la otra.

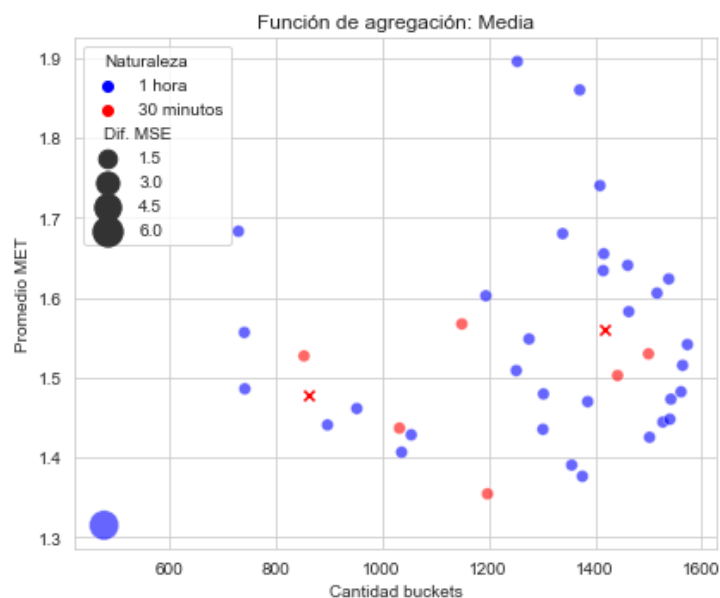


Figura 6.12. Mejores modelos según su granularidad

6.6. Número de *lags*

Como ya ha sido explicado, para llevar a cabo los experimentos se probaron diferentes números de *lags* -1, 2, 4 y 8-. Es posible deducir que cuanta más información del pasado se le dé al modelo, más información tendrá para ser entrenado y así obtener un mejor desempeño. Esto puede no ser el caso por dos razones. La primera de ellas es que el MET que tendrá un usuario en el tiempo $t+1$ puede no estar relacionado con las *features* que poseemos del mismo usuario en el tiempo $t-n$, siendo n un entero arbitrario. Esto, al tener pocos casos de entrenamiento, puede generar ruido en el modelo. La segunda razón es que, como en muchos casos no se poseen datos sobre todas las horas que duró el experimento, cuantos más *lags* se tengan, mayor es la probabilidad de que un caso de entrenamiento sea descartado. Por ejemplo, si la cantidad de *lags* es n y si se tienen los datos del usuario x desde el tiempo t al tiempo $t - n + 1$ pero no del tiempo $t - n$, el caso de entrenamiento es desechado.

Estas razones explican los resultados obtenidos, en los cuales se halló que los modelos con menor número de *lags* obtuvieron un mejor desempeño. Se llevó a cabo test de Student para todos los pares posibles de números de *lags* utilizando el conjunto de experimentos entero y el MSE como medida de comparación. Se halló una diferencia estadística (p -value menor a 0,5) para todos los pares de números de *lags* posibles salvo en el caso de 4 y 8 *lags*. Analizando más detenidamente los conjuntos de experimentos es posible observar que esto se dio principalmente porque los experimentos con un número de *lags* más alto sufrieron en mayor medida del problema de *exploding gradient*. Esto se verifica ya que al llevar a cabo el test sin tener en cuenta el último decil de cada conjunto de experimentos no se halla ninguna diferencia estadística.

Con respecto al tiempo requerido para entrenar los modelos de acuerdo al número de *lags* utilizados, se realizó el mismo procedimiento que el descrito en el párrafo anterior, salvo que se usó el tiempo promedio de los experimentos como medida de comparación para llevar a cabo el test de Student. Los resultados de los tests no indicaron, para ninguno de los números de *lags*, diferencia estadística, lo que indica que la cantidad de columnas utilizadas para entrenar los modelos no incrementa el tiempo requerido para entrenarlos. Este resultado se debe posiblemente a la alta capacidad de paralelización del GPU utilizado para entrenar los modelos.

6.7. *Exploding gradient*

Como ya se ha mencionado brevemente en otras secciones del presente capítulo, muchos de los modelos sufrieron el problema conocido como *exploding gradient*. Dicho problema se produce cuando los valores de los pesos de las redes neuronales -típicamente conocidos como los parámetros de las mismas- crecen de tal forma que sus valores son mayores que uno. De esta forma, ocurre que, al llevarse a cabo la propagación hacia delante, el resultado de las capas intermedias crezca exponencialmente, y por lo tanto también la salida del modelo, que es, para el problema que se aborda en esta tesis, el MSE. Este problema resulta en que el modelo no pueda aprender del conjunto de entrenamiento.

Todas las redes, salvo las RNN, sufrieron el problema de *exploding gradient* en mayor o menor medida. Esto ha sido así ya que las unidades LSTM que fueron utilizadas en las RNN están especialmente diseñadas para evitar este problema. En el caso de las demás redes neuronales utilizadas no se tomó

ninguna medida para evitar este problema debido a que no fue detectada al momento de realizar el proceso de *tuning*. En caso de que se hubiera detectado el *exploding gradient* durante el proceso de *tuning* se podría haber utilizado una técnica sencilla conocida como *gradient clipping* o recorte del gradiente. El *gradient clipping* consiste en que el valor de los pesos del modelo no sea mayor que un valor determinado, como por ejemplo, 1. De esta forma, cuando se realiza la actualización de los pesos del modelo al finalizar el recorrido del bache en cuestión, el valor de los pesos será igual a 1 en caso de que la actualización de como resultado un peso mayor a dicho valor.

	0	0,1	0,2	0,3	0,4	0,5	0,6	0,7	0,8	0,9	1
RNN	0,03	0,101	0,138	0,181	0,225	0,268	0,319	0,374	0,494	0,691	1,922
CNN	0,027	0,12	0,176	0,229	0,286	0,371	0,504	0,925	9,948	156,851	$1,11 \times 10^{10}$
TCN	0,024	0,137	0,199	0,255	0,346	0,499	0,796	3,255	38,039	784,318	$2,39 \times 10^{10}$
MLP	0,041	0,145	0,201	0,254	0,319	0,431	0,707	2,287	19,646	358,225	$2,16 \times 10^{10}$

Tabla 6.5, MSE de los deciles de los modelos personales

El problema de *exploding gradient* se produjo, salvo pocos casos, únicamente en los modelos personales. Esto puede ser explicado por la poca cantidad de casos de entrenamiento que estos modelos poseen. A modo de evidenciar el problema de *exploding gradient* en las siguientes tablas se muestran los deciles correspondientes al conjunto de experimentos. En la Tabla 6.5 se muestran los experimentos personales, mientras que en la segunda se muestran los impersonales.

Como puede observarse en la Tabla 6.5, en los deciles 7, 8, 9 y 10 se encuentran entre el 30% y 40% de los experimentos personales que sufrieron el problema que se trata en esta sección para todas las redes neuronales salvo las RNN. Más específicamente, el 35,7% de este conjunto de experimentos tiene un MSE mayor a 1. Es interesante notar que este porcentaje es menor si se toman individualmente cada una de las iteraciones del proceso de evaluación (26%, 19%, 12%, 9% y 5% respectivamente), lo que nos indica que el problema de *exploding gradient* se dio a lo largo de todas las iteraciones y no en una en particular, aunque es cierto que se dio en mayor magnitud en las primeras iteraciones. Lo anterior demuestra que este problema es provocado mayormente a causa de la poca cantidad de datos disponibles para el experimento en las primeras iteraciones.

	0	0,1	0,2	0,3	0,4	0,5	0,6	0,7	0,8	0,9	1
RNN	0,035	0,091	0,128	0,157	0,191	0,233	0,275	0,322	0,42	0,609	1,403
CNN	0,013	0,084	0,118	0,149	0,185	0,228	0,275	0,319	0,419	0,638	705,582
TCN	0,012	0,086	0,118	0,151	0,183	0,225	0,269	0,315	0,406	0,64	149,907
MLP	0,028	0,089	0,123	0,153	0,186	0,228	0,273	0,318	0,416	0,642	339,085

Tabla 6.6, MSE de los deciles de los modelos impersonales

En el caso de la Tabla 6.6, donde se muestran los deciles para los experimentos impersonales, únicamente se encuentran experimentos que sufrieron el problema de *exploding gradient* en el último

decil y en una magnitud radicalmente menor. Si se realiza el mismo cálculo porcentual que se realizó en el párrafo anterior, se observa que solo el 2% de este conjunto de experimentos tiene un MSE mayor a 1. En este caso, se entiende que el problema no se produjo debido a que para las primeras iteraciones se tienen los datos de todos los demás usuarios.

Resumen

En este capítulo se presentó la discusión llevada a cabo a partir de los resultados obtenidos. En la [Sección 6.1](#) se buscó comprender el impacto de los usuarios modelo seleccionados. En la [Sección 6.2](#) se estudió la diferencia entre los modelos personales y los impersonales. En ella, se discutió cómo los modelos impersonales son mejores cuando aún no se tienen muchos datos sobre un usuario y que los modelos personales mejoran su desempeño cuando se tiene una cantidad considerable de datos disponibles para ser entrenados. En la [Sección 6.3](#) se comparó el desempeño de las diferentes redes neuronales. Se observó que las RNN fueron el único tipo de red neuronal que no sufrió el problema de *Exploding Gradient*. En el caso de los modelos personales, las RNN fueron las que mejor desempeño obtuvieron, mientras que para los modelos impersonales fueron ambos tipos de redes neuronales, RNN y TCN, las que obtuvieron un mejor desempeño que las demás, no existiendo una diferencia estadística entre ellas. En la [Sección 6.4](#) se comparó el tiempo de entrenamiento para los diferentes modelos. Se identificó que la única categoría que implica una gran diferencia entre el tiempo de entrenamiento de los modelos es por la naturaleza de los mismos, es decir, si son personales o impersonales. En la [Sección 6.5](#) se analizó el impacto de las diferentes granularidades. Aunque los modelos que utilizaron una granularidad de una hora tuvieron un mejor desempeño que aquellos que utilizaron una granularidad de 30 minutos, no se halló diferencia estadística entre ellos, aunque sí se halló diferencia estadística en cuanto al tiempo requerido para entrenar los modelos, debido a que los modelos que utilizaron una granularidad de 30 minutos poseen el doble de cantidad de datos para ser entrenados. En la [Sección 6.6](#) se estudiaron las diferencias entre los diferentes números de *lags*, tanto a partir del MSE como a partir del tiempo requerido para entrenar los modelos. En cuanto a la comparación por MSE, se encontró que, a mayor número de *lags*, los modelos sufrieron el problema de *Exploding Gradient* en mayor medida. Por último, en la [Sección 6.7](#) se estudió el problema de *exploding gradient*, el cual sufrieron una gran cantidad de los modelos entrenados a partir de los diferentes experimentos. Para comprender mejor hasta qué punto se produjo este problema se dividió el conjunto de experimentos en deciles, tanto para modelos personales como impersonales y para cada uno de los tipos de redes neuronales.

Capítulo 7: Conclusión

En este capítulo se presenta la conclusión a modo de cierre del presente trabajo de tesis. Este capítulo está dividido en 3 secciones. Primero, en la [Sección 7.1](#), se presenta el resumen de lo llevado a cabo. Luego, en la [Sección 7.2](#) se enuncian las limitaciones halladas al momento de realizar el presente trabajo. Por último, en la [Sección 7.3](#) se enuncian los posibles trabajos futuros a modo de extensión de lo ya desarrollado.

7.1. Resumen

En este trabajo se propuso abordar la tarea de la Predicción del Comportamiento Sedentario Futuro mediante modelos de *Deep Learning* entrenados con datos de *lifelogging*. Se seleccionaron y compararon 4 tipos de redes neuronales: MLP, CNN, RNN y TCN, donde la primera no puede sacar provecho del carácter temporal de los datos. Además, se evaluaron modelos personales e impersonales, así como también diferentes configuraciones acerca de qué información del pasado disponen las redes neuronales para ser entrenadas: granularidad de los *buckets*, cantidad de *lags*, y el periodo.

A diferencia de trabajos relacionados, en lugar de utilizar el porcentaje de actividad sedentaria/no sedentaria, se utilizó el Equivalente Metabólico de Tareas para determinar los comportamientos sedentarios (MET). MET es la medida estándar en el área de la salud para medir la intensidad de una actividad en términos del gasto energético. Esta métrica no había sido utilizada en otros trabajos para el comportamiento sedentario futuro utilizando dispositivos móviles.

Para evaluar la hipótesis se utilizó un *dataset* disponible públicamente llamado *StudentLife* compuesto por la información obtenida de una gran variedad de sensores disponibles en *smartphones* de 49 usuarios a lo largo de 10 semanas. Se presentaron usuarios con diferentes niveles de gasto energético.

Para evaluar los modelos se desarrolló una técnica de validación que permite la comparación entre modelos personales e impersonales especialmente aplicables a problemas donde el orden temporal es importante. Dicha técnica de validación divide temporalmente al *dataset* de entrenamiento en k particiones a partir de las cuales se entrenan k modelos y se promedia su desempeño. En total, se llevaron a cabo 9.216 experimentos y se entrenaron y testearon 36.864 modelos. Para entrenar y testear cada modelo, se utilizó un *dataset* que se obtiene como resultado de la aplicación de una serie de etapas de procesamiento que varía dependiendo de las características del experimento en cuestión.

Para encontrar el valor óptimo de los hiperparámetros de los modelos de *Deep Learning* se llevó a cabo un proceso de *tuning* utilizando el método de Optimización de Bayes. Con la finalidad de ajustar los hiperparámetro de los modelos en un corto periodo de tiempo, el proceso de *tuning* se llevó a cabo para ciertos experimentos modelo. En total, se seleccionaron 16 experimentos. Se llevó a cabo el proceso de *tuning* para cada uno de ellos utilizando como máximo 100 iteraciones, donde cada iteración consiste en el entrenamiento de 5 modelos de acuerdo al modelo de validación anteriormente nombrado cuando $k = 5$.

Esta tesis parte del trabajo que se llevó a cabo en (Santillán Cooper & Armentano, 2019) -autor y director de esta tesis-, que fue el primero en evaluar el desempeño de redes neuronales que intentan sacar provecho al carácter secuencial del problema tratado.

A través del análisis de los resultados obtenidos de los experimentos, se encontró que los modelos impersonales son los más aptos a la hora de predecir el comportamiento sedentario futuro debido a la poca cantidad de datos disponibles para los modelos personales, aunque su entrenamiento lleva mucho más tiempo debido a la gran cantidad de datos disponibles. Para el caso de los modelos impersonales, tanto las TCN como las RNN obtuvieron el mejor desempeño para la mayoría de los usuarios.

7.2. Limitaciones

Una de las limitaciones halladas radica en el *dataset* utilizado para el entrenamiento de los modelos. El *dataset StudentLife* fue el único compatible con los requerimientos expresados en la [Sección 4.1](#). Aunque este *dataset* pudo ser finalmente utilizado, presentó ciertas limitaciones que resultaron en diferentes dificultades a la hora de procesarlo y utilizarlo para entrenar modelos de *Deep Learning* para PCSF.

Una de las dificultades encontradas fue la gran cantidad de inconsistencias halladas con respecto a la actividad física del usuario 48, las cuales motivaron la decisión de no considerarlo en los experimentos. La anterior no fue la única inconsistencia hallada, sino que se identificaron otras relacionadas a la discrepancia entre la descripción del *dataset* y el *dataset* disponible. Dichas discrepancias fueron tratadas adecuadamente. Aun así, no es posible verificar que se han eliminado todas las inconsistencias del *dataset*, debido a la gran cantidad de datos disponibles.

Otra dificultad relacionada al *dataset* fue que una gran cantidad de *buckets* tuvo que ser descartada por la falta de información disponible. Sin embargo, se llevó a cabo un análisis con el fin de maximizar la cantidad de información disponible. Es decir, en los casos en los que fue posible utilizó la falta de datos como información. Por ejemplo, para la característica calculada a partir del sensor GPS `total_distance` en los *buckets* para los cuales no se tienen datos disponibles se le asignó un 0, suponiendo así que el usuario había permanecido en el mismo lugar geográfico.

La granularidad de la actividad física reportada para los usuarios del *dataset* es, quizás, demasiado baja para la gran variedad de actividades físicas posibles en términos de su nivel de MET. Por ejemplo, para las actividades clasificadas como "caminando" se les otorgó un valor de 5 MET, mientras que existen más de 50 actividades dentro del espectro de "caminando" consideradas en el Compendio. Cada una de esas actividades posee un valor diferente de MET, que varía entre 2 y 12 MET. Por lo tanto, se podrían haber obtenido mejores resultados si se dispusiera de actividades detalladas con una mayor granularidad.

Otra de las limitaciones fue que no se hicieron disponibles los datos crudos del acelerómetro, que podrían haber abierto otros caminos a la hora de diseñar los modelos de *Deep Learning*, permitiendo arquitecturas más extensas y complejas, que pudieran capturar más información sobre la actividad física del usuario, pudiendo así enriquecer la baja granularidad disponible.

7.3. Trabajos futuros

Las inconsistencias encontradas en el *dataset* utilizado y su antigüedad motivan el registro de un nuevo *dataset* que podría ser recolectado a partir de un grupo poblacional, por ejemplo, un grupo de estudiantes de UNICEN, utilizando métodos y tecnologías más eficientes para recopilar y almacenar los datos de los sensores disponibles en los *smartphones*. Por ejemplo, puede ser utilizada la API que provee Google para acceder a los datos crudos de los sensores¹⁸, en lugar de tener que desarrollar un sistema *ad-hoc* como se hizo en el caso del *dataset StudentLife*.

Para lograr el objetivo anterior, sería necesario el desarrollo previo de una aplicación móvil que permita la recopilación de los datos de los sensores de los *smartphones* de los usuarios y almacenarlos en

¹⁸ <https://developers.google.com/fit/android/sensors>

un servidor. Dichos datos podrían ser utilizados para entrenar múltiples modelos de *Deep Learning*. Cuando un usuario posea pocos datos -también conocido como *cold start*- se utilizaría un modelo impersonal, mientras que al cabo de un tiempo determinado podría comenzarse a utilizar un modelo personal para dicho usuario. El modelo realizaría predicciones periódicas y, si detecta que el usuario llevará a cabo un comportamiento sedentario en un futuro cercano (dependiendo de la granularidad temporal) se le podrían aconsejar diferentes actividades, como salir a caminar o hacer ejercicio.

En esta tesis se explora la Predicción del Comportamiento Sedentario Futuro en el siguiente intervalo de tiempo al momento presente. Sin embargo, continúa inexplorada la predicción para otros intervalos de tiempo. Incluso, si el modelo es lo suficientemente preciso, se podrían predecir múltiples intervalos de tiempo consecutivos - por ejemplo, 2 horas, toda la tarde o un día entero- que enriquecería el valor de las propuestas que realizaría al usuario la aplicación introducida en el párrafo anterior.

Muchas cuestiones relacionadas a las redes neuronales utilizadas no fueron exploradas en pos de acotar el universo de los experimentos. Cada una de las redes neuronales, quizás excepto la más sencilla de ellas, la MLP, posee hiperparámetros y variaciones arquitecturales que no han sido exploradas. Además, técnicas novedosas utilizadas en redes neuronales podrían ser puestas a prueba para la tarea abordada en esta tesis, como la atención (Vaswani et al., 2017).

Bibliografía

- Abdel-Hamid, O., Deng, L., & Yu, D. (2013). Exploring convolutional neural network structures and optimization techniques for speech recognition. *Interspeech 2013*. Published.
<https://doi.org/10.21437/interspeech.2013-744>
- Agnihotri, A., & Batra, N. (2020). Exploring Bayesian Optimization. *Distill*, 5(5), e26.
- Ainsworth, B. E., Haskell, W. L., Herrmann, S. D., Meckes, N., Bassett, D. R., Jr, Tudor-Locke, C., Greer, J. L., Vezina, J., Whitt-Glover, M. C., & Leon, A. S. (2011). 2011 Compendium of Physical Activities: a second update of codes and MET values. *Medicine and Science in Sports and Exercise*, 43(8), 1575–1581.
- Ajzen, Icek. (1991). “The theory of planned behavior.” *Organizational Behavior and Human Decision Processes*, 179-211.
- Ali, S., Khusro, S., Hassan, L., & Khan, A. A. (2014). A Survey of Mobile Phones Context-Awareness Using Sensing Computing Research. *Journal of Engineering and Applied Sciences*, 33(2), 75–93.
- Atkin, A. J., Gorely, T., Clemes, S. A., Yates, T., Edwardson, C., Brage, S., Salmon, J., Marshall, S. J., & Biddle, S. J. H. (2012). Methods of Measurement in epidemiology: sedentary Behaviour. *International Journal of Epidemiology*, 41(5), 1460–1471.
- Bai, S., Zico Kolter, J., & Koltun, V. (2018). *An Empirical Evaluation of Generic Convolutional and Recurrent Networks for Sequence Modeling*. <http://dx.doi.org/>
- Benatti, F. B., & Ried-Larsen, M. (2015). The Effects of Breaking up Prolonged Sitting Time. In *Medicine & Science in Sports & Exercise* (Vol. 47, Issue 10, pp. 2053–2061).
<https://doi.org/10.1249/mss.0000000000000654>
- Bengio, Y., Simard, P., & Frasconi, P. (1994). Learning long-term dependencies with gradient descent is difficult. *IEEE Transactions on Neural Networks / a Publication of the IEEE Neural Networks*

- Council*, 5(2), 157–166.
- Berry, M. W., Mohamed, A., & Yap, B. W. (2019). *Supervised and Unsupervised Learning for Data Science*. Springer Nature.
- Bouckaert, R. R. (2003). Choosing Between Two Learning Algorithms Based on Calibrated Tests. *Machine Learning, Proceedings of the Twentieth International Conference (ICML 2003), August 21-24, 2003, Washington, DC, USA*, 51–58.
- Carter, S., Hartman, Y., Holder, S., Thijssen, D. H., & Hopkins, N. D. (2017). Sedentary Behavior and Cardiovascular Disease Risk: Mediating Mechanisms. *Exercise and Sport Sciences Reviews*, 45(2), 80–86.
- Cho, K., van Merriënboer, B., Bahdanau, D., & Bengio, Y. (2014). On the Properties of Neural Machine Translation: Encoder–Decoder Approaches. *Proceedings of SSST-8, Eighth Workshop on Syntax, Semantics and Structure in Statistical Translation*, 103–111.
- Chollet, F. (2017). *Deep Learning with Python*. Manning Publications Company.
- Cook, D. J., & Krishnan, N. C. (2015). *Activity Learning: Discovering, Recognizing, and Predicting Human Behavior from Sensor Data*. John Wiley & Sons.
- Dauphin, Y. N., Fan, A., Auli, M., & Grangier, D. (2017). Language Modeling with Gated Convolutional Networks. *International Conference on Machine Learning*, 933–941.
- Diane J. C., Narayanan C. K.. Activity Prediction. (2015). Activity Learning, 127–147.
<https://doi.org/10.1002/9781119010258.ch7>
- Dutta, M. J., Kaur-Gill, S., Tan, N., & Lam, C. (2018). mHealth, Health, and Mobility: A Culture-Centered Interrogation. In *mHealth Innovation in Asia* (pp. 91–107). Springer, Dordrecht.
- Fahim, M., Baker, T., Khattak, A. M., & Alfandi, O. (2017). Alert me: Enhancing active lifestyle via observing sedentary behavior using mobile sensing systems. *2017 IEEE 19th International Conference on E-Health Networking, Applications and Services (Healthcom)*, 1–4.

- Felez-Nobrega, M., Hillman, C. H., Dowd, K. P., Cirera, E., & Puig-Ribera, A. (2018). ActivPAL™ determined sedentary behaviour, physical activity and academic achievement in college students. In *Journal of Sports Sciences* (Vol. 36, Issue 20, pp. 2311–2316).
<https://doi.org/10.1080/02640414.2018.1451212>
- Finding structure in time. (1990). *Cognitive Science*, 14(2), 179–211.
- Gong, J., Huang, Y., Chow, P. I., Fua, K., Gerber, M. S., Teachman, B. A., & Barnes, L. E. (2019). Understanding behavioral dynamics of social anxiety among college students through smartphone sensors. In *Information Fusion* (Vol. 49, pp. 57–68). <https://doi.org/10.1016/j.inffus.2018.09.002>
- Goodfellow, I., Bengio, Y., & Courville, A. (2016). *Deep Learning*. MIT Press.
- Görtler, J., Kehlbeck, R., & Deussen, O. (2019). A Visual Exploration of Gaussian Processes. *Distill*, 4(4), e17.
- Grundgeiger, T., Pichen, J., Häfner, J., & Huber, S. (2017). Combating Sedentary Behavior: An App Based on a Distributed Prospective Memory Approach. *The 2017 CHI Conference Extended Abstracts*, 1632–1639.
- Harari, G. M., Gosling, S. D., Wang, R., Chen, F., Chen, Z., & Campbell, A. T. (2017). Patterns of behavior change in students over an academic term: A preliminary study of activity and sociability behaviors using smartphone sensing methods. In *Computers in Human Behavior* (Vol. 67, pp. 129–138). <https://doi.org/10.1016/j.chb.2016.10.027>
- He, Q., & Agu, E. (2014). *On11: An activity recommendation application to mitigate sedentary lifestyle*. <https://doi.org/10.1145/2611264.2611268>
- He, Q., & Agu, E. (2017). A Rhythm Analysis-Based Model to Predict Sedentary Behaviors. *2017 IEEE/ACM International Conference on Connected Health: Applications, Systems and Engineering Technologies (CHASE)*, 383–391.
- He, Q., & Agu, E. O. (2016a). A frequency domain algorithm to identify recurrent sedentary behaviors

- from activity time-series data. *2016 IEEE-EMBS International Conference on Biomedical and Health Informatics (BHI)*. <https://doi.org/10.1109/bhi.2016.7455831>
- He, Q., & Agu, E. O. (2016b). Smartphone usage contexts and sensible patterns as predictors of future sedentary behaviors. *2016 IEEE Healthcare Innovation Point-Of-Care Technologies Conference (HI-POCT)*. <https://doi.org/10.1109/hic.2016.7797695>
- He, Q., & Agu, E. O. (2016c). Towards sedentary lifestyle prevention: An autoregressive model for predicting sedentary behaviors. *2016 10th International Symposium on Medical Information and Communication Technology (ISMICT)*. <https://doi.org/10.1109/ismict.2016.7498879>
- Hermans, Michiel, and Benjamin Schrauwen. 2013. "Training and Analyzing Deep Recurrent Neural Networks." *Advances in Neural Information Processing Systems 26 (2013)*.
- Hochreiter, S., & Schmidhuber, J. (1997). Long short-term memory. *Neural Computation*, 9(8), 1735–1780.
- Kalchbrenner, N., Espeholt, L., Simonyan, K., van den Oord, A., Graves, A., & Kavukcuoglu, K. (2016). *Neural Machine Translation in Linear Time*. <https://research.google/pubs/pub48560/>
- Kanjo, E., Younis, E. M. G., & Ang, C. S. (2019). Deep learning analysis of mobile physiological, environmental and location sensor data for emotion detection. In *Information Fusion* (Vol. 49, pp. 46–56). <https://doi.org/10.1016/j.inffus.2018.09.001>
- Koster, A., Caserotti, P., Patel, K. V., Matthews, C. E., Berrigan, D., Van Domelen, D. R., Brychta, R. J., Chen, K. Y., & Harris, T. B. (2012). Association of sedentary time with mortality independent of moderate to vigorous physical activity. *PloS One*, 7(6), e37696.
- Kroenke, K., Spitzer, R. L., & Williams, J. B. (2001). The PHQ-9: validity of a brief depression severity measure. *Journal of General Internal Medicine*, 16(9), 606–613.
- Lane, N., Mohammad, M., Lin, M., Yang, X., Lu, H., Ali, S., Doryab, A., Berke, E., Choudhury, T., & Campbell, A. (2012). BeWell: A Smartphone Application to Monitor, Model and Promote

- Wellbeing. *5th International ICST Conference on Pervasive Computing Technologies for Healthcare*. <http://eudl.eu/pdf/10.4108/icst.pervasivehealth.2011.246161>
- Lea, C., Flynn, M. D., Vidal, R., Reiter, A., & Hager, G. D. (2017). Temporal Convolutional Networks for Action Segmentation and Detection. *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, 156–165.
- Lea, C., Vidal, R., Reiter, A., & Hager, G. D. (2016). Temporal Convolutional Networks: A Unified Approach to Action Segmentation. *Computer Vision – ECCV 2016 Workshops*, 47–54.
- Lecun, Y., & Bengio, Y. (1995). Convolutional Networks for Images, Speech, and Time-Series. *The Handbook of Brain Theory and Neural Networks*. <http://dx.doi.org/>
- Lee, H., Pham, P., Largman, Y., & Ng, A. (2009). Unsupervised feature learning for audio classification using convolutional deep belief networks. *Advances in Neural Information Processing Systems*, 22. <https://proceedings.neurips.cc/paper/2009/file/a113c1ecd3cace2237256f4c712f61b5-Paper.pdf>
- Lu, H., Yang, J., Liu, Z., Lane, N. D., Choudhury, T., & Campbell, A. T. (2010). The Jigsaw continuous sensing engine for mobile phone applications. *Proceedings of the 8th ACM Conference on Embedded Networked Sensor Systems - SenSys '10*. the 8th ACM Conference, Zürich, Switzerland. <https://doi.org/10.1145/1869983.1869992>
- Magnon, V., Vallet, G. T., & Auxiette, C. (2018). Sedentary Behavior at Work and Cognitive Functioning: A Systematic Review. *Frontiers in Public Health*, 6, 239.
- Mahata, S. K., Das, D., & Bandyopadhyay, S. (2019). MTIL2017: Machine Translation Using Recurrent Neural Network on Statistical Machine Translation. *Journal of Intelligent Systems*, 28(3), 447–453.
- Mitchell, T. M. (1997). *Machine Learning*. McGraw-Hill Education.
- Murphy, K. P. (2012). *Machine Learning: A Probabilistic Perspective*. MIT Press.
- Mustapha, A., Mohamed, L., & Ali, K. (2020). An Overview of Gradient Descent Algorithm Optimization in Machine Learning: Application in the Ophthalmology Field. *Smart Applications*

and Data Analysis, 349–359.

- Paing, A. C., McMillan, K. A., Kirk, A. F., Collier, A., Hewitt, A., & Chastin, S. F. M. (2018). The associations of sedentary time and breaks in sedentary time with 24-hour glycaemic control in type 2 diabetes. *Preventive Medicine Reports*, 12, 94–100.
- Palaz, D., Collobert, R., & Magimai.-Doss, M. (2013). *Estimating Phoneme Class Conditional Probabilities from Raw Speech Signal using Convolutional Neural Networks*. <http://dx.doi.org/>
- Pascanu, Razvan, Tomas Mikolov, and Yoshua Bengio. (2013). On the diculty of training Recurrent Neural Networks.
- Pelletier, C., Webb, G., & Petitjean, F. (2019). Temporal Convolutional Neural Network for the Classification of Satellite Image Time Series. In *Remote Sensing* (Vol. 11, Issue 5, p. 523). <https://doi.org/10.3390/rs11050523>
- Refaeilzadeh, P., Tang, L., & Liu, H. (2009). Cross-Validation. In *Encyclopedia of Database Systems* (pp. 532–538). Springer, Boston, MA.
- Russell, D. W. (1996). UCLA Loneliness Scale (Version 3): reliability, validity, and factor structure. *Journal of Personality Assessment*, 66(1). https://doi.org/10.1207/s15327752jpa6601_2
- Saeb, S., Lattie, E. G., Schueller, S. M., Kording, K. P., & Mohr, D. C. (2016). The relationship between mobile phone location sensor data and depressive symptom severity. *PeerJ*, 4, e2537.
- Saeb, S., Zhang, M., Karr, C. J., Schueller, S. M., Corden, M. E., Kording, K. P., & Mohr, D. C. (2015). Mobile Phone Sensor Correlates of Depressive Symptom Severity in Daily-Life Behavior: An Exploratory Study. *Journal of Medical Internet Research*, 17(7), e175.
- Santillán Cooper, M., & Armentano, M. G. (2019). Modelos de aprendizaje profundo para la predicción de comportamiento sedentario futuro. *XX Simposio Argentino de Inteligencia Artificial (ASAI 2019) - JAIIO 48 (Salta)*. <http://sedici.unlp.edu.ar/handle/10915/87936>
- Sedentary Behaviour Research Network. (2012). Letter to the editor: standardized use of the terms

- “sedentary” and “sedentary behaviours.” *Applied Physiology, Nutrition, and Metabolism* = *Physiologie Appliquee, Nutrition et Metabolisme*, 37(3), 540–542.
- Steinhubl, S. R., Muse, E. D., & Topol, E. J. (2015). The emerging field of mobile health. *Science Translational Medicine*, 7(283), 283rv3.
- Time series prediction for the epidemic trends of COVID-19 using the improved LSTM deep learning method: Case studies in Russia, Peru and Iran. (2020). *Chaos, Solitons & Fractals*, 140, 110214.
- van den Oord, A., & Dieleman, S. (2016). *WaveNet: A Generative Model for Raw Audio*.
<http://arxiv.org/abs/1609.03499v2>
- Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, L., & Polosukhin, I. (2017). *Attention Is All You Need*. <http://arxiv.org/abs/1706.03762>
- Wang, R., Chen, F., Chen, Z., Li, T., Harari, G., Tignor, S., Zhou, X., Ben-Zeev, D., & Campbell, A. T. (2014). StudentLife. *Proceedings of the 2014 ACM International Joint Conference on Pervasive and Ubiquitous Computing - UbiComp '14 Adjunct*. <https://doi.org/10.1145/2632048.2632054>
- Wang, Pepei, Xinqi Zheng, Gang Ai, Dongya Liu, and Bangren Zhu. (2020). Time series prediction for the epidemic trends of COVID-19 using the improved LSTM deep learning method: Case studies in Russia, Peru and Iran.
<https://www.sciencedirect.com/science/article/abs/pii/S096007792030610X>.
- Watson, D., Clark, L. A., & Tellegen, A. (1988). Development and validation of brief measures of positive and negative affect: the PANAS scales. *Journal of Personality and Social Psychology*, 54(6). <https://doi.org/10.1037//0022-3514.54.6.1063>
- Wilmot, E. G., Edwardson, C. L., Achana, F. A., Davies, M. J., Gorely, T., Gray, L. J., Khunti, K., Yates, T., & Biddle, S. J. H. (2012). Sedentary time in adults and the association with diabetes, cardiovascular disease and death: systematic review and meta-analysis. *Diabetologia*, 55(11), 2895–2905.

- Wu, C., Boukhechba, M., Cai, L., Barnes, L. E., & Gerber, M. S. (2018). Improving momentary stress measurement and prediction with bluetooth encounter networks. In *Smart Health* (Vols. 9–10, pp. 219–231). <https://doi.org/10.1016/j.smhl.2018.07.017>
- Yan, H., & Ouyang, H. (2017). Financial Time Series Prediction Based on Deep Learning. *Wireless Personal Communications*, 102(2), 683–700.
- Yerrakalva, D., Yerrakalva, D., Hajna, S., & Griffin, S. (2019). Effects of Mobile Health App Interventions on Sedentary Time, Physical Activity, and Fitness in Older Adults: Systematic Review and Meta-Analysis. *Journal of Medical Internet Research*, 21(11). <https://doi.org/10.2196/14343>
- Zia, J., Tadayon, A., McDaniel, T., & Panchanathan, S. (2016). Utilizing Neural Networks to Predict Freezing of Gait in Parkinson's Patients. In *Proceedings of the 18th International ACM SIGACCESS Conference on Computers and Accessibility - ASSETS '16*. <https://doi.org/10.1145/2982142.2982194>